

## 文档目标

- 面向面试场景，沉淀“短而全”的回答大纲。
- 每个问题只保留高频结论、核心原理、回答顺序与追问点。
- 统一成紧凑结构，方便临场快速复习、查漏补缺。

## 回答压缩原则

- 先给结论，再讲原因，最后补场景与边界。
- 优先保留“定义 / 原理 / 场景 / 对比 / 风险”五类信息。
- 单题尽量控制在 30 秒、1 分钟、3 分钟三种展开层级内。

## 统一模板

题目：

- 一句话结论：
- 30秒回答：
- 1分钟展开：
  - 
  - 
  -
- 高频关键词：
- 易追问：
- 回答雷区：

## 01. 什么是 Nginx

- 一句话结论: Nginx 是一个高性能的 HTTP 和反向代理服务器, 特点是高并发、低资源消耗, 常用于静态资源托管、反向代理和负载均衡。
- 30秒回答: Nginx 本质上是一个高性能 Web 服务器。它最常见有三大作用: 第一, 处理静态资源; 第二, 作为反向代理转发请求; 第三, 做负载均衡把流量分发到多台服务。面试里还经常会结合它问跨域, 因为前端开发经常通过 Nginx 配代理, 把浏览器的跨域问题转成同源访问。
- 1分钟展开:
  - 概念与特点: Nginx 采用事件驱动、异步非阻塞模型, 单机能扛较高并发, 请求多时比传统阻塞式模型更省资源, 所以常放在服务最前面做流量入口。
  - 三大作用: 一是静态资源管理, 比如直接返回 HTML、CSS、JS、图片; 二是反向代理, 对外暴露统一入口, 隐藏真实后端; 三是负载均衡, 把请求按策略分发到多台应用服务器, 提高可用性和扩展性。
  - 正向代理 vs 反向代理: 正向代理代理的是客户端, 典型场景是科学上网, 服务端知道请求不是用户本人直接发的; 反向代理代理的是服务端, 客户端只知道访问代理服务器, 不知道真实后端是谁, Nginx 常见就是反向代理。
  - 跨域本质: 跨域本质是浏览器的同源策略限制。协议、域名、端口任意一个不同都算跨域。注意不是“请求发不出去”, 而是“浏览器能发请求, 但会拦截响应结果”; 而且跨域问题只发生在浏览器端, 像服务端调用接口不会受这个限制。
  - 几个高频注意点: `img`、`script`、`link` 这类标签通常可以跨域加载资源, 所以 JSONP 才能利用 `script` 标签实现; WebSocket 不走传统的同源限制逻辑, 但仍然可能受服务端校验或安全策略约束。
  - CORS 解决跨域: 核心是服务端通过响应头告诉浏览器“这个源可以访问我”。记忆方式可以理解为: 浏览器发请求时带 `Origin`, 服务端回响应时声明 `Access-Control-Allow-Origin`, 如果要放行方法、请求头、凭证, 就继续返回对应的 `Allow-Methods`、`Allow-Headers`、`Allow-Credentials`。
  - 简单请求与复杂请求: 简单请求比如常见的 `GET/POST` 且请求头比较“普通”, 浏览器会直接发; 复杂请求比如自定义请求头、`PUT/DELETE`、特殊 `Content-Type`, 浏览器会先发一次 `OPTIONS` 预检请求, 确认服务端是否允许, 再决定是否发真正请求。
  - 其他方案: JSONP 只支持 `GET`, 本质是利用 `script` 标签无跨域限制; 开发阶段也常用代理服务器解决跨域, 比如本地 `devServer`、`Vite`、`Nginx`, 把前端请求先打到同源代理, 再由代理转发到真实后端。

- 高频关键词：高性能、高并发、事件驱动、异步非阻塞、静态资源、反向代理、负载均衡、同源策略、CORS、预检请求、OPTIONS、JSONP、开发代理
- 易追问：为什么 Nginx 能抗高并发？反向代理和负载均衡是什么关系？同源策略具体判断哪三项？简单请求和复杂请求怎么区分？预检请求什么时候触发？JSONP 为什么只能 GET？开发代理和线上 Nginx 代理有什么区别？
- 回答雷区：不要把正向代理和反向代理说反；不要说“跨域请求发不出去”，更准确是“浏览器限制读取响应”；不要说“所有跨域都被禁止”，资源加载、JSONP、WebSocket 都有例外或不同规则；不要把 CORS 说成前端解决方案，本质上是服务端配合浏览器协议。

## 02. 页面路由跳转

- 一句话结论：前端页面路由跳转本质上是 URL 变化与视图渲染之间的映射机制，核心包括 Hash、History、SPA/MPA、SSR/CSR 以及服务端路由兜底。
- 30秒回答：页面路由常见分两类：Hash 和 History。SPA 是单页应用，前端接管路由，根据 URL 变化动态渲染组件；MPA 是多页应用，每次跳转都向服务端请求新页面。SPA 用户体验更流畅，但首屏和 SEO 往往不如 SSR 或传统 MPA，而 History 模式上线时还要配合服务端做 404 回退。
- 1分钟展开：
  - **Hash 与 History**：Hash 模式通过 URL 中 # 后面的内容变化实现路由切换，监听的是 hashchange；优点是兼容性好、服务端配置简单，缺点是 URL 不够美观。History 模式通过 pushState、replaceState 改变地址栏路径，不会带 #，更接近真实路径，但刷新或直接访问子路径时需要服务端配合。
  - **MPA 与 SPA**：MPA 是多页应用，每次跳转都会重新请求 HTML，SEO 友好、首屏通常更直接，但页面切换成本高。SPA 是单页应用，首次加载一个主页面，之后通过 JS 接管路由和组件渲染，交互流畅、前后端分离更清晰，但首屏性能、SEO、白屏时间往往要额外优化。
  - **SPA 底层原理**：本质是“监听 URL 变化 -> 匹配路由规则 -> 找到对应组件 -> 局部更新页面”。Hash 模式通常监听 hashchange；History 模式依赖 pushState、replaceState 和 popstate。路由库内部会维护一套路由表，把路径映射到组件，再交给框架去渲染。
  - **服务端为什么要处理 404**：History 模式下，浏览器输入 /user/detail 会直接请求服务端这个路径。如果服务端没有对应资源，就会返回 404。所以线上部署时通常要把未知路径统一回退到入口页，比如 index.html，再由前端路由接管后续匹配。

- **SSR 与 CSR:** **CSR** 是客户端渲染，服务端先返回一个壳页面和 JS，真正内容由浏览器执行 JS 后生成；**SSR** 是服务端渲染，服务端先把首屏 HTML 直接生成好再返回给浏览器。**CSR** 开发灵活但首屏依赖 JS；**SSR** 首屏更快、SEO 更友好，但服务端压力和开发复杂度更高。
- **SEO 与首屏渲染:** 搜索引擎更容易抓取服务端直接输出的 HTML，所以 **SSR/MPA** 通常在 SEO 上更占优。首屏渲染上，**SSR** 往往优于纯 **CSR**，因为用户更早看到内容；但后续交互和页面切换流畅度，**SPA** 往往体验更好。
- 高频关键词：**Hash**、**History**、**hashchange**、**pushState**、**replaceState**、**popstate**、**SPA**、**MPA**、**SSR**、**CSR**、路由表、404 回退、**index.html**、SEO、首屏渲染
- 易追问：**Hash** 和 **History** 的区别是什么？为什么 **History** 刷新会 404？**pushState** 为什么不会刷新页面？**SPA** 为什么首屏可能更慢？**SSR** 为什么更利于 SEO？**SSR** 和预渲染有什么区别？前端路由和后端路由分别负责什么？
- 回答雷区：不要把 **pushState** 说成会主动触发页面刷新；不要把 **hashchange** 和 **popstate** 混为一谈；不要说 **SPA** 一定比 **MPA** 好，应该结合 SEO、首屏、交互复杂度来谈；不要把 **SSR** 简化成“服务端返回字符串”而忽略首屏直出和同构/水合过程；不要漏掉 **History** 模式必须配服务端兜底这一点。

### 03. HTTPS 攻击方式

- 一句话结论：**HTTPS** 的核心目标是解决明文 **HTTP** 容易被窃听、篡改、冒充的问题，它通过 **TLS** 建立加密通道，依靠哈希保证完整性、非对称加密参与密钥协商、数字证书完成身份认证。
- 30秒回答：常见安全问题里，像数据监听、数据劫持、中间人攻击、证书伪造，和 **HTTPS** 关系最直接；而 **XSS**、**CSRF** 属于 **Web** 安全攻击，不是 **HTTPS** 专属问题，**HTTPS** 不能直接防住。**HTTPS** 本质是在 **HTTP** 外面加了一层 **TLS**，先校证书确认服务器身份，再通过非对称加密或密钥交换算法协商出对称密钥，后续正式通信主要用对称加密，兼顾安全和性能。
- 1分钟展开：
  - 常见攻击方式：一类是网络链路层面的窃听、篡改、劫持，比如数据监听、数据劫持、中间人攻击；另一类是 **Web** 应用层攻击，比如 **XSS**、**CSRF**。要注意，**HTTPS** 主要缓解前一类问题，不能替代应用层安全防护。
  - **HTTPS** 的核心能力：第一是加密，防止明文被偷看；第二是完整性校验，防止传输内容被篡改；第三是身份认证，防止连到了假服务器。这里通常会提到哈希函数、非对称加密、对称加密、数字证书四个关键词。
  - 哈希函数的作用：哈希主要负责完整性校验。发送方对消息做摘要，接收方再校验摘要是否一致，如果不一致说明数据可能被改过。**TLS** 里完整性保护通常结合 **MAC** 或 **AEAD** 模式一起完成。

- **非对称加密的作用**：非对称加密不是拿来大规模传业务数据的，因为性能成本高。它更常用于握手阶段的身份验证和参与密钥协商，比如保护预主密钥，或者配合 **ECDHE** 这类算法安全地产生共享密钥。
- **数字证书的作用**：证书本质上是“服务器公钥 + 身份信息 + CA 签名”的组合。浏览器通过证书链去验证“这个公钥到底是不是这个网站的”，从而避免把密钥协商给了伪造服务器。
- **数字证书校验过程**：浏览器收到证书后，会先看域名是否匹配，再看证书是否在有效期内，然后检查证书链是否能追溯到本地信任的 CA 根证书，还会验证 CA 对证书的签名是否有效，必要时还会检查证书是否被吊销，比如 **CRL** 或 **OCSP**。
- **协商密钥的构建过程**：可以按 TLS 握手来讲。客户端先发 **ClientHello**，带上支持的 TLS 版本、加密套件、随机数；服务端回 **ServerHello**、证书以及自己的随机数；客户端验证证书无误后，与服务端基于密钥交换算法生成共享秘密，比如传统 **RSA** 方式会生成 **pre-master secret** 再用服务端公钥加密，现代更常见是 **ECDHE** 双方各自生成临时密钥对并交换公钥，最终各自算出相同的共享秘密；最后再结合双方随机数派生出会话密钥，后续业务数据就用对称加密传输。
- **为什么这样设计**：因为对称加密快，适合大量数据传输；非对称加密安全但慢，适合解决“如何安全地协商出同一个密钥”；证书则负责证明“你协商的对象到底是谁”。
- **高频关键词**：**HTTPS**、**TLS**、**SSL**、完整性、身份认证、对称加密、非对称加密、哈希、数字证书、CA、证书链、**MITM**、数据监听、数据劫持、**ClientHello**、**ServerHello**、**RSA**、**ECDHE**、会话密钥
- **易追问**：**HTTPS** 为什么比 **HTTP** 安全？为什么业务数据不用非对称加密直接传？数字证书里到底有什么？浏览器怎么判断证书真假？**RSA** 握手和 **ECDHE** 握手有什么区别？什么叫前向安全？**HTTPS** 能不能防 **XSS** 和 **CSRF**？
- **回答雷区**：不要把 **SSL** 和 **TLS** 完全混用，面试里可以说“**HTTPS** 现在主要依赖 **TLS**，**SSL** 是旧叫法”；不要说 **HTTPS** 能防所有攻击，**XSS/CSRF** 仍需单独防护；不要说哈希是用来加密明文的，哈希主要用于完整性校验；不要把证书说成“加密文件”，它更准确是身份认证载体；不要把密钥协商说成“直接传输对称密钥明文”。

## 04. HTTP 缓存机制

- **一句话结论**：**HTTP** 缓存分为强缓存和协商缓存，浏览器会优先看强缓存，命中则直接用本地副本；未命中再走协商缓存，通过服务端确认资源是否变更。

- 30秒回答：强缓存和协商缓存是 HTTP 缓存的两大核心。强缓存主要看 `Cache-Control: max-age` 和 `Expires`，命中时浏览器连请求都不发；协商缓存主要看 `ETag / If-None-Match` 和 `Last-Modified / If-Modified-Since`，浏览器会发请求给服务端确认资源有没有变化，没变就返回 `304`。再往上还有 `Service Worker + Cache Storage`，可以做更灵活的离线缓存和运行时缓存。
- 1分钟展开：
  - 强缓存：浏览器先看资源是否还在强缓存有效期内。常见响应头是 `Cache-Control: max-age=xxx` 和 `Expires`。其中 `max-age` 是相对时间，优先级高于 `Expires`；`Expires` 是绝对时间，受客户端本地时间影响更大。
  - 协商缓存：如果强缓存失效，就进入协商缓存。服务端如果返回过 `ETag`，浏览器下次会带上 `If-None-Match`；如果服务端返回过 `Last-Modified`，浏览器下次会带上 `If-Modified-Since`。服务端判断资源没变就返回 `304 Not Modified`，有变则返回新资源和新的缓存头。
  - 优先顺序：面试里可以概括为：先 `Cache-Control: max-age`，再 `Expires`，强缓存不命中后，再走 `ETag / If-None-Match`，最后才是 `Last-Modified / If-Modified-Since`。通常 `ETag` 精度更高，因为 `Last-Modified` 只能反映时间，不一定能准确反映内容变化。
  - 常见缓存策略：`no-store` 表示完全不缓存；`no-cache` 不是不缓存，而是“可以缓存但每次使用前必须向服务端验证”；`private` 表示只允许浏览器私有缓存，不允许共享缓存如 CDN 缓存；相对的 `public` 表示可以被共享缓存存储。
  - 为什么需要协商缓存：因为有些资源不能长期强缓存，但也不想每次都完整下载，所以通过 `304` 机制减少带宽消耗，既保证数据相对新鲜，又提升性能。
  - 进一步：**Service Worker 缓存**：这是应用层缓存，不属于传统 HTTP 头控制范畴。它依赖 `Cache Storage` API，可以把请求和响应主动存下来，实现离线访问、预缓存、运行时缓存、缓存优先/网络优先等策略，常见于 PWA。
  - **Service Worker 生命周期**：`install` 阶段通常做预缓存；`activate` 阶段负责清理旧缓存、接管页面；`fetch` 阶段拦截请求，根据策略决定是走缓存还是走网络，并可把新资源写回缓存。
  - **Service Worker vs Web Worker**：二者都是运行在主线程外的后台脚本，但定位不同。`web worker` 主要解决耗时计算，不能直接操作 DOM；`Service worker` 主要做代理和缓存，能拦截网络请求、做离线能力，但同样不能直接操作 DOM。

- 浏览器常见存储：`Cookie` 体积小，会随请求自动发送，常用于身份会话；`Web Storage` 包括 `localStorage` 和 `sessionStorage`，API 简单但容量有限且同步阻塞；`IndexedDB` 是浏览器端结构化数据库，适合复杂大数据存储；`Cache Storage` 更偏资源缓存，常与 `Service Worker` 配合使用。
- 高频关键词：强缓存、协商缓存、`Cache-Control`、`max-age`、`Expires`、`ETag`、`If-None-Match`、`Last-Modified`、`If-Modified-Since`、`304`、`no-cache`、`no-store`、`private`、`Service Worker`、`Cache Storage`、预缓存、运行时缓存、`Web Worker`、`Cookie`、`localStorage`、`sessionStorage`、`IndexedDB`
- 易追问：强缓存和协商缓存的区别是什么？为什么 `max-age` 优先于 `Expires`？`ETag` 和 `Last-Modified` 各有什么优缺点？`no-cache` 和 `no-store` 有什么区别？`304` 什么时候返回？`Service Worker` 能做什么？`Service Worker` 和 `Web Worker` 的区别是什么？浏览器存储怎么选型？
- 回答雷区：不要把 `no-cache` 说成“不缓存”，它更准确是“缓存前必须再验证”；不要把 `ETag` 和 `If-None-Match`、`Last-Modified` 和 `If-Modified-Since` 的请求头/响应头方向说反；不要说强缓存命中还会请求服务端，命中强缓存时通常不会发请求；不要把 `Service Worker` 写成 `Server Worker`；不要把 `Cache Storage` 和传统 HTTP 缓存混为一谈，它是更上层、可编程的缓存能力。

## 05. 两栏布局并实现垂直居中的方法

- 一句话结论：两栏布局常见用 `flex`、`grid`、定位等方式实现；垂直居中最常见用 `flex/grid` 对齐，或绝对定位配合 `transform`。
- 30秒回答：两栏布局如果是一维分栏，最推荐 `flex`；如果同时要控制行列关系、跨行跨列、响应式卡片布局，更适合 `grid`。垂直居中最简单的是父元素用 `display: flex; align-items: center;`，或者 `grid` 的对齐属性；老方案也可以用绝对定位加 `top: 50%` 和 `transform: translateY(-50%)`。
- 1分钟展开：
  - **Flex** 做两栏布局：`flex` 更适合一维布局，比如左侧固定宽度、右侧自适应。常见写法是左边 `width: 240px`，右边 `flex: 1`。其中 `flex: 1` 可以理解为 `flex-grow: 1; flex-shrink: 1; flex-basis: 0%`，表示可扩展、可收缩、初始基准为 0，剩余空间都给它。
  - **Grid** 做两栏布局：`grid` 更适合二维布局，可以同时控制列和行。两栏常见写法是 `grid-template-columns: 240px 1fr`，如果要更灵活，可以用 `repeat()`、`minmax()`、`1fr` 这些能力，比如自适应卡片流式布局。

- **Grid 的对齐能力：**`grid` 不仅能控制列、行划分，还能做容器整体对齐、单元格内容对齐、子元素自身对齐，所以在复杂布局里比 `flex` 更灵活。面试可概括成：`flex` 偏一维，`grid` 偏二维。
- **响应式布局：**如果页面要在不同宽度下自动换列，`grid` 很适合，例如 `repeat(auto-fit, minmax(200px, 1fr))`。这样列数会根据容器宽度自动变化，还能配合 `gap` 控制间距，配合跨行跨列实现复杂响应式布局。
- **垂直居中的主流做法：**最推荐 `flex`：父元素 `display: flex; align-items: center;`；如果还要水平居中，可以加 `justify-content: center`。`grid` 也类似，可以用 `align-items`、`justify-items`，或者 `place-items: center` 一次性设置。
- **绝对定位 + transform：**如果元素脱离文档流，比如弹窗、浮层、角标场景，也常用 `position: absolute; top: 50%; transform: translateY(-50%);`。如果要完全居中，就再加 `left: 50%; transform: translate(-50%, -50%);`。
- **常见定位方式：**`relative` 常作为绝对定位参照物；`absolute` 相对最近的定位祖先定位；`fixed` 相对视口定位，常用于吸顶/悬浮按钮；`sticky` 是粘性定位，在滚动到阈值前表现像普通流，达到阈值后表现像吸附。
- **Transform 常见能力：**`transform` 常见包括 `translate` 位移、`rotate` 旋转、`scale` 缩放。面试里可以顺带提：它通常不会影响文档流，适合做位移和动画，也常用于居中和过渡效果。
- **高频关键词：**两栏布局、`flex`、`flex: 1`、`flex-grow`、`flex-shrink`、`flex-basis`、`grid`、`grid-template-columns`、`1fr`、`repeat()`、`minmax()`、`auto-fit`、`gap`、垂直居中、`align-items`、`justify-content`、`place-items`、`position`、`absolute`、`relative`、`fixed`、`sticky`、`transform`、`translate`、`rotate`、`scale`
- **易追问：**为什么 `flex` 适合一维布局、`grid` 适合二维布局？`flex: 1` 到底展开后是什么？左侧固定右侧自适应用什么最合适？`grid-template-columns: 1fr 2fr` 是什么意思？`auto-fit` 和 `auto-fill` 区别是什么？为什么 `top: 50%` 还要配 `translateY(-50%)`？`sticky` 和 `fixed` 有什么区别？
- **回答雷区：**不要把 `flex: 1` 简单说成“宽度 100%”，它本质是弹性分配规则；不要把 `grid` 只当成“高级版 `flex`”，它核心优势是二维控制；不要说 `top: 50%` 就已经垂直居中，元素本身高度还会导致偏移，所以通常要配 `translateY(-50%)`；不要把 `absolute` 说成相对父元素定位，更准确是相对最近的非 `static` 定位祖先。

## 06. 异步函数有哪些

- 一句话结论：前端常见异步方案包括回调函数、`Promise`、`async/await`、`Generator` 配执行器，以及基于事件监听的异步处理，其中 `Promise` 和 `async/await` 是现代开发主流。
- 30秒回答：最早的异步写法是回调函数，本质是把函数作为参数传进去，等时机到了再执行；后来有了 `Promise`，把异步操作抽象成一个对象，支持状态管理和链式调用；`async/await` 又把 `Promise` 写法进一步同步化；`Generator` 配合执行器也能实现类似 `await` 的效果。除此之外，前端里很多异步其实也来自事件监听，比如点击、滚动、网络响应这些都是异步触发。
- 1分钟展开：
  - 回调函数：这是最基础的异步方案，本质是把函数当参数传给另一个函数，等任务完成后回调执行。优点是直接，缺点是多层嵌套时容易形成回调地狱，错误处理和流程控制也不够统一。
  - `Promise` 是什么：`Promise` 是对异步结果的对象化封装。它有 `pending`、`fulfilled`、`rejected` 三种状态，而且状态一旦从 `pending` 变成成功或失败后就不可逆。它的核心价值是把“未来才会得到的结果”先包装起来，让后续逻辑可以通过 `then/catch/finally` 统一组织。
  - `thenable` 与 `Promise A+`：`thenable` 指的是“只要对象上有 `then` 方法，就能被当成类似 `Promise` 的对象来处理”。`Promise A+` 更多是约束 `then` 行为，比如 `then` 接收成功和失败两个回调；回调返回普通值，会包装成成功 `Promise`；返回 `Promise` 或 `thenable`，会继续等待其结果；抛异常会变成失败态。正因为这样，`Promise` 才能做链式调用和状态传递。
  - `Promise` 链式调用：每次调用 `then` 都会返回一个新的 `Promise`，所以链式调用本质是“上一个结果决定下一个 `Promise` 的状态和值”。这也是它优于普通回调的关键：流程更线性，错误可冒泡，状态有统一存储。
  - `async / await`：`async` 函数本质上是 `Promise` 的语法糖，调用后一定返回一个 `Promise`。`await` 后面可以是普通值、同步表达式、`Promise`、`thenable`；如果是普通值会直接返回，如果是 `Promise/thenable` 就会等待其完成后继续向下执行，所以代码看起来更像同步流程。
  - `Generator`：`Generator` 函数执行后不会一次跑完，而是返回一个迭代器对象，通过 `next()` 一步步恢复执行。`yield` 相当于“暂停点”，把控制权交出去。如果配合一个执行器，就可以在每次 `yield Promise` 后等待完成，再自动 `next()`，这也是早期 `co` 库和 `async/await` 思想的重要来源。

- **Promise 静态方法**：常见的有 `Promise.resolve`、`Promise.reject`、`Promise.all`、`Promise.race`、`Promise.allSettled`、`Promise.any`。面试里可以说：`all` 关心全部成功，`race` 谁先完成用谁，`allSettled` 不关心成功失败都收集结果，`any` 只要有一个成功就返回。
- **事件监听也是异步来源**：浏览器里的点击、输入、滚动、网络请求完成，本质上都是事件驱动异步。DOM 事件传播分为捕获阶段、目标阶段、冒泡阶段。而事件委托就是把子元素事件统一交给父元素处理，利用冒泡减少监听器数量，也更适合处理动态节点。
- **高频关键词**：异步、回调函数、回调地狱、`Promise`、`pending`、`fulfilled`、`rejected`、状态不可逆、`thenable`、`Promise A+`、链式调用、`then`、`catch`、`finally`、`async`、`await`、`Generator`、`yield`、`next()`、执行器、`Promise.all`、`Promise.race`、`Promise.allSettled`、`Promise.any`、事件监听、捕获、目标、冒泡、事件委托
- **易追问**：回调函数和 `Promise` 的区别是什么？为什么 `then` 可以链式调用？`thenable` 为什么也能被 `await`？`await` 后面到底能接什么？`async` 为什么一定返回 `Promise`？`Generator` 为什么能模拟 `await`？`Promise.all` 和 `allSettled` 区别是什么？事件委托为什么能减少性能开销？捕获和冒泡分别是什么？
- **回答雷区**：不要把 `Promise` 说成“同步拿到异步结果”，它只是对异步结果的封装；不要把 `thenable` 等同于原生 `Promise`，它只是“像 `Promise` 的对象”；不要说 `await` 只能等 `Promise`，普通值和 `thenable` 也可以；不要把 `Generator` 说成异步函数本身，它本质是可暂停函数，异步能力来自执行器配合；不要把事件传播顺序说错，标准顺序是捕获 -> 目标 -> 冒泡。

## 07. call / apply / bind

- **一句话结论**：`call`、`apply`、`bind` 本质都是显式绑定 `this` 的方法，区别在于调用时机和参数传递方式；而理解它们的前提是先搞清楚 `this` 的绑定规则。
- **30秒回答**：`this` 不是函数声明时决定的，而是函数调用时决定的。常见有四种绑定方式：默认绑定、隐式绑定、显式绑定、`new` 绑定，另外箭头函数没有自己的 `this`，它会继承外层作用域的 `this`。`call` 和 `apply` 都是立即执行，只是一个接收参数列表，一个接收参数数组；`bind` 不会立刻执行，而是返回一个新函数，并且这个新函数的 `this` 后续通常不会再变，除非被 `new` 调用。
- **1分钟展开**：
  - **this 是什么**：`this` 代表函数运行时的调用上下文，不是定义时写死的“所属对象”。同一个函数在不同调用方式下，`this` 可能完全不同，所以面试里要强调“`this` 取决于调用方式”。

- 四种绑定方式：一是默认绑定，比如普通函数直接调用，非严格模式下通常指向全局对象，严格模式下是 `undefined`；二是隐式绑定，比如 `obj.fn()`，`this` 指向调用它的对象；三是显式绑定，就是 `call/apply/bind` 人为指定 `this`；四是 `new` 绑定，构造调用时 `this` 指向新创建的实例对象。
- 箭头函数的 `this`：箭头函数没有自己的 `this`，不会因为 `call/apply/bind/new` 改变，它会捕获定义时外层作用域的 `this`。所以箭头函数适合回调里保持上下文，但不适合作为构造函数。
- `call/apply/bind` 差异：`call(thisArg, a, b, c)` 是立即执行，参数逐个传；`apply(thisArg, [a, b, c])` 也是立即执行，但参数是数组或类数组；`bind(thisArg, a, b)` 不立即执行，而是返回一个绑定好 `this` 和部分参数的新函数，适合柯里化和事件回调场景。
- `bind` 的关键特点：`bind` 返回的新函数，正常调用时 `this` 基本固定为绑定对象；但如果这个新函数被 `new` 调用，那么优先级更高的是 `new` 绑定，此时 `this` 会指向新实例，而不是之前绑定的对象。
- 手写 `call` 思路：核心就是临时把目标函数挂到传入对象上，通过对象方法调用让 `this` 指向该对象，执行后再删除这个临时属性，并返回执行结果。要注意处理 `null/undefined` 时回退到全局对象，以及参数展开。
- 手写 `apply` 思路：和 `call` 基本一致，差别只是参数来自数组或类数组，需要在执行时做展开处理。
- 手写 `bind` 思路：本质是返回一个新函数，这个函数内部再去调用原函数，并把预置参数和调用时参数拼起来；同时要处理构造调用场景，也就是如果返回函数被 `new` 调用，`this` 应该指向新实例，并且原型链最好也要继承原函数原型。
- 高频关键词：`this`、默认绑定、隐式绑定、显式绑定、`new` 绑定、箭头函数、词法 `this`、`call`、`apply`、`bind`、立即执行、返回新函数、参数列表、参数数组、偏函数、柯里化、构造调用、原型链
- 易追问：`this` 到底在定义时还是调用时确定？箭头函数为什么不能改 `this`？`call` 和 `apply` 实际使用场景有什么区别？`bind` 和箭头函数固定 `this` 有什么不同？为什么 `bind` 后再 `new`，`this` 还是会变？手写 `bind` 为什么要处理原型链？`call/apply` 为什么能改变 `this`？
- 回答雷区：不要说 `this` 永远指向函数所属对象，这个说法不准确；不要漏掉箭头函数没有自己 `this` 这一点；不要说 `bind` 会立即执行，它返回的是新函数；不要说 `bind` 绑定后 `this` 绝对不会变，`new` 调用是例外；不要把 `call` 和 `apply` 区别说成“性能不同”，面试核心还是参数形式和使用语义。

## 08. 介绍一下 flex

- 一句话结论: **flex** 是弹性盒子布局, 是替代部分浮动和传统定位的一维布局方案, 适合处理一行或一列上的分配、对齐和响应式伸缩; 而 **grid** 更适合二维网格布局。
- 30秒回答: **flex** 的核心是“容器”和“项目”两个概念, 设置了 **display: flex** 的元素叫 **flex** 容器, 里面的直接子元素都会变成 **flex** 项目。它围绕主轴和交叉轴来分配空间, 常用于水平排列、垂直居中、两栏布局、自适应布局。常见容器属性有方向、换行、对齐、间距, 项目属性有 **flex-grow**、**flex-shrink**、**flex-basis**、**align-self**。如果布局从一维扩展到二维, 通常就会考虑 **grid**。
- 1分钟展开:
  - **flex** 的概念: **flex** 是弹性盒子布局, 核心目的是让子元素在不同空间下可以灵活伸缩和对齐。相比早期的浮动布局, **flex** 写法更直观, 尤其适合导航栏、列表、工具栏、两栏布局、居中布局这类一维场景。
  - 核心概念: 使用 **display: flex** 后, 元素变成 **flex** 容器, 内部直接子元素都变成 **flex** 项目。布局围绕两条轴展开: 主轴和交叉轴。**flex-direction** 决定主轴方向, 比如 **row** 是横向, **column** 是纵向; 另一条垂直于主轴的就是交叉轴。
  - 容器属性: 常见有 **justify-content** 控制主轴对齐, **align-items** 控制交叉轴上项目整体对齐, **align-content** 控制多行内容整体分布, **gap** 控制项目间距。除此之外还有 **flex-wrap** 控制是否换行, **flex-flow** 是方向和换行的简写。
  - 项目属性: **align-self** 可以覆盖单个项目自己的对齐方式; **flex-grow** 决定剩余空间如何扩张, **flex-shrink** 决定空间不足时如何收缩, **flex-basis** 决定分配空间前的基准尺寸。面试里常说的 **flex: 1**, 本质是 **flex-grow: 1; flex-shrink: 1; flex-basis: 0%**。
  - 对齐和响应式: **flex** 在居中和响应式上很好用, 比如用 **justify-content: center** 和 **align-items: center** 做水平垂直居中; 用 **flex-wrap** 配合弹性宽度实现换行卡片; 用 **flex: 1** 让内容区自动占满剩余空间。
  - 进一步对比 **grid**: **grid** 是网格布局, 更适合二维布局。**flex** 更像一条轴上分配空间, **grid** 则能同时控制行和列。它的核心概念是网格容器和网格单元格。
  - **grid** 容器属性: 常见有 **grid-template-columns**、**grid-template-rows** 定义列和行; **repeat()**、**minmax()**、**auto-fit** 适合响应式自动排布; **gap** 控制间距; **justify-content**、**align-content** 控制整个网格分布; **justify-items**、**align-items** 控制单元格内容对齐, 也可以用 **place-items** 简写。

- **grid 项目属性**：单个网格项可以用 `align-self`、`justify-self` 单独控制自身对齐，也可以通过 `grid-column`、`grid-row` 控制跨列跨行，所以复杂后台布局、卡片宫格、仪表盘页面更适合 `grid`。
- 高频关键词：`flex`、弹性盒子、一维布局、主轴、交叉轴、`flex-direction`、容器、项目、`justify-content`、`align-items`、`align-content`、`gap`、`flex-grow`、`flex-shrink`、`flex-basis`、`align-self`、`flex: 1`、响应式、`grid`、二维布局、`grid-template-columns`、`grid-template-rows`、`repeat()`、`minmax()`、`auto-fit`、`justify-items`、`place-items`、`grid-column`、`grid-row`
- 易追问：为什么说 `flex` 是一维布局？`justify-content` 和 `align-items` 区别是什么？`align-items` 和 `align-content` 区别是什么？`flex: 1` 的真实含义是什么？什么时候该用 `flex`，什么时候该用 `grid`？`auto-fit` 和 `minmax()` 怎么配合做响应式？`grid-column` 和 `grid-row` 怎么实现跨列跨行？
- 回答雷区：不要把 `flex` 说成完全替代所有布局方案，它更适合一维场景；不要把 `justify-content` 和 `align-items` 的轴向说反；不要把 `align-content` 和 `align-items` 混淆，前者更偏多行整体分布；不要把 `grid` 只理解成“能换行的 `flex`”，它本质是二维网格系统；不要漏掉“只有直接子元素才是 `flex` 项目”这一点。

## 09. 前端设计模式有哪些

- 一句话结论：前端常见设计模式包括工厂模式、单例模式、观察者/发布订阅模式、原型模式、代理模式、迭代器模式，它们本质上是在解决“创建对象、组织通信、复用能力、控制访问、统一遍历”这些问题。
- 30秒回答：前端里高频设计模式有几类：工厂模式负责封装对象创建；单例模式保证全局只有一个实例；观察者或发布订阅模式负责一对多通知；原型模式基于原型复用和继承；代理模式在不改原对象的前提下做一层控制和增强；迭代器模式则给不同数据结构提供统一遍历方式。面试里最好不要只背定义，要顺带举出前端里的实际例子。
- 1分钟展开：
  - **工厂模式**：核心是“把对象创建逻辑封装起来”。外部不直接 `new` 一堆细节，而是通过工厂函数统一生成对象或实例。这样可以隐藏创建过程、降低耦合。前端里像 `createApp()`、组件工厂、请求实例工厂都能看作工厂模式。
  - **单例模式**：核心是“一个类或模块在系统中只有一个实例”。常见方式是通过闭包或静态方法控制实例只创建一次。前端里全局状态管理、全局弹窗管理器、全局事件总线、某些插件实例都可能采用单例思想，比如全局唯一的 `store`。

- **观察者 / 发布订阅模式**：本质是一对多依赖关系，一个对象状态变化后通知多个依赖方。前端事件监听、响应式依赖收集、状态变化触发视图更新，都能从这个角度理解。严格区分的话，观察者模式是发布者直接持有观察者；发布订阅模式通常通过事件中心解耦，发布者和订阅者彼此不知道对方，只通过中间通道通信。
- **原型模式**：核心是基于已有对象或原型来创建新对象，避免重复定义公共能力。JavaScript 天然基于原型机制，`Object.create()` 就是典型例子。对象继承本质上也是让子类原型链指向父类原型，从而复用父类方法。
- **代理模式**：核心是“给目标对象加一层代理”，在访问前后织入额外逻辑，而不直接改目标对象本身。前端里常见于 `Proxy` 数据拦截、图片懒加载、缓存代理、请求代理、权限控制代理等。比如缓存代理会先查缓存，命中后直接返回，未命中再访问真实对象。
- **迭代器模式**：核心是给不同数据结构提供统一遍历接口，让使用者不用关心内部实现。前端里数组迭代器、`for...of`、`Iterator`、生成器都和这个思想有关。它解决的是“怎么以一致方式遍历集合”的问题。
- **怎么回答更像前端工程视角**：面试里可以补一句，设计模式不是死记硬背，而是为了让代码更可维护、更低耦合、更易扩展。前端框架内部其实大量使用这些模式，比如响应式系统里的观察者思想、`Proxy` 代理、工厂式 API 创建、单例式全局状态管理。
- **高频关键词**：设计模式、工厂模式、对象创建、单例模式、全局唯一、观察者模式、发布订阅、事件中心、依赖收集、触发更新、原型模式、`Object.create()`、原型链、代理模式、`Proxy`、缓存代理、迭代器模式、统一遍历接口、`Iterator`、`for...of`
- **易追问**：工厂模式和构造函数有什么区别？单例模式怎么手写？观察者模式和发布订阅模式区别是什么？Vue 响应式更像观察者还是发布订阅？`Object.create()` 做了什么？`Proxy` 为什么算代理模式？缓存代理解决了什么问题？迭代器模式为什么有意义？
- **回答雷区**：不要把观察者模式和发布订阅模式完全混为一谈，面试里最好顺口区分一下是否有事件中心；不要把原型模式只理解成“继承”，它更底层是基于已有原型创建对象；不要把单例模式理解成“只能 new 一次”，关键是“系统中只暴露一个实例”；不要只背概念不举前端例子，面试官更关注你是否能落到框架和业务场景。

## 10. Vue 和 React 的区别

- **一句话结论**：Vue 更偏渐进式框架，开箱能力更完整；React 更偏 UI 库，核心只聚焦视图层，通常需要结合生态补齐路由、状态管理等能力。

- 30秒回答：Vue 和 React 都是主流前端视图库，但定位和设计思路不同。Vue 是渐进式框架，提供模板、响应式、指令、官方路由和状态管理等更完整的方案；React 更像 UI 库，只关注组件和视图更新，很多能力依赖社区生态补充。语法上 Vue 更接近模板开发，React 更强调 JSX 和 JavaScript 驱动 UI；更新机制上 Vue 更偏依赖追踪，React 更偏状态变化触发组件重新执行再比对更新。
- 1分钟展开：
  - 框架 vs 库：Vue 一般被称为渐进式框架，因为它不仅有核心视图层，还提供了官方生态，比如 `Vue Router`、`Pinia`，整体更像一套完整方案。React 更常被称为 UI 库，它主要解决“如何声明式构建界面”，路由、状态管理、数据请求等通常要结合社区方案，比如 `React Router`、状态库等。
  - 模板语法差异：Vue 常用 `template` 模板语法，写法更接近传统 HTML，指令系统也比较直观，比如 `v-if`、`v-for`、`v-model`。React 主要使用 `JSX`，本质是 JavaScript 里写 UI 结构，所以它更强调“UI 就是状态映射出来的结果”，逻辑和视图结合得更紧。
  - 响应式思路不同：Vue 有更强的响应式系统，会对数据做依赖追踪，哪个数据被哪个组件或副作用使用过，更新时就更精确地通知对应依赖，所以常被概括为“数据变化 -> 依赖追踪 -> 精准更新”。React 则更强调状态驱动渲染，状态变化后组件函数重新执行，生成新的虚拟树，再通过 `diff` 找出最小更新。
  - 开发心智差异：Vue 更像“模板 + 响应式数据 + 指令”，上手相对平滑，适合从传统前端模板思维迁移。React 更像“函数式 UI + 状态 + JSX”，更强调 JavaScript 表达能力、组合能力和函数式思维。
  - 组件通信和状态管理：Vue 官方方案会更统一一些，比如父子通信、`emit`、`provide/inject`、官方状态管理。React 也能做同样的事情，但通常更依赖 `props`、`context` 和第三方生态组合。面试里可以说 Vue 生态更“官方化”，React 生态更“社区化”。
  - 性能和更新角度：两者底层都会尽量减少真实 DOM 操作，性能优化思路也类似，比如减少不必要更新、做缓存和拆分。区别在于 Vue 更偏细粒度依赖追踪，React 更偏组件级重渲染加调和机制，所以 React 面试里常会继续追问函数组件重渲染、`memo`、`useMemo`、`useCallback`。
  - 怎么总结更稳：如果面试官问区别，不建议说谁更好，而是说它们的设计哲学不同：Vue 偏完整方案、模板友好、上手平滑；React 偏 UI 抽象、组合灵活、生态自由度高。
- 高频关键词：Vue、React、渐进式框架、UI 库、官方生态、社区生态、`template`、`JSX`、声明式 UI、响应式、依赖追踪、状态驱动渲染、组件重渲染、虚拟 DOM、`diff`、MVVM、组合能力

- 易追问：为什么 Vue 叫渐进式框架？为什么 React 更像库不是框架？`template` 和 `JSX` 各有什么优缺点？Vue 的响应式和 React 的状态驱动有什么本质差别？为什么 React 组件会频繁重新执行？Vue 和 React 的性能优化思路分别是什么？团队选型一般看什么？
- 回答雷区：不要简单说 Vue 用模板、React 用 JSX 就结束，面试官更看重更新机制和设计理念；不要把 React 说成“没有响应式”，更准确是它的响应式粒度和实现路径不同；不要把 Vue 说成完全自动最优更新、React 一定更慢，这种绝对化表述不准确；不要用“框架比库高级”这类价值判断，重点是职责边界不同。

## 11. WebSocket 心跳机制的实现与作用

- 一句话结论：WebSocket 是一种基于 TCP 的全双工、持久化长连接通信协议，心跳机制的核心作用是检测连接是否“假活”、及时断开失效连接并触发重连，保证实时通信稳定性。
- 30秒回答：WebSocket 解决的是 HTTP 请求-响应模式下服务端不能主动高效推送的问题。它建立连接后，客户端和服务端都可以主动发消息，适合聊天、实时通知、协同编辑等场景。心跳机制通常是在连接建立后，客户端定时发 `ping`，服务端回 `pong`，如果超时未响应，就认为连接已经失效，主动关闭并触发重连，避免连接表面还在、实际上已经不可用。
- 1分钟展开：
  - 什么是 **WebSocket**：WebSocket 是一种全双工通信协议，连接建立后，客户端和服务端都能主动向对方发送消息。它是持久化长连接，不像 HTTP 那样每次请求都要走一轮请求-响应，所以特别适合实时通信场景。
  - 和轮询方案的演进关系：传统实时方案通常经历 `HTTP 轮询` -> `长轮询` -> `SSE` -> `WebSocket`。普通轮询是客户端定时问；长轮询是服务端先挂起，等有结果再返回；`SSE` 支持服务端单向推送给客户端；**WebSocket** 则支持客户端和服务端双向实时通信，效率和实时性更好。
  - **WebSocket 生命周期**：浏览器里 `readyState` 常见有 4 个值：`0` 表示连接中，`1` 表示已连接，`2` 表示关闭中，`3` 表示已关闭。面试里可以顺带说常见事件有 `open`、`message`、`error`、`close`。
  - 建立连接的大致流程：通常先走 HTTP 握手，请求头里带 `Upgrade: websocket` 等字段，把协议从 HTTP 升级为 **WebSocket**。连接建立后再进入业务阶段，包括鉴权、心跳、消息通信、异常处理、关闭连接等。
  - 鉴权怎么做：鉴权的目标是让服务端确认客户端身份。常见做法有两种：一是在 HTTP 握手阶段就通过 `Cookie`、查询参数、请求头、`JWT` 等信息完成认证；二是连接成功后，客户端立即发送一条鉴权消息，服务端校验通过后再允许正式业务通信。实际项目里常配合 `JWT` 做身份认证和权限控制。

- **心跳机制的作用**：心跳的核心不是传业务数据，而是确认链路是否还真实可用。因为有时网络断开、代理超时、服务端异常后，连接可能表现成“看起来没断”，但其实已经收不到消息了，这就叫假活。心跳就是定时探测连接活性，避免长期持有无效连接。
- **心跳机制怎么实现**：常见做法是客户端定时发送 `ping` 或业务层心跳包，服务端收到后返回 `pong` 或心跳响应；客户端记录最后一次收到响应的时间，如果超过设定阈值还没收到，就主动关闭连接并进入重连流程。服务端也可以反向做活性检测，长期收不到客户端心跳就清理连接。
- **超时重连与指数退避**：连接断开后不能立刻无限重连，否则容易造成雪崩或频繁打爆服务端。通常会做重连次数限制和指数退避，比如第一次 1 秒，第二次 2 秒，第三次 4 秒，并设置上限。这样既能尽快恢复，又能避免异常时过度重试。
- **错误码和关闭码处理**：在 `close` 或 `error` 阶段要根据状态码决定后续动作。比如 `1001` 常表示对端离开或页面跳转，通常可以按业务决定是否重连；`1008` 常表示策略违规，可能和鉴权、权限、协议格式有关，这种情况一般不能盲目重连；如果是自定义鉴权失效，比如 `JWT` 过期，更合理的做法是先刷新 `token` 或重新登录，再决定是否重连。
- **高频关键词**：`WebSocket`、全双工、长连接、持久化连接、`HTTP` 升级协议、轮询、长轮询、`SSE`、`readyState`、`open`、`message`、`error`、`close`、鉴权、`JWT`、心跳、`ping`、`pong`、假活、超时断开、自动重连、指数退避、关闭码、`1001`、`1008`
- **易追问**：`WebSocket` 和 `HTTP` 有什么本质区别？为什么 `WebSocket` 还要先走 `HTTP` 握手？轮询、长轮询、`SSE`、`WebSocket` 分别适合什么场景？心跳为什么必须做？浏览器原生 `WebSocket` 能直接发真正的 `ping/pong` 吗？为什么要指数退避？哪些错误适合重连，哪些不适合？`JWT` 过期后 `WebSocket` 怎么处理？
- **回答雷区**：不要把 `WebSocket` 说成“完全不基于 `HTTP`”，更准确是“通过 `HTTP` 握手升级协议”；不要把心跳理解成只为了保活，面试更准确的说法是“检测假活和失效连接”；不要说所有断线都立即重连，鉴权失败、权限问题、协议错误通常要先处理根因；不要把 `error` 和 `close` 混为一谈，很多关闭原因要结合关闭码判断；不要把 `SSE` 说成双向通信，它本质是服务端到客户端的单向推送。

## 12. CDN 和 Service Worker

- **一句话结论**：`CDN` 和 `Service worker` 都能提升资源访问性能，但层级不同：`CDN` 发生在网络分发层，靠边缘节点就近分发内容；`Service worker` 发生在浏览器应用层，靠脚本拦截请求和 `cache storage` 做可编程缓存。

- 30秒回答：**CDN** 是内容分发网络，本质是把静态资源缓存到离用户更近的边缘服务器上，减少回源距离，提升访问速度，也能降低源站压力。**Service worker** 则是浏览器后台独立脚本，它能拦截请求、读写 **Cache Storage**、做预缓存和运行时缓存，是 **PWA** 的核心能力之一。简单说，**CDN** 解决“资源离用户太远”，**Service worker** 解决“浏览器如何更智能地拿和管资源”。
- 1分钟展开：
  - 什么是 **CDN**：**CDN** 全称内容分发网络，它会把图片、JS、CSS、视频等静态资源分发到多个边缘节点。用户访问时，优先从离自己最近的节点获取内容，而不是每次都直接访问源服务器，所以延迟更低、速度更快。
  - **CDN** 的作用：第一是加速访问，减少网络传输距离；第二是缓存热点资源，减轻源服务器压力；第三是提升可用性和抗流量能力，大流量请求能由边缘节点分摊。前端项目里常把静态资源、上传文件、媒体文件放到 **CDN**。
  - 缓存链路怎么理解：用户请求资源时，通常会先看浏览器缓存；浏览器没有命中，再可能命中 **CDN** 缓存；再往后可能还有反向代理或 **Web** 服务器缓存；最终才回源到真正的源服务器。可以理解成一层层向后兜底，尽量让请求不要打到最重的那一层。
  - 什么是 **Service Worker**：**Service worker** 是运行在浏览器主线程之外的独立脚本，不能直接操作 **DOM**，但能拦截网络请求、管理缓存、处理离线能力。它是 **PWA** 的核心基础设施之一。
  - **Service Worker** 生命周期：**install** 阶段通常做预缓存，把关键静态资源提前放进 **Cache Storage**；**activate** 阶段负责清理旧缓存、接管页面；**fetch** 阶段拦截请求，根据缓存优先、网络优先、**stale-while-revalidate** 等策略决定如何返回响应。
  - **Cache Storage** 存什么：它本质上存的是“请求 - 响应报文”的映射，不只是单纯存一个文件字符串。也正因为这样，**Service worker** 能按请求维度精细控制缓存逻辑。
  - **Service Worker** 除了缓存还能做什么：除了离线缓存和资源加速，它还可以拦截并修改请求、统一注入请求头、做降级兜底页面、消息推送配合、后台同步等，所以它不只是“浏览器缓存工具”，而是浏览器侧网络代理能力。
  - **CDN** 和 **Service Worker** 的区别：**CDN** 更偏服务器分发层，由基础设施决定资源如何就近返回；**Service worker** 更偏客户端运行时控制，由前端脚本决定请求怎么处理。二者不是替代关系，而是可以叠加：先由浏览器和 **Service worker** 决定是否本地命中，再走网络链路时由 **CDN** 决定是否边缘命中。
- 高频关键词：**CDN**、内容分发网络、边缘节点、就近访问、回源、浏览器缓存、**CDN** 缓存、代理服务器缓存、源服务器、**Service worker**、独立脚本、**install**、**activate**、**fetch**、预缓存、运行时缓存、请求拦截、**Cache Storage**、**PWA**、离线能力

- 易追问：CDN 和浏览器缓存有什么区别？CDN 为什么能加速？什么叫回源？Service Worker 为什么不能操作 DOM？install、activate、fetch 分别做什么？Cache Storage 和 HTTP 缓存有什么区别？Service Worker 除了缓存还有哪些能力？CDN 和 Service Worker 会不会冲突？
- 回答雷区：不要把 CDN 说成浏览器缓存，它属于网络分发层；不要把 service worker 写成 Server worker；不要说 Service worker 只会缓存，它还可以拦截和改写请求；不要把 Cache Storage 说成简单 key-value，它更准确是 Request/Response 映射；不要把 CDN 和 SW 讲成二选一，它们通常是协同关系。

## 13. 浏览器的存储方式

- 一句话结论：浏览器常见存储方式包括 Cookie、Web Storage、IndexedDB、Cache Storage，它们分别适合身份会话、简单业务数据、本地结构化大数据、资源级缓存与离线能力。
- 30秒回答：浏览器存储常见有四类。Cookie 是协议级存储，常用于身份认证，会随着请求自动携带；Web Storage 包括 localStorage 和 sessionStorage，适合存简单业务信息；IndexedDB 是浏览器本地数据库，适合大容量结构化数据；Cache Storage 更偏资源缓存，通常配合 Service worker 做离线访问和策略缓存。面试里重点不是背名字，而是讲清楚它们的“作用、容量、是否自动发送、是否适合存敏感信息”。
- 1分钟展开：
  - **Cookie**: Cookie 是协议级存储，最典型用途是身份认证和会话维持，比如登录态。它容量较小，会在符合域名和路径规则时自动随 HTTP 请求发送给服务端，所以特别适合服务端识别用户状态。它也是几类存储里安全属性最丰富的一类，可以配置 HttpOnly、Secure、SameSite 等。
  - **Cookie 的安全点**: HttpOnly 可以避免前端 JS 直接读取，降低被 XSS 窃取风险；Secure 限制只在 HTTPS 下传输；SameSite 可以降低 CSRF 风险。所以面试里可以说：如果涉及服务端认证和安全控制，Cookie 仍然有很强的实际价值。
  - **Web Storage**: 它是网页级存储，分为 localStorage 和 sessionStorage。localStorage 生命周期更长，关闭页面后数据还在；sessionStorage 生命周期通常跟当前标签页会话绑定，标签页关闭后数据清除。二者容量一般比 Cookie 大，常说大约 5MB 左右，适合存一些简单的业务数据。
  - **Web Storage 的限制**: 它不会自动随请求发送，所以不适合作为传统服务端自动识别的会话机制；而且前端 JS 能直接读写，如果页面存在 XSS 漏洞，里面的数据就可能被盗取，所以敏感令牌放这里要非常谨慎。它和 CSRF 的关系则不一样，CSRF 更多和“请求自动携带凭证”有关，所以 Cookie 风险更典型。

- **IndexedDB:** IndexedDB 是浏览器本地的异步 NoSQL 数据库，容量通常远大于 localStorage，更适合存储结构化数据、离线业务数据、大量列表、草稿、文件索引等。它支持事务、索引、对象仓库，属于更工程化的浏览器端持久化方案。
- **IndexedDB 的使用特点:** 它是异步的，传统用法比较偏事件驱动，比如 onSuccess、onerror；实际开发中通常会再封装成 Promise 风格来使用。面试里可以强调：它不是简单 key-value，而是浏览器端数据库。
- **Cache API / Cache Storage:** 这类存储更偏资源缓存，不是拿来存普通业务对象的。它保存的是请求和响应报文映射，常和 Service Worker 搭配，用于预缓存、运行时缓存、离线访问、PWA 等场景。
- **怎么选型:** 如果是登录态、会话和需要服务端自动识别的凭证，更偏向 Cookie；如果是小体量、简单键值业务数据，用 localStorage/sessionStorage；如果是大容量结构化本地数据，用 IndexedDB；如果是静态资源、接口响应、离线缓存，则更适合 Cache Storage。
- **高频关键词:** 浏览器存储、Cookie、协议级存储、自动携带、HttpOnly、Secure、SameSite、Web Storage、localStorage、sessionStorage、5MB、XSS、CSRF、IndexedDB、异步、NoSQL、对象仓库、索引、onSuccess、onerror、Cache API、Cache Storage、请求响应报文、Service Worker、离线访问、PWA
- **易追问:** Cookie 和 localStorage 最大区别是什么？为什么 Cookie 更适合身份认证？HttpOnly、Secure、SameSite 分别解决什么问题？localStorage 和 sessionStorage 区别是什么？为什么不建议把敏感 token 放 localStorage？IndexedDB 适合什么场景？Cache Storage 和 localStorage 有什么本质区别？
- **回答雷区:** 不要把 Cookie 说成前端专属存储，它本质和 HTTP 协议强相关；不要说 Web Storage 比 Cookie 更安全，它只是“不自动发送”，但依然容易受 XSS 影响；不要把 IndexedDB 说成“大一点的 localStorage”，它是异步数据库模型；不要把 Cache Storage 当作普通业务数据存储，它更适合缓存资源和响应对象；不要把 CSRF 和 XSS 风险混成一类，它们产生机制不同。

## 14. 包管理器

- **一句话结论:** 前端常见包管理器有 npm、yarn、pnpm，它们都负责依赖安装、版本管理和脚本执行；其中 pnpm 近几年更受欢迎，原因是更省磁盘、更快、更严格。

- 30秒回答：**npm**、**yarn**、**pnpm** 本质都是 Node 生态的包管理工具，用来安装依赖、管理锁文件、执行脚本。**pnpm** 的优势比较明显：它通过内容寻址存储和硬链接共享依赖，能显著减少磁盘占用；安装时并行和缓存机制也更高效；同时它的依赖管理更严格，不允许项目隐式访问未声明依赖。**npx** 则不是包管理器本身，而是 Node 提供的命令行执行工具，常用于临时执行包里的命令。
- 1分钟展开：
  - **npm / yarn / pnpm 是什么**：它们都是前端项目里最常见的包管理工具，主要负责依赖下载、版本解析、锁定依赖树、执行 **scripts**。区别主要体现在安装策略、依赖组织方式、性能和工程约束上。
  - **pnpm 为什么更省空间**：**pnpm** 不会像传统方式那样给每个项目都复制一份完整依赖，而是把包统一存到全局内容仓库，再通过硬链接或符号链接组织到项目中。这样多个项目可以共享同一份依赖内容，所以磁盘占用明显更低。
  - **pnpm 为什么更快**：一方面它复用全局依赖仓库，命中缓存时不需要重复下载；另一方面安装流程支持并行处理，依赖解析和写入也做了优化，所以在多项目和大仓库场景里通常性能表现更好。
  - **pnpm 的严格依赖管理**：**pnpm** 默认不鼓励隐式依赖访问，也就是项目只能访问自己 **package.json** 里明确声明的依赖，而不能偷偷用别的包间接装进来的模块。这能减少“本地能跑、换环境就挂”的问题，对 **monorepo** 和团队协作很有价值。
  - **兼容性**：**pnpm** 在命令使用习惯上和 **npm/yarn** 比较接近，也兼容大多数 Node 项目生态，所以迁移成本通常不算高。现在很多新项目、**monorepo**、前端工程化场景都会优先考虑 **pnpm**。
  - **npx 是什么**：**npx** 是 Node 生态里的命令行执行工具，常见作用是临时执行某个包暴露出来的命令，比如脚手架初始化、一次性工具运行，而不一定要全局安装。可以理解成“帮你找到并执行包里的二进制命令”。
  - **怎么回答更完整**：面试里可以总结成一句：**npm/yarn/pnpm** 解决的是依赖管理问题，而 **npx** 解决的是命令执行问题；**pnpm** 的核心优势是共享存储、安装更快、依赖更严格。
- 高频关键词：包管理器、**npm**、**yarn**、**pnpm**、依赖管理、锁文件、内容寻址存储、硬链接、符号链接、共享依赖仓库、缓存、并行安装、严格依赖、隐式依赖、兼容性、**npx**、命令执行
- 易追问：**npm**、**yarn**、**pnpm** 有什么本质区别？**pnpm** 为什么更省磁盘？硬链接和复制依赖有什么区别？为什么说 **pnpm** 更严格？什么叫隐式依赖？**npx** 和 **npm install -g** 有什么区别？**monorepo** 为什么常用 **pnpm**？

- 回答雷区：不要把 `npx` 说成包管理器，它更准确是命令执行工具；不要把 `pnpm` 的优势只说成“更快”，面试更看重你是否知道共享存储和严格依赖；不要把“严格依赖管理”说成无法使用间接依赖的缺点，工程上这反而能减少隐患；不要把硬链接和软链接混为一谈，面试里即使不展开，也别混着说。

## 15. 你了解过 RESTful 规则吗？

- 一句话结论：**RESTful** 不是具体协议，而是一种基于 **HTTP** 的接口设计风格，核心思想是“面向资源设计接口”，通过统一的请求方法、路径规范、参数规则和状态码表达操作语义。
- **30秒回答**：**RESTful** 可以理解成一种更规范的 **HTTP API** 设计方式。它强调把数据对象抽象成资源，用 **URL** 表示资源，用不同 **HTTP** 方法表示操作，比如 **GET** 查、**POST** 增、**PUT/PATCH** 改、**DELETE** 删；路径参数通常表示资源 **ID**，请求参数通常表示筛选条件；再配合统一的状态码和响应结构，让接口更清晰、可读、可维护。
- 1分钟展开：
  - **RESTful 是什么**：它本质是一种 **HTTP** 接口设计风格，不是强制标准。它强调围绕“资源”来设计接口，而不是围绕“动作”命名接口。比如更推荐 `/users/123`，而不是 `/getUserById` 这种动作式路径。
  - **HTTP 设计风格**：**RESTful** 主要体现在三个方面：一是请求方式要有语义，二是参数传递要清晰，三是响应状态和结构要统一。这样接口更容易理解，也更方便前后端协作和后续维护。
  - **请求方式的语义**：常见约定是 **GET** 用于获取资源，**POST** 用于创建资源，**PUT** 通常表示整体更新，**PATCH** 表示部分更新，**DELETE** 用于删除资源。面试里可以补一句：实际项目也会受历史包袱或业务特点影响，但 **RESTful** 倾向于让方法语义尽量和操作一致。
  - **路径参数和查询参数**：路径参数通常表示资源的唯一标识，比如 `/users/123` 里的 `123`；查询参数通常表示筛选、分页、排序或操作条件，比如 `/users?page=1&size=10`。简单说，路径更偏“你要操作谁”，查询参数更偏“你想怎么查”。
  - **路径设计原则**：**RESTful** 路径要尽量名词化、层级化、简洁直观。比如 `/users/123/orders/456` 能体现资源之间的隶属关系。一般不推荐路径里堆太多动词，除非是确实难以资源化的特殊动作接口。
  - **HTTP 状态码规范**：常见大类有 **1xx** 表示信息性状态，**2xx** 表示成功，**3xx** 表示重定向，**4xx** 表示客户端错误，**5xx** 表示服务端错误。高频细分里，**200** 成功、**201** 创建成功、**204** 删除成功无返回体、**400** 参数错误、**401** 未认证、**403** 无权限、**404** 资源不存在、**500** 服务器异常。

- 怎么回答更完整：面试里可以总结成一句，RESTful 的目标不是形式化背规范，而是让接口“资源清晰、方法语义明确、路径直观、状态码统一”，这样整个 API 体系更可维护。
- 高频关键词：RESTful、HTTP 设计风格、面向资源、URL、请求方法、GET、POST、PUT、PATCH、DELETE、路径参数、查询参数、资源层级、状态码、2xx、3xx、4xx、5xx
- 易追问：RESTful 和 RPC 风格有什么区别？为什么 RESTful 强调名词化路径？PUT 和 PATCH 有什么区别？路径参数和查询参数怎么区分？删除接口为什么常返回 204？业务里为什么很多接口不完全 RESTful？状态码和业务码怎么配合？
- 回答雷区：不要把 RESTful 说成强制标准，它更准确是设计风格；不要把路径写成一堆动词却说自己是 RESTful；不要把路径参数和查询参数的职责说反；不要说 POST 就一定代表“所有非查询操作”，更规范的回答是根据资源语义选方法；不要只背状态码大类，最好能顺带举几个常用码。

## 16. CORS（跨域资源共享）

- 一句话结论：CORS 是为了解决跨域访问而定义的一套跨源资源共享机制，本质是浏览器先发请求，再根据服务端返回的 CORS 响应头决定是否允许前端读取响应结果。
- 30秒回答：跨域问题本质来自浏览器的同源策略，协议、域名、端口任意一个不同都算跨域。它存在的目的是隔离不同来源的页面，防止恶意站点随意读取其他站点数据。CORS 是最主流的跨域解决方案，核心是浏览器请求时带上 Origin，服务端通过 Access-Control-Allow-Origin 等响应头声明允许哪些源访问；如果是复杂请求，浏览器还会先发一次 OPTIONS 预检请求确认是否允许。
- 1分钟展开：
  - 什么是跨域问题：跨域本质上是浏览器的同源策略限制。所谓同源，指协议、域名、端口三者都相同；只要其中任意一项不同，就属于跨源访问。它的核心目标是做安全隔离，避免一个站点的脚本随意读另一个站点的数据。
  - 为什么要有跨域限制：如果没有同源策略，恶意网站可以在用户不知情的情况下读取其他网站的接口数据、页面内容或敏感信息，安全风险会非常高。所以跨域问题不是网络层“不能请求”，而是浏览器层“限制读取响应”。
  - 常见场景：最典型的是前后端分离开发时，前端页面跑在 localhost:3000，接口跑在 localhost:8080；或者正式环境里页面在一个域名、接口在另一个域名。面试里也可以补充：图片、脚本等资源加载和 Ajax 读取数据的跨域规则并不完全一样。

- **CORS 的核心机制**：浏览器发跨域请求时，会自动带上 `Origin` 请求头；服务端如果允许，就在响应头里返回 `Access-Control-Allow-Origin`。如果还要允许特定方法、请求头或携带凭证，就继续返回 `Access-Control-Allow-Methods`、`Access-Control-Allow-Headers`、`Access-Control-Allow-Credentials` 等。
- **你提到的请求头**：浏览器在预检阶段常会发 `Access-Control-Request-Method` 和 `Access-Control-Request-Headers`，告诉服务端“我后面真正要用什么方法、带哪些请求头”；服务端再根据策略决定是否放行。这里要注意，`Origin` 是请求头，`Allow-*` 通常是响应头。
- **简单请求和复杂请求**：简单请求一般是方法较普通、请求头较受限、`Content-Type` 也属于浏览器认可范围，比如常见 `GET/POST` 且没有自定义复杂头。复杂请求比如 `PUT/DELETE`、带自定义请求头、或者特殊 `Content-Type`，浏览器会在正式请求前先发一次预检请求。
- **预检请求（OPTIONS）**：预检请求的本质是“先问服务端允不允许”。浏览器会发送 `OPTIONS` 请求，询问目标服务是否允许当前源、方法、头信息。服务端同意后，浏览器才会发送真正业务请求。为了减少预检开销，服务端可以通过 `Access-Control-Max-Age` 告诉浏览器这个预检结果可以缓存多久。
- **高频关键词**：跨域、同源策略、协议、域名、端口、浏览器安全、`CORS`、`Origin`、`Access-Control-Allow-Origin`、`Access-Control-Allow-Methods`、`Access-Control-Allow-Headers`、`Access-Control-Allow-Credentials`、`Access-Control-Request-Method`、`Access-Control-Request-Headers`、简单请求、复杂请求、预检请求、`OPTIONS`、`Access-Control-Max-Age`
- **易追问**：跨域是请求发不出去还是响应拿不到？为什么浏览器要做同源限制？哪些情况算跨域？`CORS` 的请求头和响应头分别有哪些？简单请求和复杂请求怎么区分？为什么会触发 `OPTIONS`？`Access-Control-Max-Age` 有什么作用？带 `cookie` 的跨域请求需要注意什么？
- **回答雷区**：不要说“跨域请求发不出去”，更准确是“浏览器限制前端读取响应”；不要把 `Origin`、`Access-Control-Request-*` 和 `Access-Control-Allow-*` 的请求/响应方向说反；不要说所有跨域都会先发预检，只有复杂请求才会；不要把 `OPTIONS` 说成真正业务请求，它只是预检；不要漏掉同源判断是协议、域名、端口三项。

## 17. Promise.all() vs Promise.race()

- 一句话结论：`Promise.all` 关心“全部成功后统一返回”，`Promise.race` 关心“谁先结束就用谁”，而 `allSettled` 和 `any` 则分别解决“收集所有结果”和“只要一个成功就行”的问题。

- 30秒回答: **Promise** 本质上是对异步结果的对象化封装, 状态有 **pending**、**fulfilled**、**rejected**, 并且状态一旦确定就不可逆。静态方法里, **Promise.all** 适合并发等多个任务全部成功; **Promise.race** 谁先完成就返回谁; **Promise.allSettled** 不管成功失败都收集结果; **Promise.any** 只要有一个成功就返回, 全部失败才报错。手写这些方法的核心就是统一把输入转成 **Promise**, 然后控制计数、顺序和状态收敛规则。
- 1分钟展开:
  - **Promise 原理简述:** **Promise** 可以理解成“未来才会拿到结果的占位对象”。它内部维护状态和值, 初始是 **pending**, 后续只能变成成功或失败, 而且状态一旦确定不能再改。 **then** 之所以能链式调用, 是因为每次执行后都会返回一个新的 **Promise**。
  - **Promise 静态方法的共同点:** 这几个方法都会先把输入项通过 **Promise.resolve()** 统一包装, 这样普通值、**thenable**、**Promise** 都能一起处理。真正的区别在于: 它们什么时候结束、成功失败的条件是什么、结果怎么组织。
  - **Promise.all:** 只有当所有 **Promise** 都成功时才整体成功, 并按原输入顺序返回结果数组; 只要有一个失败, 就立即整体失败。适合多个接口必须都成功后才能继续的场景, 比如页面初始化并发请求多个基础数据。
  - **Promise.race:** 不关心谁成功谁失败, 只关心“谁先结束”。第一个 **settle** 的结果会直接决定最终结果, 所以既可能先成功, 也可能先失败。适合超时控制、主备请求竞速等场景。
  - **Promise.allSettled:** 它会等所有任务都结束, 不管成功还是失败, 最终返回每一项的状态和结果。适合批量任务汇总、日志统计、部分失败也要继续展示结果的场景。
  - **Promise.any:** 只要有一个成功就整体成功; 只有全部失败时才整体失败, 并抛出聚合错误。适合从多个镜像源、多个服务节点里挑最快成功结果的场景。
  - **手写 all 思路:** 核心是维护一个结果数组和完成计数器。遍历输入, 把每一项包装成 **Promise**; 每项成功后按原下标存入结果数组并计数; 当计数等于总数时 **resolve(results)**; 任意一项失败就立刻 **reject**。
  - **手写 race 思路:** 遍历输入后直接给每一项绑定 **then(resolve, reject)**, 谁最先触发就直接敲定外层 **Promise** 状态, 因为 **Promise** 状态不可逆, 后面的结果会被忽略。
  - **手写 allSettled / any 思路:** **allSettled** 也是等全部结束, 只不过无论成功失败都记录 **{ status, value/reason }**; **any** 则反过来, 先成功就直接 **resolve**, 只有在统计到全部失败时才统一 **reject**。

- 高频关键词：`Promise`、`pending`、`fulfilled`、`rejected`、状态不可逆、链式调用、静态方法、`Promise.resolve`、`Promise.all`、`Promise.race`、`Promise.allSettled`、`Promise.any`、并发、竞速、结果顺序、聚合错误、手写实现、计数器
- 易追问：`all` 和 `race` 最本质的区别是什么？`race` 会不会先失败？`all` 返回结果顺序按完成顺序还是输入顺序？`allSettled` 和 `all` 的使用场景区别是什么？`any` 和 `race` 有什么区别？为什么手写时要先 `Promise.resolve` 包一层？如果传普通值进去会怎样？
- 回答雷区：不要把 `race` 说成“谁先成功返回谁”，更准确是“谁先结束用谁”；不要把 `all` 的结果顺序说成按完成顺序，实际应按输入顺序；不要把 `allSettled` 和 `any` 混淆，一个是全收集，一个是挑任意成功；不要漏掉“输入可以是普通值、thenable、Promise”这一点；不要把“静态方法”讲成实例方法。

## 18. 说一下事件循环

- 一句话结论：事件循环是 JavaScript 运行时协调同步代码、微任务和宏任务执行顺序的机制，核心规则是：主线程执行栈清空后，先清空微任务队列，再取下一个宏任务执行。
- 30秒回答：JavaScript 是单线程执行的，脚本会先进入调用栈按顺序运行。运行过程中遇到异步任务，不会马上执行，而是交给宿主环境处理，等时机成熟后放进任务队列。任务主要分成微任务和宏任务：当前同步代码执行完后，会先把所有微任务清空，再执行下一个宏任务，这个不断重复的过程就是事件循环。
- 1分钟展开：
  - 主线程和调用栈：JavaScript 代码最开始作为一个宏任务进入主线程执行栈，按顺序一行行执行。同步代码会一直占用调用栈，只有当前执行栈清空，事件循环才有机会调度后续任务。
  - 为什么需要事件循环：因为 JavaScript 是单线程，如果所有异步操作都阻塞主线程，页面就无法响应。事件循环的作用就是让耗时任务先挂起，等结果准备好后再回到主线程执行回调。
  - 宏任务和微任务：常见微任务有 `Promise.then/catch/finally`、`queueMicrotask`、`MutationObserver`，在 Node 里还有 `process.nextTick`。常见宏任务有整体脚本、`setTimeout`、`setInterval`、I/O、UI 事件、消息事件等。面试里如果提 `fetch`，更稳的说法是“`fetch` 的网络过程由宿主处理，Promise 回调进入微任务”；`requestAnimationFrame` 也更偏浏览器渲染调度阶段，不建议简单粗暴归到普通宏任务里。
  - 执行顺序核心规则：先执行当前宏任务里的同步代码；同步代码跑完后，检查并依次执行所有微任务；微任务清空后，事件循环再从宏任务队列里取出下一个宏任务。然后重复这个流程。

- **为什么微任务优先：**因为微任务通常用于更细粒度的状态收尾和异步衔接，比如 `Promise` 链式调用。如果不先清空微任务，很多逻辑的时序会变得不可预测，所以规范要求每轮宏任务结束后优先清空微任务队列。
- **浏览器里的表现：**比如一个 `setTimeout` 和一个 `Promise.then` 同时存在时，即使 `setTimeout(..., 0)`，通常也是同步代码先执行，再执行 `Promise.then`，最后才轮到 `setTimeout` 回调，因为 `Promise` 回调属于微任务。
- **和渲染的关系：**浏览器通常会在合适时机处理渲染，事件循环、微任务清空、页面重绘之间是相互配合的，所以如果微任务链过长，也可能阻塞页面渲染和交互响应。
- **高频关键词：**事件循环、Event Loop、单线程、主线程、调用栈、执行栈、同步代码、异步任务、任务队列、宏任务、微任务、`Promise.then`、`queueMicrotask`、`process.nextTick`、`setTimeout`、I/O、UI 事件、栈清空、先微后宏
- **易追问：**为什么 JavaScript 需要事件循环？宏任务和微任务分别有哪些？为什么 `Promise.then` 比 `setTimeout` 先执行？`setTimeout(fn, 0)` 为什么不是立即执行？`fetch` 回调到底算微任务还是宏任务？`requestAnimationFrame` 和事件循环是什么关系？Node 里的事件循环和浏览器有什么差别？
- **回答雷区：**不要把所有异步都简单说成“放进宏任务队列”，`Promise` 回调属于微任务；不要把 `fetch` 直接粗暴归类成宏任务，面试里更准确是“网络由宿主处理，`then` 回调进微任务”；不要说“每执行一行代码就检查一次队列”，更准确是当前调用栈清空后才调度；不要漏掉“微任务会在当前宏任务结束后一次性清空”这一点。

## 19. Chrome 开发者工具

- **一句话结论：**Chrome 开发者工具是前端排查页面结构、脚本逻辑、性能瓶颈、内存泄漏、渲染问题、网络请求和缓存状态的核心调试工具集。
- **30秒回答：**Chrome DevTools 常用面板有几个：`Elements` 用来查看和改 HTML/CSS，`Console` 用来调试 JavaScript，`Network` 看请求链路，`Performance` 做性能分析，`Memory` 查内存问题，`Rendering` 看渲染辅助信息，`Application` 查缓存、存储和 Service Worker，另外 `Lighthouse` 可以做页面质量和性能审计。面试里如果能把“哪个问题用哪个面板查”说清楚，就会比较加分。
- **1分钟展开：**
  - **Elements：**主要用于查看和修改 DOM 结构、样式、盒模型、布局信息。前端排查样式覆盖、布局错位、元素状态变化时，第一反应通常就是看 `Elements`。

- **Console:** 用于输出日志、执行临时代码、查看报错堆栈、调试 JavaScript 表达式。日常开发里常结合 `console.log`、`console.table`、断点和对象展开一起使用。
- **Network:** 用于查看请求地址、请求方法、状态码、请求头、响应头、响应体、耗时瀑布图。排查接口失败、缓存命中、资源加载慢、跨域问题、预检请求、静态资源体积时都非常关键。
- **Performance:** 用于录制页面运行过程，分析脚本执行、样式计算、布局、绘制、合成等阶段耗时。页面卡顿、掉帧、长任务、首屏慢、交互延迟等问题，通常要靠它定位。
- **Memory:** 用于分析内存占用和泄漏，比如快照对比、分配时间线、查看哪些对象被意外持有没释放。排查频繁切页后内存持续上涨、定时器未清理、事件监听未解绑等问题很常用。
- **Rendering:** 这是浏览器渲染辅助工具，常用来开启 FPS、重绘区域、布局偏移等可视化信息。做渲染优化、分析重排重绘、定位页面抖动时比较有用。
- **Application:** 用于查看浏览器端存储和应用状态，包括 `Cookie`、`localStorage`、`sessionStorage`、`IndexedDB`、`Cache Storage`、`Service worker`、Manifest 等，尤其适合排查缓存、登录态、PWA、离线能力问题。
- **Lighthouse:** 它更像自动化审计工具，可以从性能、可访问性、最佳实践、SEO、PWA 等维度给出评分和建议，适合做页面整体质量扫描和优化方向参考。
- 高频关键词: Chrome DevTools、`Elements`、`Console`、`Network`、`Performance`、`Memory`、`Rendering`、`Application`、`Lighthouse`、DOM、CSS、断点调试、瀑布图、长任务、内存泄漏、重排重绘、缓存、`Service worker`
- 易追问: 排查页面白屏先看哪个面板? 接口超时或跨域看哪里? 页面卡顿和掉帧用什么定位? 内存泄漏怎么排查? `Application` 面板能看哪些存储? `Lighthouse` 和 `Performance` 有什么区别? 如何判断资源有没有命中缓存?
- 回答雷区: 不要只背面板名字不说用途，面试官更看重“遇到问题怎么用”; 不要把 `Performance` 和 `Memory` 混为一谈，一个偏耗时链路，一个偏对象占用和泄漏; 不要把 `Rendering` 说成独立开发功能区，它更多是渲染辅助开关集合; 不要把 `Lighthouse` 说成实时调试工具，它更偏自动化审计和建议输出。

## 20. 从输入 URL 到页面渲染，发生了什么？

- 一句话结论: 浏览器从输入 URL 到页面显示，核心会经历 URL 解析、DNS 解析、建立连接、发送请求、服务端响应、浏览器解析资源、构建渲染树、布局绘制与合成几个阶段。

- **30秒回答**：这个过程可以分成两大部分。前半段是网络过程，包括解析 URL、查 DNS、建立 TCP/HTTPS 连接、发送 HTTP 请求、服务端返回 HTML；后半段是渲染过程，浏览器解析 HTML 构建 DOM，解析 CSS 构建 CSSOM，合成渲染树后进行布局、绘制和合成，最后把页面显示出来。如果中间有 JS，还会影响 HTML 解析和页面渲染时机。
- **1分钟展开**：
  - **URL 解析与导航准备**：浏览器先解析输入的是搜索词还是 URL，如果是 URL，就解析协议、域名、端口、路径等信息，并检查缓存、HSTS、Service Worker、是否需要重定向等。
  - **DNS 与连接建立**：浏览器先查本地缓存、系统缓存、DNS 缓存，拿到 IP 后建立连接。HTTP 一般走 TCP 三次握手；HTTPS 还要在 TCP 之后做 TLS 握手、校验证书、协商会话密钥。现代场景下还可能复用已有连接，HTTP/2 支持多路复用。
  - **请求与响应**：连接建立后，浏览器发送 HTTP 请求，请求里带上方法、请求头、Cookie 等；服务端处理后返回状态码、响应头和响应体。若有缓存命中，可能直接走本地缓存或返回 **304**；若是重定向，则浏览器会继续发起下一跳请求。
  - **HTML 解析与资源发现**：浏览器拿到 HTML 后边下载边解析，逐步构建 DOM 树。在解析过程中如果遇到 **link**、**script**、**img** 等标签，会继续请求 CSS、JS、图片等子资源。这里可以顺带提“预加载扫描器”会提前发现部分资源并并行下载。
  - **CSSOM、DOM 与渲染树**：CSS 下载并解析后生成 CSSOM，DOM 和 CSSOM 结合形成 Render Tree。只有能参与渲染的节点才会进入渲染树，比如 **display: none** 的元素不会进入。
  - **布局、绘制、合成**：有了渲染树后，浏览器会计算每个节点的几何信息，这一步叫布局（Layout/Reflow）；再把颜色、边框、文字、阴影等画出来，这一步叫绘制（Paint）；最后由合成线程把不同图层合成到屏幕上，页面就显示出来了。
  - **JavaScript 的影响**：默认情况下，普通 **script** 会阻塞 HTML 解析，因为浏览器要先执行 JS，防止脚本修改前面的 DOM 导致解析结果不一致。CSS 虽然不阻塞 DOM 构建，但会阻塞渲染，而且也可能阻塞后续依赖样式信息的 JS 执行，所以首屏优化经常围绕“减少阻塞资源”展开。
- **高频关键词**：URL 解析、导航、DNS、TCP 三次握手、TLS 握手、HTTPS、HTTP 请求、缓存、**304**、重定向、HTML 解析、DOM、CSSOM、Render Tree、布局、重排、绘制、重绘、合成、阻塞渲染、**script**、**defer**、**async**、预加载扫描器、HTTP/2、多路复用
- **易追问**：DNS 查找顺序是什么？HTTPS 比 HTTP 多了哪些步骤？为什么 JS 会阻塞页面解析？CSS 为什么会阻塞渲染？DOM 和 CSSOM 是怎么变成渲染树的？什么是重排和重绘？**defer** 和 **async** 有什么区别？输入 URL 到首屏显示，哪些环节最影响性能？

- 回答雷区：不要把“页面渲染”简单说成“拿到 HTML 就显示了”，中间还有 DOM/CSSOM、渲染树、布局和绘制；不要说 CSS 会阻塞 DOM 解析，更准确是它主要阻塞渲染，并可能影响后续脚本执行；不要把普通 `script`、`defer`、`async` 的执行时机说混；不要漏掉 HTTPS 下还有 TLS 握手和证书校验；不要把重绘和重排当成一回事，重排通常比重绘开销更大。

## 21. 浏览器中的进程和线程

- 一句话结论：现代浏览器通常采用多进程架构来做隔离和稳定性保障，每个进程内部再通过多线程分工完成渲染、脚本执行、网络、合成、解码等任务。
- 30秒回答：先区分概念，进程是资源分配的基本单位，线程是 CPU 调度的基本单位。浏览器为了安全性和稳定性，一般会拆成浏览器主进程、渲染进程、网络进程、GPU 进程、工具进程等；其中页面相关逻辑主要在渲染进程里执行。渲染进程内部又有主线程、合成线程、栅格线程、Worker 线程等，不同线程各司其职，进程之间通过 IPC 通信。
- 1分钟展开：
  - 进程和线程的区别：进程更像资源容器，拥有独立地址空间；线程是进程内部真正执行任务的单位。同一进程内线程共享大部分资源，不同进程之间天然隔离，所以浏览器更适合用多进程提高稳定性和安全性。
  - 浏览器常见进程：通常包括浏览器主进程、渲染进程、网络进程、GPU 进程和工具进程。主进程负责标签页管理、窗口管理、权限、调度等；渲染进程负责页面解析、布局、绘制、JS 执行；网络进程负责请求收发、缓存、协议处理；GPU 进程负责图层合成和 GPU 加速；工具进程常处理音视频解码、扩展、存储或其他辅助能力。
  - 为什么要多进程：第一是隔离，一个页面崩了不一定拖死整个浏览器；第二是安全，渲染进程通常运行在沙箱里，即使页面脚本被攻击，也更难直接影响系统和其他页面；第三是更容易利用多核 CPU 并行处理不同任务。
  - 进程之间怎么通信：不同进程内存隔离，不能直接共享对象，所以要通过 IPC 传递结构化消息。浏览器主进程通常扮演调度者角色，负责把渲染进程、网络进程、GPU 进程之间的信息串起来。
  - 渲染进程里的线程：最核心的是主线程，负责 HTML/CSS 解析、DOM 构建、样式计算、布局、JS 执行和大部分事件处理；合成线程负责图层合成；栅格线程负责把图层栅格化；`web worker` 在线程层面承接计算密集型任务；`service worker` 负责离线缓存、请求拦截等后台能力，但它不能直接操作 DOM。
  - 关于网络和定时器：前端常说“JS 发请求”“定时器到时执行”，但真正的网络收发、计时和部分异步调度并不是都在 JS 主线程里完成，而是由浏览器底层其他线程或进程协作完成，最后再把回调放回事件循环等待主线程处理。

- 死锁补充：如果面试官追问操作系统知识，可以说死锁通常满足四个条件：互斥、请求并保持、不可剥夺、循环等待。常见预防思路是破坏其中至少一个条件，比如统一加锁顺序、一次性申请资源、超时释放、避免循环等待。
- 高频关键词：进程、线程、资源分配、CPU 调度、多进程架构、浏览器主进程、渲染进程、网络进程、GPU 进程、工具进程、IPC、沙箱、主线程、合成线程、栅格线程、**web worker**、**Service worker**、隔离、安全性、稳定性、多核并行、死锁
- 易追问：为什么浏览器要做多进程？标签页是不是一个进程？渲染进程里有哪些线程？为什么 **web worker** 不能操作 DOM？**Service worker** 和 **web worker** 的区别是什么？GPU 进程主要做什么？进程通信为什么要靠 IPC？死锁四个必要条件是什么？
- 回答雷区：不要把“一个浏览器只有一个进程”说错，现代浏览器通常是多进程；不要把网络请求简单说成全在 JS 线程里执行，JS 线程主要负责发起和消费回调；不要把 **web worker** 和 **Service worker** 混为一谈；不要把进程和线程的定义说反；不要把“多进程一定更快”绝对化，更准确是它在隔离、稳定性和并行利用上更有优势，但也有通信和内存成本。

## 22. 浏览器渲染触发的两个事件

- 一句话结论：页面加载阶段最常被问的两个事件是 **DOMContentLoaded** 和 **load**，前者表示 DOM 结构已解析完成，后者表示页面及其依赖资源基本加载完成。
- 30秒回答：**DOMContentLoaded** 触发得更早，表示 HTML 已经被解析成 DOM，可以开始操作 DOM 了，但图片、样式、**iframe** 等资源不一定都加载完；**load** 触发得更晚，表示页面依赖资源基本都加载完成。实际开发里，如果只是想尽早操作 DOM，通常监听 **DOMContentLoaded**；如果依赖图片尺寸或完整资源状态，再看 **load**。
- 1分钟展开：
  - **DOMContentLoaded** 是什么：它表示浏览器已经把 HTML 文档解析完成，DOM 树已经建立起来。此时脚本通常可以安全地查询和操作页面节点，所以很多初始化逻辑会放在这个阶段。
  - 它不等于所有资源都加载完：**DOMContentLoaded** 并不要求图片、视频、字体、**iframe** 等全部下载结束，所以它更适合做“页面结构可用”的判断，而不是“页面资源全部 ready”的判断。
  - **load** 是什么：**load** 一般在页面及其依赖资源加载完成后触发，比如图片、样式、脚本、**iframe** 等都完成加载。它比 **DOMContentLoaded** 更晚，所以不适合所有初始化都压到这里。
  - 二者的使用场景：如果你只是要挂事件、读取 DOM、初始化组件，优先考虑 **DOMContentLoaded**；如果你要拿图片真实宽高、等字体或外部资源都到位，**load** 更稳妥。

- 和脚本加载的关系：普通同步 `script` 会阻塞 HTML 解析，因此也会影响 `DOMContentLoaded` 的触发时机；而 `defer` 脚本会在 HTML 解析完成后、`DOMContentLoaded` 之前按顺序执行；`async` 脚本下载完就执行，时机更不稳定，不保证和 DOM 解析顺序一致。
- 和页面性能的关系：面试里可以顺手补一句，很多首屏交互初始化会尽量放在更早的阶段，不必等 `load`，否则容易让可交互时间变晚。
- 高频关键词：`DOMContentLoaded`、`load`、DOM 解析完成、资源加载完成、初始化时机、同步脚本、`defer`、`async`、页面可交互、图片尺寸、`iframe`、样式资源
- 易追问：`DOMContentLoaded` 和 `load` 的区别是什么？为什么 `DOMContentLoaded` 更早？同步脚本为什么会影响它？`defer` 和 `async` 会怎么影响页面初始化时机？什么时候必须等到 `load`？如果脚本写在 `body` 底部，还需不需要监听这些事件？
- 回答雷区：不要把 `DOMContentLoaded` 说成“所有资源加载完成”；不要把 `load` 写成 `Loaded`，标准事件名是 `load`；不要忽略同步脚本对 DOM 解析的阻塞影响；不要所有初始化都等 `load`，这通常不是更优实践。

## 23. Cookie 设置的几种方式

- 一句话结论：Cookie 可以由前端通过 `document.cookie` 设置，也可以由服务端通过 `Set-Cookie` 响应头设置，但像 `HttpOnly` 这类安全属性只能由服务端控制，前端脚本无法读取。
- 30秒回答：设置 Cookie 主要有两种方式。第一种是前端用 `document.cookie` 写入简单 Cookie，但它能力有限，像 `HttpOnly` 就不能设置；第二种是服务端在响应头里返回 `Set-Cookie`，这种方式更完整，也更安全，常用于登录态、会话标识、刷新令牌等敏感信息。Cookie 还能配合 `Expires/Max-Age`、`Domain`、`Path`、`Secure`、`SameSite` 等属性控制作用范围和安全策略。
- 1分钟展开：
  - 前端设置：前端可以通过 `document.cookie = 'key=value'` 写入 Cookie，也可以追加 `path`、过期时间等属性。但它本质是字符串接口，读写都不够直观，而且无法设置 `HttpOnly`，所以不适合保存高敏感信息。
  - 服务端设置：服务端通过 `Set-Cookie` 响应头下发 Cookie，是更常见、更安全的方式。因为它可以完整控制 `HttpOnly`、`Secure`、`SameSite` 等属性，适合登录态、`sessionId`、刷新令牌等场景。
  - 常见属性：`Expires` 或 `Max-Age` 控制过期时间；`Domain` 控制生效域名；`Path` 控制生效路径；`Secure` 表示只在 HTTPS 下发送；`HttpOnly` 表示前端 JS 不能读写；`SameSite` 用来限制跨站发送策略，常见有 `Strict`、`Lax`、`None`。

- **Cookie 自动发送**: Cookie 的一个重要特点是符合域名和路径规则时会随请求自动发送给服务端，这也是它常被用来承载会话标识的原因；但同时也意味着它更容易受到 CSRF 场景影响，所以安全属性配置很重要。
- **前后端分工建议**: 一般业务里，非敏感偏展示型信息才考虑前端写 Cookie；真正和认证、会话、安全相关的 Cookie，更建议由服务端下发并加好安全属性。
- **高频关键词**: Cookie、document.cookie、Set-Cookie、HttpOnly、Secure、SameSite、Expires、Max-Age、Domain、Path、自动发送、登录态、sessionId、HTTPS、跨站请求
- **易追问**: 前端和后端设置 Cookie 的区别是什么？为什么前端不能设置 HttpOnly？Secure 和 HttpOnly 分别防什么？SameSite 有哪几种模式？Cookie 为什么会自动发送？什么时候不适合把数据放进 Cookie？
- **回答雷区**: 不要说前端能设置 HttpOnly，这是服务端属性；不要把 Set-Cookie 说成请求头，它是响应头；不要忽略 Cookie 会自动随请求发送这一特性；不要把 Cookie 当成通用大容量存储，它容量有限且每次请求都会带上。

## 24. Cookie 和 Session 的区别和联系

- **一句话结论**: Cookie 是客户端存储机制，Session 是服务端会话机制，二者常配合使用，最典型的模式是服务端保存会话状态，客户端通过 Cookie 携带 sessionId 来关联这段会话。
- **30秒回答**: Cookie 在浏览器端，符合规则时会随请求自动发送；Session 在服务端，本质是“sessionId -> 会话数据”的映射，数据通常存在内存、Redis 或数据库里。二者的联系是：服务端常把 sessionId 放进 Cookie，让浏览器下次请求自动带回来，服务端再据此找到对应的用户会话状态。
- **1分钟展开**:
  - **Cookie 是什么**: Cookie 是浏览器保存的一小段键值信息，特点是会自动跟随请求发送给服务端，所以很适合保存一些需要“请求自动携带”的标识数据。
  - **Session 是什么**: Session 是服务端维护的一段会话状态。可以把它理解成服务端保存的用户上下文，比如登录状态、用户标识、权限信息、购物车状态等，而不是直接暴露给前端。
  - **二者怎么配合**: 常见流程是用户登录后，服务端创建一条 Session，生成唯一 sessionId，然后通过 Set-Cookie 把这个 sessionId 发给浏览器；浏览器后续请求自动带上它，服务端再根据 sessionId 找到对应的 Session 数据。
  - **核心区别**: Cookie 在客户端，Session 在服务端；Cookie 更像载体或凭证，Session 更像真实状态存储；Cookie 容量小、容易被看到或篡改，所以不适合直接放太多敏感业务数据；Session 更安全，但会增加服务端存储和维护成本。

- **Session 的存储位置**：小型应用可以放内存，重启会丢；生产环境更常见的是 Redis，也可能是数据库或其他集中式存储。这样多台服务才能共享会话，避免单机 Session 导致的登录丢失问题。
- **现代补充**：面试里还可以顺手提一句，很多前后端分离项目会用 Token 体系代替传统 Session，但 Cookie + Session 仍然是经典会话方案，尤其在服务端渲染、传统 Web 系统里很常见。
- **高频关键词**：Cookie、Session、客户端、服务端、会话机制、`sessionId`、`Set-Cookie`、自动发送、状态存储、内存、Redis、数据库、有状态、登录态、会话共享
- **易追问**：Cookie 和 Session 最大区别是什么？Session 为什么通常要配 Cookie？如果浏览器禁用了 Cookie，Session 还能不能用？分布式系统里 Session 怎么共享？为什么说 Session 更有状态？Token 和 Session 的思路差别是什么？
- **回答雷区**：不要把 Session 说成“存在浏览器里”，存在浏览器里的通常只是 `sessionId`；不要说 Cookie 一定不安全，更准确是它是否安全取决于存什么、怎么配属性；不要把 Cookie 和 Session 说成互相替代关系，更常见是配合关系；不要漏掉分布式场景下 Session 共享的问题。

## 25. HTTPS 握手

- **一句话结论**：HTTPS 握手的核心目标是先验证服务器身份，再协商出一把只有通信双方知道的会话密钥，后续业务数据主要用这把对称密钥加密传输。
- **30秒回答**：可以把 HTTPS 握手理解成四步。第一步客户端发 `ClientHello`，告诉服务端自己支持哪些 TLS 版本、加密套件和一个客户端随机数；第二步服务端回 `ServerHello`，确定要用的加密套件，并返回服务端随机数和数字证书；第三步客户端验证证书合法后，双方通过密钥交换算法协商出同一个会话密钥；第四步双方各自发送握手完成消息，之后正式进入加密通信。面试里要强调：真正传业务数据时，主要依赖的是对称加密，不是全程非对称加密。
- **1分钟展开**：
  - **ClientHello**：客户端先发起握手，请求里会带上支持的 TLS 版本、加密套件列表、客户端随机数等信息。现代 TLS 里如果使用 `ECDHE`，客户端通常还会带上自己的密钥交换参数；但面试里不建议简单说成“客户端直接发公钥”，更准确是“发送支持能力和握手参数”。
  - **ServerHello + 证书**：服务端收到后，会从客户端给出的能力里挑一个可用的 TLS 版本和加密套件，返回 `ServerHello`、服务端随机数，以及数字证书。证书里包含服务端公钥、域名信息、签发机构等内容，用来证明“你连到的确实是这个网站”。

- **客户端校证书：**客户端会检查证书域名是否匹配、是否在有效期内、证书链是否可信、签名是否有效，必要时还会检查吊销状态。如果校验失败，浏览器通常会给出证书风险提示。
- **协商会话密钥：**证书验证通过后，双方基于密钥交换算法生成共享秘密。传统 **RSA** 模式里，客户端会生成 **pre-master secret** 并用服务端公钥加密发过去；现代更常见的是 **ECDHE**，双方交换临时密钥参数，各自独立算出同一个共享秘密。再结合客户端随机数和服务端随机数，最终派生出会话密钥。
- **握手完成：**双方会再发送握手完成相关消息，表示“后续都按刚才协商好的密钥和算法通信”。如果这些完成消息能被正常校验，说明握手阶段没有被篡改。
- **正式加密通信：**握手完成后，真正的 **HTTP** 请求和响应数据主要使用对称加密传输，因为它性能更高，适合大量业务数据。
- **为什么这样设计：**对称加密速度快，但前提是双方得先安全地拿到同一把密钥；非对称加密更适合解决“身份认证”和“密钥协商”的问题；数字证书则负责证明服务端身份，所以三者是协作关系，不是替代关系。
- **高频关键词：****HTTPS**、**TLS**、握手、**ClientHello**、**ServerHello**、随机数、加密套件、数字证书、服务端公钥、证书链、CA、身份认证、密钥交换、**RSA**、**ECDHE**、共享秘密、会话密钥、对称加密、握手完成消息
- **易追问：****HTTPS** 为什么比 **HTTP** 安全？为什么业务数据不用非对称加密直接传？证书里到底有什么？客户端怎么校证书？**RSA** 握手和 **ECDHE** 握手有什么区别？什么是前向安全？客户端随机数和服务端随机数有什么作用？
- **回答雷区：**不要把 **HTTPS** 说成“全程都用非对称加密”，实际业务数据主要用对称加密；不要把证书说成“加密文件”，它更准确是身份认证载体；不要把 **ClientHello** 简化成“客户端发公钥”，更稳妥的说法是发送支持能力和握手参数；不要漏掉客户端要先校证书；不要把“协商密钥”和“直接明文传密钥”混为一谈。

## 26. 你为什么选择前端？

- **一句话结论：**我选择前端，一开始是因为它反馈直接、成就感强，后来真正吸引我的是它把业务、交互、工程和用户体验结合在一起的综合性。
- **30秒回答：**最初我选择前端，是因为它能给我非常直接的反馈，代码一改页面就能立刻看到效果，这种正反馈让我很有动力。随着接触更深，我发现自己也很喜欢把复杂需求拆解成清晰界面和交互流程的过程。前端不只是写页面，它还涉及性能优化、组件设计、工程化、用户体验这些内容，所以我会比较有兴趣持续往这个方向深挖。再往后看，我也希望在大前端甚至 **AI** 全栈方向继续扩展自己的能力边界。
- **1分钟展开：**

- **最初的吸引点：**前端的结果可视化非常强，需求实现后能马上看到页面和交互效果，这种反馈比很多纯后台逻辑更直接，也更容易建立成就感和持续投入的动力。
- **真正让我留下来的原因：**随着项目做多，我发现自己不仅喜欢“把东西做出来”，更喜欢把一个模糊需求拆成页面结构、交互流程、状态管理和组件设计，这个过程很符合我的思考方式。
- **前端的综合性：**前端其实是一个很综合的岗位，不只是写 HTML/CSS/JS，还要考虑性能、可维护性、组件复用、工程化规范、协作效率和用户体验。我会比较主动去关注首屏速度、页面交互流畅度、代码结构是否清晰、组件是否可复用这些问题。
- **持续学习空间大：**前端技术变化很快，从框架、构建工具到工程化体系，再到跨端、Node、SSR、边缘计算，延展性很强。我比较喜欢这种需要持续学习、同时又能快速落地的技术方向。
- **未来规划：**从职业规划上，我还是会先把前端基础、工程能力和架构能力打扎实；但长期看，我也愿意往大前端甚至 AI 全栈方向发展，让自己既能理解用户侧体验，也能理解更完整的产品和技术链路。
- **高频关键词：**前端、反馈直接、成就感、界面交互、需求拆解、用户体验、性能优化、组件复用、工程化、代码结构、大前端、持续学习、AI 全栈
- **易追问：**你说的“反馈直接”具体体现在哪？前端最吸引你的点是视觉效果还是工程能力？你做过哪些性能优化或组件复用实践？为什么说前端是综合能力岗位？你怎么理解大前端？为什么会考虑 AI 全栈方向？
- **回答雷区：**不要只回答“因为前端好找工作”或“门槛低”，这种动机会显得太弱；不要只强调页面好看，忽略工程化和业务价值；不要把“喜欢前端”讲得太空，最好落到你关注过的性能、组件、交互、工程问题上；不要一上来就把未来方向说得过于发散，先立足前端主线更稳。

## 27. 你对前端哪些方向感兴趣？

- **一句话结论：**我比较感兴趣的方向主要有三块：前端工程化、性能优化和前端架构，这三者既能解决真实项目问题，也能持续提升团队开发效率和用户体验。
- **30秒回答：**我现在比较关注三个方向。第一是前端工程化，比如脚手架、自动化构建、Monorepo、CI/CD，这些能明显提升开发和协作效率；第二是性能优化，比如首屏优化、懒加载、SSR、PWA、Tree-shaking，这些直接影响用户体验；第三是前端架构，比如微前端、模块化、Serverless，这类方向更偏中大型项目的组织和演进。我比较喜欢那些既有技术深度、又能在业务里真正产生价值的方向。
- **1分钟展开：**

- **前端工程化**：这是我比较感兴趣的一块，因为它直接影响团队开发效率和项目可维护性。比如脚手架能统一项目初始化规范，**webpack/Vite** 影响构建体验和产物质量，**Monorepo** 适合多包协作和公共模块复用，**CI/CD** 则能把测试、构建、发布流程自动化。
- **性能优化**：这是最容易和用户体验直接挂钩的方向。像首屏优化、资源懒加载、代码分包、**Tree-shaking**、SSR、PWA，本质上都在解决“页面更快打开、更快可交互、更稳定可用”的问题。我会比较关注从指标到方案再到验证的完整优化闭环。
- **前端架构**：当项目规模变大后，单纯写功能已经不够了，还要考虑模块边界、团队协作、部署方式和系统演进。像微前端适合多团队独立开发和部署，模块化是基础能力，**Serverless** 更偏向降低基础设施运维成本、提升弹性和交付效率。
- **为什么是这三块**：因为它们分别对应“开发效率”“用户体验”“系统演进”三个维度。我比较希望自己不只是完成页面开发，而是能从更完整的视角看待前端在业务中的价值。
- **后续方向**：短期我会继续把工程化和性能优化做实，因为这两块和日常业务结合最紧；中长期会继续补前端架构以及跨端、Node、AI 结合场景，让自己的能力更完整。
- **高频关键词**：前端工程化、脚手架、自动化构建、**webpack**、**Vite**、**Monorepo**、**CI/CD**、性能优化、首屏优化、懒加载、SSR、PWA、**Tree-shaking**、前端架构、微前端、模块化、**Serverless**、用户体验、系统演进
- **易追问**：为什么你会先关注工程化？你做过哪些性能优化？如何衡量首屏优化效果？**webpack** 和 **vite** 的差别你怎么看？**Monorepo** 适合什么场景？微前端解决什么问题？**Serverless** 适合哪些前端场景？
- **回答雷区**：不要把兴趣方向说成“什么都感兴趣”，这样会显得没有重点；不要只堆技术名词，不解释它们解决了什么问题；不要把微前端、**Serverless** 说成所有项目都必须上，更稳妥的说法是按规模和场景选型；不要只讲技术热词，最好落到效率、体验、维护成本这些业务价值上。

## 28. 讲讲你做的脚手架，怎么开发的，怎么发布的

- **一句话结论**：我做脚手架的核心思路是先把项目初始化流程标准化，再把可变部分做成可配置或插件化，最后通过包管理器发布到 **npm** 仓库并持续维护版本。
- **30秒回答**：我的脚手架大致分三步。第一步先基于 **pnpm create vite@latest** 或类似模板做项目初始化，把基础工程跑起来；第二步在这个基础上加命令行交互、自定义配置、模板选择和插件扩展能力，把重复搭建流程收敛成标准化工具；第三步把脚手架作为 **npm** 包发布，比如配置好 **package.json** 的 **bin** 字段，登录仓库后用 **pnpm publish** 发布，后续再通过版

本迭代持续维护。

- 1分钟展开：
  - 为什么做脚手架：脚手架本质上是在解决“重复初始化项目、目录、依赖、配置”的问题。把这些高频重复动作沉淀下来后，能统一项目规范、减少人工出错，也能提升多人协作时的起步效率。
  - 初始化阶段：我一般会先用 `pnpm create vite@latest` 这类方式生成一个基础项目，把框架、构建工具和基础目录先确定下来。这样可以站在成熟工具链之上开发，而不是从零造轮子。
  - 核心开发思路：脚手架本身通常是一个命令行工具，核心能力包括命令入口、交互式提问、模板拷贝、依赖安装、配置注入和初始化提示。比如根据用户选择决定用什么模板、是否启用 TypeScript、状态管理、路由、代码规范、提交规范等。
  - 自定义配置和模板化：如果不同项目差异较大，我会把可变部分抽出来做成配置项或模板变量，让脚手架根据答案动态生成文件内容，而不是写死一个固定模板。
  - 插件支持：如果功能继续增多，可以把一些能力拆成插件，比如代码规范插件、请求封装插件、国际化插件、测试插件。这样核心脚手架负责调度，插件负责扩展，后续维护也更清晰。
  - 发布流程：发布前会先检查包名、版本号、入口文件和 `bin` 配置是否正确，再在 `npm` 仓库登录账号，然后执行 `pnpm publish`。如果是组织内部使用，也可以发到私有仓库。发布后通常会用 `npx` 或 `pnpm dlx` 实测安装链路是否正常。
  - 维护更新：后续主要是三类工作，一是修复模板 bug；二是适配依赖升级，比如 `Vite`、`ESLint`、框架版本变化；三是做版本管理和变更记录，必要时区分破坏性更新和非破坏性更新。
- 高频关键词：脚手架、CLI、`pnpm create vite@latest`、模板初始化、交互式命令行、自定义配置、模板变量、插件化、`package.json`、`bin`、`npm`、`pnpm publish`、私有仓库、版本管理、维护更新
- 易追问：为什么基于 `Vite` 做而不是从零做？脚手架和模板仓库有什么区别？`bin` 字段是做什么的？如何实现交互式提问？插件化为什么有必要？怎么处理模板升级？发布到 `npm` 后用户如何使用？如果要支持多模板你会怎么设计？
- 回答雷区：不要把脚手架讲成“就是复制一个模板”，面试官更想听到初始化流程、交互、配置注入和扩展机制；不要只说发布命令，不解释包入口和 `bin` 配置；不要忽略后续维护问题，脚手架不是一次性发布完就结束；不要把“基于 `Vite`”说成脚手架全部工作，`Vite` 只是初始化基础。

## 29. Babel 能做什么？

- 一句话结论：Babel 的核心作用是把现代 JavaScript、JSX、TypeScript 等代码转换成目标环境能理解的 JavaScript，并在需要时按策略补充缺失能力。
- 30秒回答：Babel 主要做三件事。第一是语法转换，把新语法降级成老环境可运行的代码；第二是把 JSX、TypeScript 这类非原生语法编译成标准 JavaScript；第三是在需要时配合 `core-js` 等方案按需注入 `polyfill`，补齐目标环境缺失的 API。简单说，Babel 既能“改写语法”，也能“协助补能力”，但它本身更偏编译层，不是完整的打包工具。
- 1分钟展开：
  - 语法转换：这是 Babel 最常见的作用。比如箭头函数、类、可选链、空值合并等现代语法，在低版本浏览器里可能不支持，Babel 可以把它们转换成兼容性更高的写法。
  - 转换 JSX / TypeScript：React 里的 JSX 浏览器不能直接识别，Babel 会把它转换成 `React.createElement` 或新的 JSX runtime 调用；TypeScript 代码也可以经过 Babel 去掉类型标注并输出普通 JavaScript，但类型检查通常还是交给 `tsc` 或其他工具。
  - Polyfill 注入：Babel 不只处理语法，还能配合 `@babel/preset-env` 和 `core-js` 按目标环境按需注入 `polyfill`。比如某些浏览器不支持 `Promise`、`Array.from` 这类 API，就可以在构建时补进去。
  - 它和 `polyfill` 的边界：语法问题和 API 能力问题要分开。Babel 很擅长改写“语法”，但像 `Promise`、`Map`、`Set` 这些运行时 API，不是简单改写代码就能解决，通常需要 `polyfill` 补充实现。
  - 工作机制：Babel 大致分成解析、转换、生成三步。先把源代码解析成 AST，再通过插件和预设修改 AST，最后生成新的代码。所以 Babel 的可扩展性很强，很多框架和工具链都基于它做语法能力扩展。
  - 常见配置：面试里常会提到 `@babel/preset-env`、`@babel/preset-react`、插件系统、`targets` 配置等。`preset-env` 会根据目标浏览器环境决定需要做哪些转换和补丁。
  - 它不做什么：Babel 不是打包工具，不负责模块依赖图构建、资源合并、代码分包这些工作，这些通常由 `webpack`、`Vite/Rollup`、`esbuild` 等工具承担。
- 高频关键词：Babel、语法转换、语法降级、JSX、TypeScript、AST、插件、预设、`@babel/preset-env`、`@babel/preset-react`、`targets`、`polyfill`、`core-js`、运行时 API、编译器、兼容性

- 易追问：Babel 和 TypeScript 编译器的区别是什么？为什么 Babel 只能解决部分兼容问题？语法转换和 `polyfill` 有什么区别？`preset-env` 是怎么工作的？Babel 插件和 `preset` 的区别是什么？Babel 和打包工具有什么关系？为什么现在有些项目会用 `swc` 或 `esbuild` 替代 Babel？
- 回答雷区：不要把 Babel 说成打包工具，它本质是编译转换工具；不要把“语法降级”和“API 兼容”混成一件事，前者靠转换，后者常需要 `polyfill`；不要说 Babel 能做 TypeScript 类型检查，通常它只负责转换；不要把 Babel 拼成 `Bable`，面试里这个细节也容易被注意到。

## 30. 页面加载时间长，怎么判断和优化？

- 一句话结论：页面加载慢要先用指标和性能工具定位瓶颈，再按网络、资源、主线程、渲染和框架层几个维度做针对性优化，而不是一上来就盲目压缩代码。
- 30秒回答：我一般会先判断是“网络慢、资源大、主线程阻塞，还是渲染卡顿”。指标上重点看 `TTFB`、`FCP`、`LCP`、`CLS`、`INP`，流畅度会看 `FPS`；工具上主要用 Chrome DevTools 的 `Performance` 面板，看主线程长任务、`Layout Shifts`、`Layout/Paint` 峰值、图像解码和光栅化耗时。定位清楚后再分层优化，比如资源压缩、代码分割、图片格式升级、缓存和懒加载、减少首屏长任务、把高频动画改成 `transform/opacity`，以及在 Vue3 里做更细粒度的响应式和异步任务拆分。
- 1分钟展开：
  - 先看指标，不凭感觉判断：如果页面“感觉慢”，我会先看关键指标。`TTFB` 反映服务端和网络首包速度，`FCP` 反映首次内容出现时间，`LCP` 更接近用户感知的主要内容完成时间，`CLS` 看布局抖动，`INP` 看交互响应延迟，`FPS` 则用来辅助判断页面是否有掉帧卡顿。
  - 用 `Performance` 做定位：录制性能后，先看整体时间线，再看主线程是否有长任务。如果是长任务，就去 `Main` 轨道选中对应任务，在 `Bottom-up` 里找热点函数，在 `Call Tree` 里梳理调用链路，继续确认到底是第三方脚本、框架运行时，还是自己业务代码导致的阻塞。
  - 看渲染链路问题：如果怀疑是绘制或布局问题，会重点看 `Rendering` 相关轨道、`Layout`、`Paint`、`Composite` 是否有明显峰值，同时结合 `Layout Shifts` 轨道确认是否有严重布局偏移。如果是图片场景，还会看图像解码和栅格化是否异常耗时。
  - 资源优化：资源层面通常包括 JS、CSS、图片和字体。JS 可以压缩混淆，比如 `Terser`；CSS 可以压缩，比如 `cssnano`；图片要做压缩、尺寸裁剪、响应式多分辨率，并优先考虑 `webP`、`AVIF` 等更高压缩率格式。

- 请求与打包优化：可以减少无效 HTTP 请求，做代码分包、按路由或组件动态加载，配合浏览器缓存、协商缓存、本地存储、懒加载、预加载、预连接、预请求等手段，把首屏真正需要的资源优先送达。
- 主线程与首屏优化：如果瓶颈在主线程，就要减少首屏长任务，把大计算拆小，必要时放到 `web worker`；动画和高频更新尽量交给 `requestAnimationFrame` 协调；低优先级任务可以用 `requestIdleCallback`；视觉动画尽量优先用 `transform` 和 `opacity`，减少触发布局和重绘。
- 框架层补充：在 Vue3 里可以利用更细粒度的响应式降低无效更新，比如合理使用 `shallowRef`、异步组件、懒挂载、持久化缓存等；如果有重计算或数据转换，也可以考虑 `web worker`，避免阻塞主线程。
- 优化顺序：我通常会优先处理影响最大的几个瓶颈，比如首屏关键资源、主线程长任务、超大图片和高频重排，而不是把所有优化点平均用力，因为性能优化本质是投入产出比管理。
- 高频关键词：页面加载慢、性能指标、`TTFB`、`FCP`、`LCP`、`CLS`、`INP`、`FPS`、`Performance`、长任务、`Main` 轨道、`Bottom-up`、`Call Tree`、`Layout Shifts`、`Layout`、`Paint`、图像解码、栅格化、资源压缩、`Terser`、`cssnano`、`WebP`、`AVIF`、代码分割、懒加载、预加载、`requestAnimationFrame`、`requestIdleCallback`、`transform`、`opacity`、Vue3、`shallowRef`、`web worker`
- 易追问：`LCP` 和 `FCP` 的区别是什么？为什么 `TTFB` 高会拖累首屏？长任务怎么定义？`Bottom-up` 和 `Call Tree` 分别怎么看？为什么 `transform` 和 `opacity` 更适合动画？图片格式怎么选？什么时候该用 `web worker`？Vue3 里怎么减少无效渲染？`requestIdleCallback` 适合哪些场景？
- 回答雷区：不要只说“开 gzip、压图片”这种泛泛优化，先定位再优化更重要；不要把所有慢都归因到前端，`TTFB` 高可能是服务端或网络问题；不要把 `FPS` 当首屏指标，它更偏流畅度；不要把 `requestIdleCallback` 当高优先级调度工具；不要把 Vue3 的优化只理解成“换成 Vue3 就更快”，关键还是数据粒度、组件边界和任务拆分。

## 31. 讲讲 JavaScript 的原型链

- 一句话结论：JavaScript 的原型链本质上是对象之间基于原型的查找机制，访问一个属性时会先找对象自身，找不到就沿着 `[[Prototype]]` 一层层向上查，直到 `null` 为止。
- 30秒回答：在 JavaScript 里，每个对象内部都有一个 `[[Prototype]]`，通常可以通过 `__proto__` 或 `Object.getPrototypeOf()` 理解和访问。这个原型本身也是对象，也有自己的原型，于是就形成了原型链。属性查找时会先看对象自己，没有就沿着原型链往上找。实例对象的 `__proto__` 一般会指向构造函数的 `prototype`，所以我们常说“对象共享的方法都

挂在原型上”。`new`、`instanceof`、继承，本质上都和这条原型链有关。

- 1分钟展开：

- 什么是原型和原型链：对象内部有一个隐式原型 `[[Prototype]]`，它指向另一个对象。这个对象如果也有自己的原型，就继续往上连，最终形成一条链，最顶层通常是 `Object.prototype`，再往上就是 `null`。
- 属性查找机制：当你访问 `obj.x` 时，JS 引擎会先查 `obj` 自身有没有 `x`；如果没有，就去 `obj` 的原型上找；再没有就继续往上找，直到链顶为止。这也是为什么原型上的方法可以被多个实例共享。
- 实例和构造函数的关系：常见关系是 `实例对象.__proto__ === 构造函数.prototype`。也就是说，实例之所以能访问构造函数原型上的方法，是因为它的原型正好指向了那个 `prototype` 对象。
- 函数对象的特殊性：函数也是对象，所以函数本身也有 `__proto__`；同时函数还能被调用，因为它内部还有 `[[Call]]` 能力。作为构造函数时，函数又会配套有一个 `prototype` 属性，用来给将来创建出来的实例提供共享原型。
- **constructor** 关系：通常 `构造函数.prototype.constructor` 会指回这个构造函数，比如 `Function.prototype.constructor === Function`。但面试里也可以补一句，`constructor` 只是普通属性，可以被改写，不能把它当成绝对可靠的类型判断依据。
- **new** 做了什么：可以概括成四步。第一，创建一个新对象；第二，把这个对象的原型指向构造函数的 `prototype`，近似可理解为 `Object.create(Constructor.prototype)`；第三，把构造函数里的 `this` 绑定到这个新对象并执行初始化逻辑；第四，如果构造函数显式返回一个对象，就返回那个对象，否则返回新创建的对象。
- **instanceof** 原理：`a instanceof B` 本质上是在看 `B.prototype` 是否出现在 `a` 的原型链上。只要能沿着 `a` 的原型链找到 `B.prototype`，结果就是 `true`。
- 对象创建方式：常见有工厂函数、构造函数、原型模式、组合模式（构造函数 + 原型）、以及 `class` 写法。工程里组合模式比较常见，因为实例属性放构造函数里，方法放原型上，既清晰又节省内存。
- 继承方式：JS 继承本质是原型继承。历史上有构造函数继承、原型链继承、组合继承、寄生组合继承等；ES6 的 `class extends` 只是更现代的语法糖，底层还是在搭建原型链关系。
- **class** 为什么是语法糖：因为它底层仍然依赖两条原型链。一条是“实例对象 -> 子类原型 -> 父类原型”，另一条是“子类构造函数 -> 父类构造函数”，所以它没有改变 JS 基于原型的本质，只是把写法变得更接近传统面向对象语言。

- 高频关键词：原型、原型链、`[[Prototype]]`、`__proto__`、`Object.getPrototypeOf`、`prototype`、构造函数、属性查找、共享方法、`Function.prototype`、`constructor`、`new`、`Object.create`、`instanceof`、工厂函数、组合继承、`class`、`extends`、语法糖
- 易追问：`__proto__` 和 `prototype` 有什么区别？为什么函数既是对象又能调用？`new` 底层做了什么？`instanceof` 为什么能判断类型？`constructor` 靠不靠谱？为什么说 `class` 是语法糖？组合继承和寄生组合继承有什么区别？`Object.create()` 有什么作用？
- 回答雷区：不要把 `__proto__` 和 `prototype` 混为一谈，一个常指对象的隐式原型，一个是构造函数的原型属性；不要说“所有对象都有 `prototype`”，普通对象通常没有这个属性，函数对象才有；不要把 `instanceof` 说成比较 `constructor`，它本质看的是原型链；不要把 `class` 说成完全不同于原型的机制；不要把 `Function.prototype.constructor === Function` 这类关系讲对了，却把更基础的“实例对象.`proto` === 构造函数.`prototype`”说错。

## 32. 常见 HTTP 状态码

- 一句话结论：HTTP 状态码是服务端对请求处理结果的标准化表达，面试里重点掌握 `2xx` 成功、`3xx` 重定向、`4xx` 客户端错误、`5xx` 服务端错误，以及几个最常用的细分码。
- 30秒回答：HTTP 状态码大致分五类：`1xx` 是信息性状态，`2xx` 表示成功，`3xx` 表示重定向，`4xx` 表示客户端请求有问题，`5xx` 表示服务端处理失败。高频里最常见的是 `200` 成功、`201` 创建成功、`301/302` 重定向、`304` 资源未修改、`400` 参数错误、`401` 未认证、`403` 无权限、`404` 资源不存在、`500` 服务器异常、`502` 网关错误、`503` 服务不可用。
- 1分钟展开：
  - 状态码分类：`1xx` 多用于协议交互过程中的提示；`2xx` 表示请求被正常接收和处理；`3xx` 表示资源位置或缓存状态发生了跳转含义；`4xx` 说明问题出在请求方；`5xx` 说明问题主要出在服务端。
  - `2xx` 成功类：`200 OK` 表示普通成功；`201 Created` 常用于创建资源成功，比如新增用户、创建订单；如果是删除成功且无需返回内容，也常见 `204 No Content`。
  - `3xx` 重定向类：`301 Moved Permanently` 是永久重定向，适合域名迁移、旧地址永久废弃；`302 Found` 是临时重定向；`304 Not Modified` 比较特殊，它通常出现在协商缓存里，表示资源没有变化，客户端可以继续使用缓存副本。
  - `4xx` 客户端错误类：`400 Bad Request` 常表示参数或请求格式错误；`401 Unauthorized` 更准确是“未认证”或“认证失效”，常见于没带 `token` 或 `token` 过期；`403 Forbidden` 表示服务端理解请求，但当前用户没有权限访问；`404 Not Found` 表示资源不存在或路径不匹配。

- **5xx 服务端错误类**：**500 Internal Server Error** 表示服务端内部异常；**502 Bad Gateway** 常见于网关或反向代理转发上游服务失败；**503 Service Unavailable** 通常表示服务临时不可用，比如过载、维护、限流等。
- **怎么回答更像实战**：面试里不要只背数字，最好顺手带一下使用场景，比如**304**和缓存、**401/403**的区别、**502/503**常见于网关和服务治理场景，这样会更像真正用过。
- **高频关键词**：**HTTP 状态码**、**1xx**、**2xx**、**3xx**、**4xx**、**5xx**、**200**、**201**、**204**、**301**、**302**、**304**、**400**、**401**、**403**、**404**、**500**、**502**、**503**、协商缓存、重定向、认证、权限
- **易追问**：**301**和**302**的区别是什么？**304**为什么属于**3xx**？**401**和**403**有什么区别？为什么删除成功常返回**204**？**502**和**503**分别说明什么问题？状态码和业务码要不要同时存在？
- **回答雷区**：不要把**401**简单说成“未授权”，更准确通常是“未认证”；不要把**403**和**401**混为一谈；不要把**304**说成真正返回了资源内容，它表示可以继续用缓存；不要只背大类不讲场景，这样很容易显得只是死记硬背。

### 33. JWT 的优缺点

- **一句话结论**：JWT 是一种自包含的令牌格式，优点是天然适合无状态认证和分布式系统，缺点是令牌较大、泄露后难撤销，而且默认只是可验签，不代表内容加密。
- **30秒回答**：JWT 全称 **JSON web Token**，结构通常是 **header.payload.signature**。**header**里声明算法类型，**payload**里放用户标识、过期时间等声明，**signature**用来做签名校验，保证内容没被篡改。它的优点是无状态、跨服务传递方便、适合分布式；缺点是体积比 **sessionId** 大，泄露后在过期前很难立即撤销，而且 **payload** 默认只是可解码，不适合放敏感信息。
- **1分钟展开**：
  - **JWT 是什么**：它本质是一种令牌规范，不是某一种具体登录方案。服务端签发后，客户端后续请求带着这个 **token**，服务端根据签名和声明判断身份是否合法。
  - **三段结构**：**header**说明签名算法和类型；**payload**放声明信息，比如用户 ID、角色、过期时间、签发时间等；**signature**是用密钥对前两段做签名，用于校验 **token** 是否被篡改。
  - **优点一：无状态**：服务端不一定需要像 **Session** 那样保存每个用户的会话数据，只要能校验签名，就能完成基本认证。这在多服务、分布式、网关转发场景里比较方便。
  - **优点二：跨服务传递方便**：JWT 自带身份声明，服务之间转发时不一定都去查中心会话存储，所以很适合微服务、前后端分离、移动端等场景。

- 缺点一：体积偏大：JWT 通常比一个短的 `sessionId` 更长，每次请求都会增加传输成本，特别是声明多的时候更明显。
- 缺点二：撤销困难：如果 `token` 泄露，只要它还没过期，服务端通常很难像删 `Session` 一样立即让它失效，除非额外做黑名单、短有效期、双 `token`、刷新令牌轮换等补充机制。
- 缺点三：默认不加密：JWT 默认只是 `Base64URL` 编码加签，不等于加密，所以 `payload` 内容不是绝对保密的，不适合直接放密码、身份证号这类敏感信息。
- 适用建议：如果系统强调水平扩展、网关认证、跨服务传递，JWT 很方便；如果更看重会话可控、可随时失效、服务端集中管理，`Session` 或有状态 `token` 方案往往更稳。
- 高频关键词：JWT、`JSON web Token`、`header.payload.signature`、签名、声明、过期时间、无状态、分布式、认证、验签、体积大、撤销困难、`Base64URL`、黑名单、双 `token`、`Refresh Token`
- 易追问：JWT 为什么适合分布式？JWT 和 `Session` 的核心区别是什么？为什么说 JWT 默认不安全地存敏感信息？签名和加密有什么区别？JWT 泄露后怎么处理？为什么很多系统会搭配 `Refresh Token`？JWT 一定完全无状态吗？
- 回答雷区：不要说 JWT “安全性高”就结束了，更准确是“可验签、防篡改，但默认不加密”；不要把 `Base64` 编码当成加密；不要忽略“泄露后难撤销”这个核心缺点；不要把 JWT 讲成天然优于 `Session`，选型要看系统场景。

## 34. 如何隐藏一个元素？

- 一句话结论：隐藏元素的方式很多，区别主要在于它是否占据布局空间、是否仍在 `DOM` 中、是否还能响应事件，以及会触发哪些渲染开销。
- 30秒回答：最常见的几种方式里，`display: none` 会让元素不参与布局和渲染，相当于从页面流里移除；`visibility: hidden` 元素还占位置，但看不见；`opacity: 0` 元素还在布局里，也可能仍然响应事件；移到屏幕外适合某些特殊场景，但元素依然存在；`transform: scale(0)` 主要影响视觉，不会改变布局。Vue 里 `v-if` 是直接销毁和重建 `DOM`，`v-show` 本质是切换 `display`，不是 `visibility`。
- 1分钟展开：
  - **`display: none`**：元素不参与布局和绘制，页面上相当于没有这个节点对应的视觉占位。切换时通常会引起布局变化，常伴随重排。适合“确实不需要显示，也不需要占位”的场景。
  - **`visibility: hidden`**：元素仍然占据原来的布局空间，只是不可见。通常不会引起布局变化，但依然保留位置。适合“需要保留占位，但先不显示内容”的场景。

- **opacity: 0:** 元素依旧参与布局，也依旧存在于 DOM 中，只是完全透明。它通常仍可能响应点击和事件，所以如果不希望被交互，还要额外配合 **pointer-events: none**。常用于淡入淡出动画。
- **移出可视区域:** 比如绝对定位到屏幕外，元素依然存在于 DOM 中。因为 **position: absolute** 本身会脱离普通文档流，但这不是“隐藏”的本质，只是把它挪到看不见的位置。它常见于某些可访问性、测量尺寸、旧方案兼容场景。
- **transform: scale(0):** 它不会影响文档流，所以通常不会触发布局重排，更多是触发绘制或合成层更新。适合和动画一起使用，但元素本身依然存在，交互和可访问性要单独考虑。
- **v-if 和 v-show:** **v-if** 是条件不满足时直接不渲染节点，切换时会销毁和重建 DOM；**v-show** 是一直保留 DOM，只切换 CSS 的 **display**。所以如果切换频率高，通常 **v-show** 更合适；如果一开始就不一定会渲染，**v-if** 更省初始开销。
- **怎么选:** 如果你要“彻底不占位”，优先 **display: none** 或 **v-if**；如果要“占位但不显示”，用 **visibility: hidden**；如果要做过渡动画，常用 **opacity** 或 **transform**；如果担心误触，要额外考虑事件穿透和可访问性。
- 高频关键词：隐藏元素、**display: none**、**visibility: hidden**、**opacity: 0**、**position: absolute**、移出屏幕、**transform: scale(0)**、布局、重排、重绘、合成、DOM、事件响应、**pointer-events**、**v-if**、**v-show**
- 易追问：**display: none** 和 **visibility: hidden** 的区别是什么？为什么 **opacity: 0** 还能点到？**transform** 为什么通常比改布局属性更适合动画？**v-if** 和 **v-show** 的使用场景怎么选？移出屏幕和真正隐藏有什么区别？无障碍场景下隐藏元素要注意什么？
- 回答雷区：不要把 **v-show** 说成 **visibility: hidden**，它本质是切换 **display**；不要说 **transform: scale(0)** 会触发回流，它通常不影响布局；不要把 **opacity: 0** 说成“元素不存在”，它仍然占位且可能可交互；不要把“绝对定位移出屏幕”理解成标准隐藏方案，它更多是特殊场景技巧。

## 35. 如何判断数据类型？

- 一句话结论：JavaScript 判断数据类型要分场景选择工具，基础类型常用 **typeof**，引用类型常结合 **Array.isArray()**、**instanceof** 和 **Object.prototype.toString.call()** 来判断。
- 30秒回答：类型判断最常见有四种。**typeof** 适合判断基础类型和函数，但 **typeof null** 会得到 **object**；**instanceof** 适合判断对象是否在某个构造函数的原型链上；数组推荐直接用 **Array.isArray()**；如果想更稳定地区分对象类型，通常用 **Object.prototype.toString.call(value)**。面试里一般会总结成一句：基础类型优先 **typeof**，对象细分优先 **toString**，数组优先 **Array.isArray**。

- 1分钟展开：
  - **typeof**: 适合快速判断 `string`、`number`、`boolean`、`undefined`、`symbol`、`bigint` 和 `function`。它的经典坑是 `typeof null === 'object'`，这是历史遗留问题。
  - **instanceof**: 本质上是看右侧构造函数的 `prototype` 是否出现在左侧对象的原型链上，所以它更适合判断引用类型，比如数组、日期、正则、自定义实例。它不适合准确判断基础类型字面量，比如 `1 instanceof Number` 是 `false`。
  - **Array.isArray()**: 判断数组最直接，也比 `instanceof Array` 更稳，尤其在跨 `iframe`、跨上下文场景下更可靠。
  - **Object.prototype.toString.call()**: 这是判断对象细分类型的经典方式，它会返回类似 `[object Array]`、`[object Date]`、`[object RegExp]` 的结果。相比 `typeof`，它更适合区分各种内置对象。
  - 怎么选工具: 如果只想区分是不是基础类型，用 `typeof` 就够；如果要判断某个对象是否来自某个构造函数体系，用 `instanceof`；如果是数组，用 `Array.isArray()`；如果要做更通用、更稳的类型检测，优先 `Object.prototype.toString.call()`。
  - 工程实践补充: 很多项目会封装一个 `getType` 方法，本质也是基于 `Object.prototype.toString.call(value)` 再做字符串提取和统一处理。
- 高频关键词: 数据类型、`typeof`、`null`、`instanceof`、原型链、`Array.isArray()`、`Object.prototype.toString.call()`、基础类型、引用类型、数组、跨 `iframe`、类型判断
- 易追问: 为什么 `typeof null` 是 `object`? `instanceof` 的原理是什么? 为什么数组推荐 `Array.isArray()`? `Object.prototype.toString.call()` 为什么更稳? 基础类型包装对象会不会影响判断? 跨 `iframe` 场景下为什么 `instanceof Array` 可能不准?
- 回答雷区: 不要把 `instanceof` 说成万能类型判断，它更适合引用类型和原型链判断；不要说 `typeof` 能精确区分所有对象类型；不要忘记 `typeof null` 这个经典陷阱；不要把 `Array.isArray()` 和 `instanceof Array` 当成完全等价。

## 36. toString() 的原理

- 一句话结论: `toString()` 本质上是对象转字符串时调用的方法，默认的 `Object.prototype.toString()` 会根据内部类型标签返回标准结果，而很多内置对象又会重写自己的 `toString()` 行为。

- 30秒回答: `toString()` 不能只理解成“转字符串”这么简单。默认情况下, `Object.prototype.toString.call(value)` 会返回像 `[object Array]`、`[object Date]` 这样的类型标签, 所以它常被用来做类型判断; 而像数组、函数、日期等对象, 通常又各自重写了 `toString()`, 让返回值更符合自己的语义。所以面试里要区分“默认的 `Object.prototype.toString`”和“对象自己重写后的 `toString`”。
- 1分钟展开:
  - 默认 `toString`: `Object.prototype.toString()` 是所有普通对象都能沿原型链拿到的方法。它的经典返回格式是 `[object Type]`, 其中 `Type` 来自对象内部的类型标识。
  - 为什么类型判断常用它: 因为直接调用 `obj.toString()` 可能会被对象自身重写, 所以做类型判断时常写成 `Object.prototype.toString.call(obj)`, 强行借用最原始的那个实现, 避免被覆盖。
  - 内置对象为什么会重写: 很多内置对象会提供更符合业务语义的字符串结果。比如数组的 `toString()` 更接近元素拼接后的字符串, 函数的 `toString()` 常返回源码文本, 日期对象也有自己的字符串格式化表现。
  - 底层标签来源: 现代规范里, 它和内部类型以及 `Symbol.toStringTag` 有关系, 所以某些对象的 `[object xxx]` 标签也可以被自定义或影响。面试里不一定非要展开到很深, 但知道它不是纯手写字符串就够了。
  - 使用场景: 一个是隐式类型转换, 比如对象参与字符串拼接时可能会触发; 另一个就是显式类型判断, 特别是区分数组、日期、正则、`Map`、`Set` 这类对象时很常见。
- 高频关键词: `toString()`、`Object.prototype.toString`、类型标签、`[object Type]`、`call`、类型判断、数组重写、函数重写、日期重写、`Symbol.toStringTag`、隐式类型转换
- 易追问: 为什么类型判断要写 `Object.prototype.toString.call(obj)`? 数组的 `toString()` 和对象默认的 `toString()` 有什么区别? `Symbol.toStringTag` 是做什么的? 对象在字符串拼接时会发生什么? `toString()` 和 `valueOf()` 有什么关系?
- 回答雷区: 不要把所有 `toString()` 的行为都当成 `Object.prototype.toString()`; 不要说 `toString()` 只负责类型判断, 它本质上是字符串表示方法; 不要忽略很多内置对象会重写这个方法; 不要把 `Object.prototype.toString.call()` 和直接 `obj.toString()` 混用。

## 37. Webpack 的作用

- 一句话结论: `Webpack` 是以前端模块为核心的打包构建工具, 它会从入口出发分析依赖图, 把开发阶段的多种资源处理并产出浏览器可运行的静态资源。

- 30秒回答：Webpack 本质上是一个前端工程化工具。它会从 `entry` 入口出发，递归分析项目依赖，把 JavaScript、CSS、图片等资源都当成模块处理，再通过 `loader` 和 `plugin` 完成转换、优化和打包，最后输出浏览器可以直接使用的文件。它解决的核心问题包括模块化、兼容性、构建优化和生产环境性能优化，比如压缩、代码分割、Tree Shaking 等。
- 1分钟展开：
  - 它解决什么问题：前端项目里不仅有 JS，还有 CSS、图片、字体、TypeScript、JSX 等资源，浏览器本身并不能直接理解所有开发态写法。Webpack 的作用就是把这些资源统一纳入构建流程，最终输出可部署的静态产物。
  - 模块为核心：Webpack 的核心思想是“一切皆模块”。它会把项目中引用到的资源都视为模块，再从入口开始构建一张依赖图，最后根据依赖关系输出 bundle 或多个 chunk。
  - 核心配置：最常见的是 `entry`、`output`、`module.rules`、`plugins`、`mode`。`entry` 决定从哪里开始分析，`output` 决定产物输出方式，`loader` 负责把非 JS 资源或新语法转换成 Webpack 能处理的模块，`plugin` 负责扩展构建生命周期能力。
  - `loader` 和 `plugin` 区别：`loader` 更像资源转换器，通常一个资源会经过一串 `loader`，并且执行顺序通常是从后往前；`plugin` 则能介入更广的构建流程，比如注入环境变量、提取 CSS、生成 HTML、压缩资源、分析包体积等。
  - 工程化与兼容性：Webpack 常和 Babel、PostCSS、TypeScript 等工具配合使用，解决语法降级、样式处理、模块化拆分等工程化问题。
  - 性能优化能力：在生产环境里，Webpack 可以做代码压缩、Tree Shaking、代码分割、懒加载、缓存优化、资源指纹命名等。开发阶段则常关注 Source Map、热更新和增量编译体验。
  - 常见优化配置：比如 `devtool: 'eval-cheap-module-source-map'` 兼顾开发调试速度；`splitChunks` 处理公共依赖拆分；`watchOptions` 可以优化监听行为；生产环境还会关注缓存、并行压缩和构建性能。
- 高频关键词：Webpack、打包工具、工程化、模块化、依赖图、`entry`、`output`、`loader`、`plugin`、`mode`、bundle、chunk、Babel、Tree Shaking、代码分割、`splitChunks`、Source Map、热更新、构建优化
- 易追问：Webpack 为什么说“一切皆模块”？`loader` 和 `plugin` 的区别是什么？为什么 `loader` 从后往前执行？`entry` 和 `output` 分别做什么？什么是 Tree Shaking？`splitChunks` 解决什么问题？开发和生产配置为什么不同？
- 回答雷区：不要把 Webpack 只说成“打包 JS”，它处理的是整个前端资源构建链路；不要混淆 `loader` 和 `plugin`；不要把 Tree Shaking 说成 Webpack 自动对所有代码都生效，它通常依赖 ES Module 和生产模式；不要只背配置名，不解释它们分别解决什么问题。

## 38. Vite 为什么快？

- 一句话结论：Vite 开发阶段快的核心原因是“不先整体打包，而是基于原生 ES Module 按需提供模块”，再配合高性能编译工具和依赖预构建降低启动与更新成本。
- 30秒回答：Vite 快主要体现在开发阶段。传统打包工具启动时通常会先把整个项目依赖打成 bundle，而 Vite 采用更接近 bundleless 的思路，先启动一个开发服务器，浏览器请求哪个模块就返回哪个模块，所以冷启动更快。它还会用 esbuild 做依赖预构建和部分代码转换，速度明显快于传统基于 Babel 的完整编译链。到了生产构建阶段，Vite 通常再交给 Rollup 来做正式打包，因此既保留了开发体验，也兼顾了生产产物能力。
- 1分钟展开：
  - 开发阶段快的原因一：按需加载：Vite 在开发时不会先把整个项目都打成一个 bundle，而是启动 dev server 后，浏览器通过原生 ES Module 去请求各个模块。也就是说，你打开哪个页面、用到哪些模块，就只处理那些模块。
  - 原因二：冷启动成本低：传统工具可能在项目一启动时就要遍历和编译整张依赖图，项目越大，启动越慢；Vite 启动阶段更像“先把服务起起来”，再按需处理模块，所以冷启动通常更轻。
  - 原因三：esbuild 很快：Vite 会用 esbuild 做依赖预构建和一部分转换工作。esbuild 是 Go 实现，速度非常快，所以在依赖较多的项目里，开发体验通常会比传统 Babel 链更轻快。面试里可以说“明显更快”，没必要死背固定倍数。
  - 依赖预构建：Vite 会提前把第三方依赖转成浏览器更容易处理的 ESM 形式，并做一定整合，这样浏览器后续请求时不会被一堆深层依赖拖慢。
  - 热更新更轻量：因为开发时不是整包重建，所以文件改动后通常只需要重新处理受影响的模块，热更新链路更短，反馈更快。
  - 生产阶段怎么做：Vite 并不是完全不打包。到了生产构建阶段，通常还是交给 Rollup 去做正式打包、代码分割和产物优化。Rollup 在 ESM 场景下生态成熟，适合产出质量较高的前端静态资源。
  - 怎么更完整地回答：一句话概括就是，Vite 把“开发时的快”和“生产时的稳”拆开处理了，开发阶段靠原生 ESM + 按需加载 + esbuild，生产阶段靠 Rollup 打包优化。
- 高频关键词：Vite、快、dev server、原生 ES Module、按需加载、bundleless、冷启动、esbuild、依赖预构建、热更新、HMR、生产构建、Rollup、代码分割、开发体验
- 易追问：Vite 为什么冷启动快？什么叫 bundleless？为什么 esbuild 比 Babel 快很多？依赖预构建解决什么问题？Vite 的 HMR 为什么快？为什么生产阶段还要交给 Rollup？Vite 和 Webpack 适合哪些不同场景？
- 回答雷区：不要把 Vite 说成“完全不打包”，生产环境仍然会打包；不要把它快的原因只归结为 esbuild，按需加载和原生 ESM 也很关键；不要死背“快多少倍”这种固定数字；不要忽略开发阶段快和生产阶段构建是两套思路。

## 39. TypeScript 的作用

- 一句话结论：TypeScript 是 JavaScript 的超集，它在不改变 JS 运行本质的前提下，引入静态类型系统和更强的工程化能力，帮助我们更早发现错误、提高代码可维护性。
- 30秒回答：TypeScript 可以理解成“带类型系统的 JavaScript”。JavaScript 本身很灵活，但大型项目里也容易出现变量类型混乱、对象属性拼写错误、函数参数传错、全局污染难排查等问题。TypeScript 通过静态类型检查把很多问题提前到开发和编译阶段暴露出来。它最终会通过 `tsc` 等工具编译成普通 JavaScript 运行，所以本质上是在增强开发体验和工程质量。
- 1分钟展开：
  - 为什么需要 **TypeScript**：JavaScript 动态性强，开发效率高，但项目一大就容易出现“变量被重复赋不同类型”“访问了不存在的属性”“参数顺序和类型传错却运行时才报错”等问题。TypeScript 的价值就是把这些问题的前置。
  - 它的核心作用：第一是静态类型检查，第二是更好的编辑器提示和自动补全，第三是让更多人协作时接口和数据结构更清晰，第四是让重构更安全。面试里可以总结成一句：TS 主要提升的是大型工程的可维护性和可预测性。
  - 编译过程：TypeScript 代码不能直接在浏览器里运行，通常需要通过 `tsc` 编译成 JavaScript。开发中可以用 `tsc --watch` 持续监听编译，实际项目里也常由 `Vite`、`Webpack`、`ts-loader`、`esbuild` 等工具接入构建链路。
  - 常见类型：高频类型包括 `void`、`unknown`、`never`、`any`、元组 `tuple`、枚举 `enum` 等。`any` 表示放弃类型检查；`unknown` 更安全，使用前必须先收窄；`never` 表示不会有值，比如抛异常函数或穷尽判断后的不可能分支；`tuple` 适合“长度固定且每个位置类型明确”的数组。
  - **enum** 枚举：枚举适合表示一组有限常量。数字枚举支持双向映射；字符串枚举更直观；`const enum` 在编译时通常会被直接内联成值，减少运行时代码体积，但也要注意构建工具和调试体验的兼容性。
  - **type** 的作用：`type` 是类型别名，适合给联合类型、交叉类型、元组、字面量类型等起名字，比如 `type status = 'idle' | 'loading' | 'done'`。它更偏灵活的类型表达能力。
  - **interface** 的作用：`interface` 更常用于描述对象结构和类契约，也能被 `class implements` 使用，还支持 `extends` 继承和同名自动合并，这在声明扩展和类型补充时很有用。
  - **type** 和 **interface** 怎么选：如果主要描述对象结构，两者都能做；如果要表达联合类型、交叉类型、条件组合，`type` 更灵活；如果要做接口继承、类实现、声明合并，`interface` 往往更自然。工程里常见做法是对象结构偏向 `interface`，复杂组合类型偏向 `type`。

- 高频关键词：TypeScript、JavaScript 超集、静态类型检查、`tsc`、`tsc --watch`、工程化、类型系统、`enum`、`void`、`unknown`、`never`、`any`、`tuple`、`type`、联合类型、交叉类型、`interface`、`implements`、`extends`、声明合并
- 易追问：TypeScript 为什么适合大型项目？`any` 和 `unknown` 有什么区别？`never` 常见于什么场景？`type` 和 `interface` 的区别是什么？为什么说 `interface` 支持自动合并？`enum` 有哪些类型？为什么有些团队不推荐大量使用 `enum`？TS 最终为什么还要编译成 JS？
- 回答雷区：不要把 TypeScript 说成“运行时类型检查”，它主要是编译时静态检查；不要把 `unknown` 和 `any` 混为一谈，`unknown` 更安全；不要把 `type` 和 `interface` 简化成“完全一样”，它们有交集但也有侧重点；不要把 TS 的价值只说成“防报错”，它更大的价值是可维护性、可重构性和协作效率。

## 40. let 和 const 的作用

- 一句话结论：`let` 和 `const` 是 ES6 引入的块级作用域变量声明方式，用来替代 `var` 的很多历史问题，其中 `let` 可重新赋值，`const` 不能重新赋值。
- 30秒回答：`let` 和 `const` 最重要的意义，不只是“一个能改一个不能改”，而是它们都拥有块级作用域，能避免 `var` 只有函数作用域和全局作用域带来的很多问题，比如变量污染、循环闭包混乱等。它们还存在暂时性死区，也就是在声明前不能访问。`const` 表示变量绑定不能重新赋值，但如果绑定的是对象，对象内部属性仍然可以改。
- 1分钟展开：
  - 为什么要有 `let` 和 `const`：`var` 是早期 JS 的变量声明方式，只有函数作用域和全局作用域，容易造成变量提升、重复声明、作用域污染等问题。`let` 和 `const` 的出现，就是为了让变量作用域更清晰、行为更可控。
  - 块级作用域：`let` 和 `const` 都属于块级作用域，像 `if`、`for`、`while`、花括号代码块内部声明的变量，出了块就访问不到了；而 `var` 不认普通代码块，只认函数作用域和全局作用域。
  - `let` 和 `const` 的区别：`let` 可以重新赋值，适合后续值会变化的变量；`const` 不能重新绑定，适合不希望变量引用被改掉的场景。要注意，`const` 保证的是“绑定不变”，不是“值绝对不可变”，所以对象和数组内部内容仍然能改。
  - 暂时性死区：`let` 和 `const` 从进入作用域开始，到真正执行声明语句之前，这段区域叫暂时性死区。在这个阶段访问变量会直接报错，而不是像 `var` 那样得到 `undefined`。

- **变量提升怎么理解**：面试里更稳妥的说法是：`var`、函数声明、`let`、`const` 都会在编译阶段被登记到作用域里，但表现不一样。函数声明在其作用域内通常可以提前使用；`var` 被提升后初始化为 `undefined`；`let` 和 `const` 虽然也会被创建绑定，但在声明前处于暂时性死区，不能访问。
- **函数声明和 `var` 重名**：如果同一作用域里函数声明和 `var` 同名，初始化阶段通常函数声明优先，所以在赋值前访问时更像拿到函数本身；但后续如果 `var` 再赋值，会把这个名字覆盖掉。
- **实践建议**：工程里通常优先用 `const`，只有明确需要重新赋值时再用 `let`，尽量少用 `var`。这样代码意图更清晰，也更利于维护。
- **高频关键词**：`let`、`const`、`var`、块级作用域、函数作用域、全局作用域、词法环境、暂时性死区、TDZ、变量提升、函数声明、重复声明、作用域污染、重新赋值、绑定不变
- **易追问**：`let` 和 `const` 的核心区别是什么？为什么说 `const` 定义的对象还能改？什么是暂时性死区？`let/const` 到底会不会提升？为什么 `for + let` 更适合闭包场景？函数声明和 `var` 同名时谁生效？
- **回答雷区**：不要把 `const` 说成“完全不可变”，更准确是变量绑定不可重新赋值；不要说 `let/const` “完全没有提升”，更准确是它们也会在作用域中被创建，但声明前不能访问；不要把块级作用域和函数作用域混为一谈；不要只回答“`let` 能改、`const` 不能改”，这样太浅。

## 41. 移动端适配方案

- **一句话结论**：移动端适配本质上是在不同屏幕尺寸和像素密度下，让页面布局和视觉保持可用且稳定，常见方案包括媒体查询、相对单位、弹性布局和栅格布局。
- **30秒回答**：移动端适配常见有几类。第一类是媒体查询，根据不同断点切换样式；第二类是相对单位，比如 `rem`、`em`、`vw/vh`，让尺寸跟随根字体或视口变化；第三类是布局方案，比如 `flex` 和 `grid`，让页面天然具备弹性。实际项目里通常不是只选一种，而是“媒体查询 + 相对单位 + 弹性布局”组合使用。
- **1分钟展开**：
  - **媒体查询**：通过 `@media` 按屏幕宽度、方向、分辨率等条件切换样式，是最直观的响应式手段。常见断点会按小屏、中屏、大屏去分，比如你说的 `sm / md / xl` 思路，本质就是不同宽度区间用不同布局规则。
  - **`rem / em` 适配**：`rem` 相对根元素字体大小，适合做整体缩放型适配；`em` 相对当前元素字体大小，更适合局部相对尺寸。移动端里常见做法是根据视口宽度动态设置根字体大小，再用 `rem` 驱动组件尺寸联动。

- **vw / vh:** **vw**、**vh** 相对视口宽高，天然适合响应式场景。比如 **1vw** 等于视口宽度的 **1%**，适合做全屏布局、响应式字号和间距。但也要注意某些移动端浏览器地址栏变化会影响视口高度表现。
- **flex 布局:** **flex** 特别适合一维布局，比如横向导航、两栏布局、按钮组、列表项分配空间。像 **flex: 1** 就常用来让元素占据剩余空间，是移动端最常见的基础布局方式之一。
- **grid 布局:** **grid** 更适合二维布局，尤其是卡片列表、瀑布式规则栅格、自适应列布局。比如 **repeat(auto-fit, minmax(200px, 1fr))** 能根据容器宽度自动决定列数，非常适合响应式页面。
- **工程实践:** 真实项目通常会把几种方案组合起来，比如整体断点靠媒体查询，尺寸单位用 **rem/vw**，局部结构用 **flex/grid**，再配合图片自适应、最大宽度限制和设计稿规范一起落地。
- **高频关键词:** 移动端适配、响应式、媒体查询、断点、**rem**、**em**、**vw**、**vh**、视口、**flex**、**flex: 1**、**grid**、**auto-fit**、**minmax()**、**1fr**、响应式布局
- **易追问:** **rem** 和 **vw** 各适合什么场景？为什么移动端常配合根字体方案？媒体查询断点怎么定？**flex** 和 **grid** 的区别是什么？**auto-fit** 和 **minmax()** 怎么理解？为什么有些项目现在更偏向 **vw** 方案？
- **回答雷区:** 不要把移动端适配理解成只调宽高，它本质是布局、单位、内容和交互的综合适配；不要说 **rem**、**vw** 只能二选一，项目里常常组合使用；不要把 **flex** 和 **grid** 的适用场景混掉，前者更偏一维，后者更偏二维。

## 42. BFC 是什么？

- **一句话结论:** BFC 是块级格式化上下文，可以理解成一个独立的布局区域，内部元素的排版规则主要在这个区域内生效，不会轻易影响外部布局。
- **30秒回答:** BFC 全称 **Block Formatting Context**，中文叫块级格式化上下文。你可以把它理解成页面中的一个独立容器，容器内部的布局计算相对封闭，外面的元素不会和里面随意互相干扰。面试里最常问两个点：一是怎么触发 BFC，二是它能解决什么问题，比如清除浮动、防止外边距重叠、避免元素被浮动覆盖。
- **1分钟展开:**
  - **BFC 是什么:** 它是一种布局规则环境，块级盒子在这个上下文里按特定规则排列。最实用的理解不是背定义，而是记住“BFC 是一个相对独立的布局容器”。

- 怎么触发 BFC：常见方式包括设置 `overflow` 为 `hidden/auto/scroll`、设置 `display: flow-root`、浮动元素、绝对定位元素、某些 `inline-block`、`table-cell`、`flex` 项、`grid` 容器等。实际开发里最常见的触发方式通常是 `overflow: hidden` 或 `display: flow-root`。
- 作用一：清除浮动影响：父元素如果不建立 BFC，内部全是浮动子元素时，高度可能塌陷；让父元素形成 BFC 后，它就能把内部浮动元素包起来，这是一种常见清浮动思路。
- 作用二：避免外边距重叠：普通块级元素垂直方向的 `margin` 可能发生折叠；如果放进不同的 BFC 中，外边距折叠问题通常就不会互相影响。
- 作用三：避免被浮动元素覆盖：如果一个普通块级元素和旁边浮动元素发生覆盖或环绕问题，让它建立 BFC 后，它会形成一个独立区域，从而和浮动元素更清晰地分开。
- 怎么回答更实战：面试里可以一句话总结成“BFC 常用于清浮动、解决 `margin` 重叠、隔离布局影响”，然后再补一个最熟悉的触发方式就够了。
- 高频关键词：BFC、`Block Formatting Context`、独立布局区域、`overflow: hidden`、`display: flow-root`、浮动、绝对定位、`inline-block`、`table-cell`、清除浮动、`margin` 重叠、布局隔离
- 易追问：BFC 怎么触发？为什么 `overflow: hidden` 能形成 BFC？BFC 为什么能清除浮动影响？`margin` 重叠是什么？`display: flow-root` 和 `overflow: hidden` 哪个更语义化？BFC 和普通块级容器有什么区别？
- 回答雷区：不要把 BFC 说成某个具体 CSS 属性，它是一种布局上下文；不要只背定义不说用途，面试官更想听你能解决什么问题；不要把“内容不对外部产生任何影响”说得太绝对，更准确是布局计算相对独立、影响被隔离。

## 43. 如何实现居中？

- 一句话结论：元素居中要先区分是水平居中、垂直居中还是水平垂直同时居中，最常用方案是 `flex`、`grid`、绝对定位配合 `transform`，以及定宽定高场景下的 `margin: auto`。
- 30秒回答：现在实现居中最推荐的是 `flex` 和 `grid`。`flex` 下父元素用 `justify-content: center` 做主轴居中，`align-items: center` 做交叉轴居中；`grid` 可以直接用 `place-items: center`。如果是弹窗这类脱离文档流场景，也常用绝对定位 `top: 50%; left: 50%; transform: translate(-50%, -50%)`。如果元素尺寸已知，还可以配合 `top/right/bottom/left: 0` 和 `margin: auto` 做居中。
- 1分钟展开：

- **flex 居中**：这是最常见方案。父元素设成 `display: flex`，水平居中通常用 `justify-content: center`，垂直居中通常用 `align-items: center`。如果是列方向布局，主轴和交叉轴的理解要跟着 `flex-direction` 变化。
- **grid 居中**：`grid` 在做单个元素居中时也很方便，可以用 `justify-items`、`align-items`，或者更简洁地用 `place-items: center`。如果是整个容器内容居中，也可以用 `justify-content`、`align-content`。
- **绝对定位 + transform**：对于弹窗、浮层、遮罩层中的内容，常见写法是子元素 `position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%)`。它的优点是不依赖父容器布局流，通用性强。
- **四边为 0 + margin auto**：如果元素有明确宽高，也可以设置绝对定位并让 `top/right/bottom/left: 0; margin: auto;`，浏览器会在四个方向自动分配剩余空间，实现居中。
- **文本和行内元素居中**：如果面试官补问简单场景，还可以提 `text-align: center` 适合水平居中文本和行内元素；单行文本垂直居中有时可借助 `line-height` 等于容器高度，但这是偏特定场景的技巧。
- **怎么选方案**：现代布局优先选 `flex/grid`；脱离文档流的弹窗类元素适合绝对定位配合 `transform`；已知尺寸的盒子可以用 `margin: auto`；文本类居中则用 `text-align` 或行高技巧。
- **高频关键词**：居中、水平居中、垂直居中、`flex`、`justify-content`、`align-items`、`grid`、`place-items`、绝对定位、`transform: translate(-50%, -50%)`、`margin: auto`、`text-align: center`、`line-height`
- **易追问**：`justify-content` 和 `align-items` 分别控制什么？为什么 `transform` 方案常用于弹窗？`grid` 和 `flex` 哪个更适合居中？`margin: auto` 为什么能居中？如果元素宽高未知怎么办？单行文本和多行文本的居中方案有什么差别？
- **回答雷区**：不要把 `justify-content` 和 `align-items` 的方向说反；不要写成 `align-content: center` 就以为等于所有垂直居中，很多场景它并不是你要的属性；不要漏掉“不同场景选不同方案”这一点；不要把 `top: 50%; left: 50%` 当成完整居中方案，通常还要配合 `transform` 修正自身宽高偏移。

## 44. 写 CSS 遇到问题，怎么排查？

- 一句话结论：CSS 问题排查的核心不是盲改样式，而是按“有没有命中、为什么没生效、是不是受父级和布局上下文影响、是不是被裁剪或遮挡”这条链路逐步定位。

- 30秒回答：如果 CSS 和预期不一致，我一般先打开 DevTools 的 **Elements** 面板，看样式有没有命中目标元素。没命中就先查选择器、优先级、文件是否引入、类名是否写对；命中了但没生效，就看属性值是否合法、当前元素类型是否支持这个属性、有没有被别的样式覆盖。再往下就看父元素和布局上下文，比如宽高、定位、**flex/grid** 对齐、**overflow** 裁剪、透明度、层级和缩放。我的排查顺序通常是“命中 -> 生效 -> 布局 -> 可见性”。
- 1分钟展开：
  - 第一步：看有没有命中元素：先去 **Elements** 面板选中目标节点，看对应的选择器和样式规则有没有出现在右侧样式面板里。如果压根没出现，通常就是选择器写错、类名没挂上、样式文件没引入、作用域限制导致没匹配到，或者写错了层级位置。
  - 第二步：看是否被覆盖：如果样式出现了但被划线，通常说明它被更高优先级规则覆盖了。这时要检查选择器优先级、样式引入顺序、是否用了内联样式、是否存在 **!important**，以及 CSS Modules、Scoped CSS 等局部作用域的影响。
  - 第三步：看属性值是否合法：有些情况不是没命中，而是值本身无效，比如单位写错、拼写错误、属性和取值不匹配。也可能是当前元素类型不支持这个表现，比如对 **inline** 元素设宽高不一定如你预期，这时可以先加一些基础样式验证元素当前行为。
  - 第四步：看父元素和布局上下文：很多 CSS 问题其实不在当前元素，而在父级。比如元素为什么没有居中、为什么宽高不对、为什么定位偏了，往往要看父元素是不是 **flex**、**grid**、绝对定位容器、有没有明确宽高、是否建立了 BFC、是否有 **overflow** 或 **transform** 影响布局参考系。
  - 第五步：看可见性问题：样式都对，但页面还是看不到时，要排查是否被裁剪、遮挡、透明、缩放到很小、层级太低，或者跑到了视口外。常见办法是临时加背景色、边框、固定宽高、提高 **z-index**，快速判断元素到底在不在、在哪、有没有被盖住。
  - 第六步：结合 **DevTools** 辅助：可以看盒模型、计算后样式（Computed）、布局高亮、层叠来源、**flex/grid** 调试辅助线。复杂问题时，我会先把场景最小化，只保留相关结构和样式，缩小排查范围。
  - 排查思路总结：我一般不靠猜，而是按“命中没命中、覆盖没覆盖、值对不对、父级限制、可见性问题”这五步去拆。这样多数 CSS 问题都能比较快定位到根因。
- 高频关键词：CSS 排查、**Elements**、命中、选择器、优先级、覆盖、**!important**、样式引入顺序、属性值合法性、元素类型、父元素、布局上下文、**flex**、**grid**、定位、**overflow**、裁剪、遮挡、透明度、缩放、**z-index**、Computed、盒模型

- 易追问：为什么样式命中了却不生效？CSS 优先级怎么判断？什么情况下是父元素的问题？`overflow: hidden` 会带来哪些表现？`z-index` 为什么不生效？如何快速判断元素是不是被遮挡？Scoped CSS 或 CSS Modules 会带来什么排查特点？
- 回答雷区：不要一上来就说“多半是优先级问题”，很多 CSS 问题其实是布局上下文或父元素约束；不要只看源码不看 DevTools，样式排查最有效的工具就是浏览器面板；不要忽略元素类型差异，比如 `inline`、`block`、定位元素、`flex item` 的表现差异；不要遇到问题就先加 `!important`，那通常只是在掩盖原因。

## 45. async 和 defer 的区别

- 一句话结论：`async` 和 `defer` 都能让脚本下载不阻塞 HTML 解析，但执行时机不同，`async` 更强调“下载完立刻执行”，`defer` 更强调“等 HTML 解析完成后按顺序执行”。
- 30秒回答：普通 `script` 默认会阻塞 HTML 解析。加了 `async` 后，脚本下载和 HTML 解析可以并行进行，但脚本一旦下载完成就会立即执行，所以执行顺序不保证；加了 `defer` 后，脚本同样异步下载，但会等 HTML 全部解析完成后再按文档顺序执行，并且所有 `defer` 脚本执行完之后才触发 `DOMContentLoaded`。所以如果脚本依赖 DOM 结构，通常 `defer` 更稳。
- 1分钟展开：
  - 普通 `script`：默认情况下，浏览器解析 HTML 遇到普通 `script` 会暂停解析，先下载并执行脚本，执行完再继续往下解析。这就是为什么它容易阻塞页面解析和首屏展示。
  - `async` 的特点：`async` 脚本下载时不阻塞 HTML 解析，但它一旦下载完成，就会立刻中断当前解析去执行。所以它适合彼此独立、不依赖 DOM 完整结构、也不依赖其他脚本顺序的场景，比如埋点、统计、广告脚本。
  - `defer` 的特点：`defer` 脚本也会异步下载，但不会在下载完就立刻执行，而是等 HTML 解析完成后再执行。多个 `defer` 脚本会按它们在文档中的顺序执行，这一点对主业务脚本非常重要。
  - 和 `DOMContentLoaded` 的关系：`DOMContentLoaded` 会在 HTML 解析完成且所有 `defer` 脚本执行完之后触发，所以如果脚本需要操作 DOM，`defer` 通常比普通 `script` 和 `async` 更适合作为主逻辑脚本加载方式。
  - 和 CSS 的关系：`defer` 本身不会阻塞 HTML 解析，但它并不是和 CSS 完全无关。如果 `defer` 脚本执行时依赖样式计算、布局信息或相关渲染结果，浏览器可能会等 CSSOM 准备好后再执行相关逻辑，所以 CSS 仍可能间接影响脚本执行时机和首屏体验。
  - 怎么选：如果脚本完全独立，只求尽快执行，可以考虑 `async`；如果脚本属于页面主逻辑、依赖 DOM、还可能依赖其他脚本顺序，一般优先用 `defer`。

- 高频关键词：`async`、`defer`、普通 `script`、阻塞解析、异步下载、执行时机、执行顺序、HTML 解析、`DOMContentLoaded`、CSSOM、主业务脚本、埋点脚本
- 易追问：`async` 和 `defer` 最核心的区别是什么？为什么 `defer` 更适合主业务脚本？为什么 `async` 不保证顺序？`DOMContentLoaded` 和它们的关系是什么？CSS 为什么可能间接影响 `defer` 执行？模块脚本默认更像哪一种？
- 回答雷区：不要说 `async/defer` 完全不阻塞任何东西，更准确是它们不阻塞下载阶段的 HTML 解析，但执行时机仍可能打断流程；不要把 `async` 说成“HTML 解析完成后执行”，那是 `defer` 的特点；不要忽略多个 `defer` 脚本会按顺序执行这一点。

## 46. 块级元素、行内元素、行内块元素的区别

- 一句话结论：这三类元素的核心区别在于是否独占一行、宽高是否容易直接设置，以及它们在文档流中的排版方式。
- 30秒回答：块级元素通常独占一行，默认宽度会尽量撑满父容器，且可以直接设置宽高；行内元素不独占一行，宽度主要由内容决定，直接设置宽高通常不生效；行内块元素介于两者之间，不独占一行，但又可以设置宽高。常见块级元素有 `div`、`p`、`h1`，行内元素有 `span`、`a`、`strong`，行内块元素常见有 `img`、`button`、`input`。
- 1分钟展开：
  - 块级元素：块级元素在普通文档流里通常会独占一行，默认宽度会尽量占满父级可用空间，所以很适合做页面结构划分、容器、段落和区块布局。它们一般可以直接设置 `width`、`height`、`margin`、`padding`。
  - 行内元素：行内元素不会独占一行，而是跟文字一样在一行内排布，宽度通常由内容决定。对纯 `inline` 元素来说，直接设置 `width` 和 `height` 往往不会按块级盒模型那样生效，因此更适合承载文本或局部强调内容。
  - 行内块元素：`inline-block` 可以理解成“既保留行内排布特性，又拥有块级盒模型的一部分能力”。它不独占一行，可以和其他元素并排，同时也能设置宽高，所以非常适合做导航项、按钮组、卡片列表等。
  - 默认 `display` 的角度：本质上这三类元素的差别可以从 `display` 理解。块级元素常见默认是 `block`，行内元素是 `inline`，行内块元素则是 `inline-block`。所以很多时候元素类型不是绝对的，也可以通过 CSS 改变其显示方式。
  - 布局实践补充：现代布局里，很多元素最终都会进入 `flex` 或 `grid` 容器，这时它们的原始 `block/inline` 表现会被一定程度弱化。但面试里仍然要理解这些基础差异，因为它们会影响宽高、换行和普通文档流表现。
- 高频关键词：块级元素、行内元素、行内块元素、`block`、`inline`、`inline-block`、独占一行、宽高设置、内容决定宽度、文档流、换行、盒模型、`div`、`span`、`img`

- 易追问：为什么行内元素不能像块级元素那样设置宽高？`inline-block` 为什么常用于并排布局？元素类型能不能通过 CSS 改？在 `flex` 和 `grid` 下这些区别还重要吗？`a` 标签默认是行内元素，那怎么让它表现成块？
- 回答雷区：不要把“行内元素不能设置宽高”说得太绝对，更准确是普通 `inline` 元素对宽高支持有限、表现不同于块级；不要忽略这些差异本质上和 `display` 有关；不要只背标签名，不解释它们在布局上的区别。

## 47. Vite 和 Webpack 的详细对比

- 一句话结论：Vite 和 Webpack 都能完成前端构建，但它们在开发阶段的思路差异很大，Vite 更强调原生 ESM 和按需加载带来的启动速度，Webpack 更强调以依赖图为核心的统一打包与高度可定制的工程能力。
- 30秒回答：如果只看开发体验，Vite 通常更快，因为它启动时不先整体打包，而是起一个 dev server，让浏览器按需请求模块，并配合 `esbuild` 做依赖预构建；Webpack 开发时更偏传统 bundle 思路，会先分析依赖图，再通过 loader、plugin、Babel 等链路处理模块，所以大型项目冷启动和热更新成本往往更高。生产环境下两者都能做高质量打包，Vite 通常基于 Rollup，Webpack 则基于自己的打包体系，差别更多体现在设计理念、生态兼容和工程控制力上。
- 1分钟展开：
  - 开发阶段的核心差异：Webpack 在开发阶段通常也要先从入口构建依赖图，再把模块转成浏览器可运行的结果，所以项目越大，冷启动和重编译成本越明显。Vite 则采用更接近 bundleless 的思路，先启动服务器，浏览器要哪个模块就返回哪个模块，因此开发阶段反馈通常更快。
  - 为什么 Vite 开发更快：一方面是原生 ES Module 按需加载，另一方面是它会用 `esbuild` 做依赖预构建和部分高性能转换。`esbuild` 用 Go 编写，确实比传统基于 JS 的 Babel 链快很多，但面试里更稳妥的说法是“速度明显更快”，不必死背固定倍数。
  - Webpack 的优势点：Webpack 的强项在于它是非常成熟的模块化构建工具，loader 和 plugin 生态庞大，能把各种前端资源统一纳入构建流程，对复杂工程、历史项目、深度定制构建链路的支持非常强。
  - 生产构建差异：Vite 在生产环境下一般交给 Rollup 做打包。Rollup 在处理 ES Module、Tree Shaking、产物精简方面体验很好，所以 Vite 的生产产物通常也比较干净。Webpack 生产构建同样能做压缩、Tree Shaking、代码分割、缓存优化，只是它走的是自己的完整打包体系。

- **Babel 和 esbuild 的位置：**Webpack 经常和 Babel 搭配，用于语法转换和兼容性处理；Vite 开发阶段很多工作交给 **esbuild**，但并不代表完全不用 Babel，碰到更复杂的语法能力、插件转换或兼容需求时，依然可能接入 Babel 或其他编译工具。
- **设计理念差异：**Webpack 更像一个“大而全”的通用构建平台，以模块依赖图和构建流程控制为核心；Vite 更像“开发时轻量、生产时稳妥”的组合方案，开发靠原生 ESM + dev server，生产靠 Rollup。你提到的“微内核、插件化”也可以理解成 Vite 通过插件体系把能力拆开，整体心智更轻。
- **适用场景：**新项目、现代框架项目、强调开发体验的场景，Vite 往往更受欢迎；老项目迁移、复杂构建需求、多年沉淀的企业工程体系里，Webpack 仍然非常常见。
- **高频关键词：**Vite、Webpack、开发模式、生产模式、dev server、按需加载、原生 ESM、bundleless、依赖图、**esbuild**、Go、Babel、Rollup、模块化构建、Tree Shaking、loader、plugin、冷启动、热更新、插件生态
- **易追问：**为什么 Vite 开发更快？bundleless 到底是什么意思？Vite 生产环境为什么还要 Rollup？Webpack 为什么在复杂工程里仍然常见？**esbuild** 和 Babel 的差别是什么？两者的插件生态有什么区别？什么时候你会选 Webpack 而不是 Vite？
- **回答雷区：**不要把对比简化成“Vite 全面替代 Webpack”，它们适用场景并不完全相同；不要说 Vite 完全不打包，生产阶段仍然会打包；不要把 Vite 快的原因只归结于 **esbuild**，原生 ESM 和按需加载同样关键；不要把 Webpack 说成“落后”，它在复杂工程能力上依然很强。

## 48. Vue3 的响应式原理

- **一句话结论：**Vue3 的响应式本质上是“代理数据访问 -> 收集依赖 -> 数据变化时触发副作用重新执行”，核心由 **Proxy**、依赖收集和更新触发机制组成。
- **30秒回答：**Vue3 的响应式可以概括成三步：第一步用 **Proxy** 对数据做代理，拦截属性读取和修改；第二步在读取时做依赖收集，也就是 **track**，把当前正在执行的副作用函数和这个对象属性建立关联；第三步在修改时触发更新，也就是 **trigger**，找到相关依赖重新执行，所以页面会自动更新，**computed**、**watchEffect**、**watch** 等能力也都建立在这套机制之上。相比 Vue2，Vue3 解决了新增/删除属性、数组下标修改等响应式局限，性能和可维护性也更好。
- **1分钟展开：**
  - **响应式要解决什么问题：**本质上就是“数据一变，依赖它的视图和逻辑能自动重新执行”。这不仅包括页面渲染更新，也包括计算属性、监听器和副作用函数的重新触发。
  - **Vue3 的核心链路：**可以概括成“数据劫持 + 依赖收集 + 触发更新”。数据被访问时收集依赖，被修改时通知依赖执行，这就是响应式系统的骨架。

- **Proxy 做数据代理**：Vue3 用 `Proxy` 代理整个对象，而不是像 Vue2 那样遍历每个属性用 `Object.defineProperty` 劫持。这样在 `get` 时能拦截读取，在 `set` 时能拦截修改，也更容易处理新增属性、删除属性和数组索引变化。
- **track 依赖收集**：当响应式属性在副作用函数中被读取时，Vue 会把“目标对象 -> 属性 -> 依赖集合”建立起映射关系。底层常见的数据结构是 `WeakMap -> Map -> Set`，也就是对象映射到属性，再映射到一组依赖函数（比如 `effect`）。
- **trigger 触发更新**：当属性值变化时，Vue 会找到这个属性对应的依赖集合，把相关副作用函数重新执行，于是模板渲染函数、计算属性依赖链和监听逻辑都会更新。
- **为什么 WeakMap 合适**：因为响应式系统是围绕对象建立依赖映射的，用 `WeakMap` 以对象为键可以更利于垃圾回收，避免无用对象长期被强引用。
- **Vue2 和 Vue3 的差异**：Vue2 基于 `Object.defineProperty`，初始化时要递归遍历对象属性，新增/删除属性监听不自然，数组索引和长度变更也要额外处理；Vue3 用 `Proxy` 代理整个对象，在能力覆盖和性能模型上都更自然。
- **懒代理**：Vue3 的响应式对象通常是按访问路径逐层代理的，也就是你真正访问到某个嵌套对象时，才继续把它包装成响应式代理，这能减少一开始的无效递归成本。
- **reactive 和 ref**：`reactive` 更适合代理对象和数组，底层是基于 `Proxy`；`ref` 更适合包装基本类型，也能包装对象。`ref` 返回的是一个带 `.value` 的响应式容器对象，它内部通过 `getter/setter` 暴露响应能力，本质上也是为了把“对值的读取和修改”纳入依赖收集与触发更新流程。
- **watch / computed 的关系**：`computed` 本质上也是基于响应式依赖收集的惰性副作用；`watch` 则是在依赖变化后执行指定回调。所以面试里可以把它们都归到“同一套响应式调度系统”下面来理解。
- **高频关键词**：Vue3、响应式、数据劫持、依赖收集、触发更新、`Proxy`、`get`、`set`、`track`、`trigger`、`WeakMap`、`Map`、`Set`、`effect`、懒代理、`reactive`、`ref`、`.value`、`computed`、`watch`
- **易追问**：Vue3 为什么改用 `Proxy`？`track` 和 `trigger` 分别做什么？为什么依赖表通常用 `WeakMap`？`reactive` 和 `ref` 的区别是什么？Vue2 为什么监听不到新增属性和数组下标？什么叫懒代理？`watch` 和 `computed` 底层和响应式是什么关系？
- **回答雷区**：不要把 Vue3 响应式简单说成“就是 `Proxy`”，核心还包括依赖收集和调度；不要说 `ref` 完全等于 `Object.defineProperty`，更准确是它通过对 `.value` 的访问控制建立响应式；不要把 `watch` 说成独立机制，它本质依赖同一套响应式系统；不要漏掉 Vue3 相比 Vue2 解决了新增属性、删除属性和数组索引等局限。

## 49. 为什么选择 TypeScript?

- 一句话结论：选择 TypeScript 的核心原因，是它能在大型项目和多人协作里把很多问题提前到开发阶段暴露出来，从而提升代码质量、可维护性和协作效率。
- 30秒回答：我选择 TypeScript，最核心的原因是它能提供静态类型检查，把很多原本运行时才暴露的问题提前到编码阶段发现，比如参数传错、属性名写错、返回值不匹配等。除此之外，TypeScript 还能帮助我们吧接口、数据结构和公共规范表达得更清晰，提高代码的扩展性和维护性。对于团队协作来说，它也能减少沟通成本，让不同人对同一份数据结构和函数约定有一致认知。
- 1分钟展开：
  - 第一点：静态检查，尽早发现问题：JavaScript 很灵活，但项目复杂后，很多错误要等运行时才发现。TypeScript 能在开发和编译阶段提前提示类型问题，把问题前置，这对中大型项目价值很高。
  - 第二点：类型和接口更清晰：通过接口、类型别名、泛型等能力，可以把对象结构、函数参数、返回值、接口响应格式描述清楚。代码不只是“能跑”，而是更容易读懂、约束和复用。
  - 第三点：更利于扩展和维护：随着项目变大，模块之间的依赖关系会越来越复杂。TypeScript 能在重构时帮你快速发现受影响的位置，降低改一处坏一片的风险，所以对长期维护很友好。
  - 第四点：团队协作更顺畅：当多人一起开发时，类型本身就像一份可执行的文档。大家看到类型声明，就能知道这个函数该怎么用、这个对象应该长什么样，从而减少口头沟通和低级错误。
  - 第五点：表达能力更强：TypeScript 提供了联合类型、交叉类型、枚举、泛型等更丰富的类型表达方式，能更准确地描述业务状态和数据约束，比纯 JavaScript 更适合复杂业务建模。
  - 实际落点：所以我会把 TypeScript 理解成一套“提高工程质量的基础设施”，它不只是让代码更严谨，也能让后续维护、重构和团队协作更高效。
- 高频关键词：TypeScript、静态类型检查、提前发现错误、接口定义、类型定义、代码规范、可维护性、可扩展性、团队协作、联合类型、交叉类型、枚举、重构、工程质量
- 易追问：TypeScript 解决了 JavaScript 的哪些痛点？为什么它更适合大型项目？类型系统怎么帮助团队协作？联合类型和交叉类型分别适合什么场景？为什么说类型声明像文档？TypeScript 会不会降低开发效率？
- 回答雷区：不要只回答“因为类型安全”，太空；不要把 TypeScript 说成只是在“减少编译错误”，更准确是减少开发期和运行时错误风险；不要忽略它对协作、重构和长期维护的价值；不要把它讲成纯语法升级，本质是工程能力升级。

## 50. HTTP 的详细介绍

- 一句话结论：HTTP 是应用层的请求-响应协议，负责规定客户端和服务端如何传输超文本及其他资源，本身是无状态的，早期明文传输通常基于 TCP，而 HTTPS 则是在 HTTP 之上加了 TLS 安全层。
- 30秒回答：HTTP 全称是超文本传输协议，本质上是一套客户端和服务端通信的规则，属于应用层协议。它的特点是请求-响应、无状态、可扩展，传统 HTTP 基于 TCP 传输，默认明文，所以才有了 HTTPS。版本演进上，HTTP/1.0 主要是短连接，HTTP/1.1 默认支持长连接并增强缓存和 Host 机制，HTTP/2 引入二进制分帧和多路复用，HTTP/3 则进一步基于 QUIC 运行在 UDP 之上，重点优化连接建立和丢包场景下的传输效率。
- 1分钟展开：
  - **HTTP 是什么**：HTTP 是 HyperText Transfer Protocol，最早用于传输超文本，但现在也用于图片、JSON、视频、文件等各种资源。它工作在应用层，下面通常依赖 TCP 提供可靠传输。
  - **核心通信模型**：HTTP 的基本模式是请求-响应。客户端发送请求报文，服务端返回响应报文。前端开发日常接触的页面加载、接口请求、资源下载，本质上都在走这套模型。
  - **无状态特性**：HTTP 本身不保存会话状态，也就是说服务端默认不会记住“上一次请求是不是同一个用户”。这让协议更简单、扩展性更强，但如果业务需要登录态和用户上下文，就要借助 Cookie、Session、Token 等机制补足状态管理。
  - **报文结构**：请求报文通常包含请求行、请求头、请求体。请求行里有请求方法、URL、协议版本；请求头里带上内容类型、缓存控制、认证信息等；请求体则承载提交的数据。响应报文对应包含状态行、响应头和响应体。
  - **HTTP/1.0**：特点是连接复用能力弱，很多场景下一个请求就对应一个 TCP 连接，请求完成后连接关闭，所以开销较大。
  - **HTTP/1.1**：默认支持长连接，也就是一个 TCP 连接上可以复用多个请求，减少重复握手成本；它还引入了 Host 头等机制。理论上支持管道化（pipelining），但因为队头阻塞等问题，实际生产中并没有广泛普及。
  - **HTTP/2**：核心升级包括二进制分帧、多路复用、头部压缩、服务端推送等。它能让多个请求共享一个连接并并发传输，明显改善 HTTP/1.1 下的队头阻塞问题。它并不是协议层面强制要求 HTTPS，但现实中浏览器环境通常都和 HTTPS 一起使用。
  - **HTTP/3**：HTTP/3 运行在 QUIC 之上，而 QUIC 基于 UDP 实现可靠传输和更快的连接建立。它进一步优化了丢包和连接迁移场景，减少传统 TCP + TLS 组合带来的握手和队头阻塞影响。

- 怎么更完整地回答：面试里通常可以按“定义 -> 特点 -> 报文结构 -> 版本演进 -> 无状态如何补状态”这条顺序讲，结构会比较清晰。
- 高频关键词：HTTP、应用层、请求-响应、超文本传输协议、TCP、无状态、明文传输、HTTPS、请求报文、响应报文、请求行、状态行、请求头、响应头、请求体、HTTP/1.0、HTTP/1.1、HTTP/2、HTTP/3、长连接、多路复用、二进制分帧、QUIC
- 易追问：HTTP 为什么说是无状态？无状态为什么既是优点也是缺点？HTTP 和 HTTPS 的关系是什么？HTTP/1.1 和 HTTP/2 的核心区别是什么？为什么 HTTP/2 更快？HTTP/3 为什么改用 QUIC？请求报文和响应报文分别包含什么？
- 回答雷区：不要把 HTTP 说成传输层协议，它属于应用层；不要把 HTTP/1.1 的管道化说成现实里广泛使用的主流能力；不要说 HTTP/2 协议层面必须 HTTPS，更准确是浏览器环境里通常与 HTTPS 一起使用；不要只背版本号，不解释每个版本到底解决了什么问题。

## 51. TCP 连接

- 一句话结论：TCP 是面向连接、可靠、面向字节流的传输层协议，它通过确认应答、重传、滑动窗口、流量控制和拥塞控制来保证数据尽可能稳定有序地送达。
- 30秒回答：TCP 和 UDP 的核心区别在于，TCP 更重，但可靠；UDP 更轻，但不保证可靠性。TCP 的特点包括面向连接、可靠传输、面向字节流、有流量控制和拥塞控制，所以它适合网页、文件、数据库这类对可靠性要求高的场景。它之所以可靠，靠的是确认应答、超时重传、ARQ、滑动窗口这些机制；而它之所以不会把网络和接收方压垮，靠的是流量控制和拥塞控制。
- 1分钟展开：
  - **TCP 和 UDP 的区别**：TCP 是面向连接的，通信前要先建立连接；UDP 是无连接的，直接发数据报。TCP 追求可靠、有序、可重传；UDP 更强调轻量和低延迟，所以更适合音视频、实时通信、DNS 这类能容忍少量丢包的场景。
  - **TCP 的核心特点**：第一是面向连接；第二是可靠传输；第三是面向字节流，也就是把数据看成连续字节流而不是一条条独立消息；第四是有流量控制和拥塞控制；代价就是协议复杂、状态多、开销也更大。
  - **可靠传输怎么实现**：TCP 会给数据分段编号，接收方收到后返回确认应答（ACK）。如果发送方在超时时间内没有等到确认，就会触发重传。这里常提到 ARQ 和滑动窗口机制，前者负责“确认与重传”，后者负责在等待 ACK 的同时允许连续发送多个报文段，提高传输效率。
  - **流量控制**：流量控制解决的是“接收方来不及处理怎么办”。接收方会通过窗口大小告诉发送方自己当前还能接收多少字节，发送方就按这个能力控制发送速度，避免把接收缓冲区打满。

- **拥塞控制**：拥塞控制解决的是“网络本身扛不住怎么办”。经典机制包括慢开始、拥塞避免、快重传、快恢复。简单理解就是：一开始先小心试探，确认网络能承受后再逐步放大；一旦检测到丢包或拥塞迹象，就及时收缩发送窗口，避免把整个网络压垮。
- **为什么 TCP 开销更大**：因为它要维护连接状态、序号、确认号、窗口、重传计时器等信息，还要处理重排、丢包、重传和拥塞判断，所以相比 UDP 明显更重，但这正是它可靠性的来源。
- **怎么更完整地回答**：面试里可以先对比 TCP 和 UDP，再顺着“可靠传输、流量控制、拥塞控制”三个关键词展开，这样结构最清晰。
- **高频关键词**：TCP、UDP、面向连接、无连接、可靠传输、面向字节流、面向报文、确认应答、ACK、ARQ、滑动窗口、超时重传、流量控制、接收窗口、拥塞控制、慢开始、拥塞避免、快重传、快恢复、传输层
- **易追问**：TCP 和 UDP 的核心区别是什么？为什么说 TCP 是面向字节流？滑动窗口解决什么问题？流量控制和拥塞控制有什么区别？为什么 TCP 可靠？快重传和超时重传有什么区别？哪些业务更适合 UDP？
- **回答雷区**：不要把流量控制和拥塞控制混为一谈，一个关注接收方能力，一个关注网络整体状况；不要把 TCP 的可靠性简单说成“不会丢包”，更准确是它通过确认和重传尽量保证可靠送达；不要把 TCP 和 HTTP 混为同一层协议，TCP 属于传输层，HTTP 属于应用层。

## 52. Js V8 的垃圾回收机制

- **一句话结论**：V8 的垃圾回收核心不是“谁引用我几次”，而是基于可达性做分代式回收，新生代重点追求回收快，老年代重点解决大对象和长期存活对象的空间整理与稳定性。
- **30秒回答**：V8 判断对象是不是垃圾，核心看它从根对象出发是否还能被访问到，也就是可达性，而不是主要靠引用计数。整体上它采用分代垃圾回收：新生代对象多、生命周期短，主要用 Scavenge 这类复制/疏散算法，回收快；老年代对象存活时间长、内存更大，主要用标记清除和标记整理，避免大规模复制带来的高成本。如果代码里一直保留无用引用，比如全局变量、没解绑的事件、没清掉的定时器、闭包滥用、缓存无限增长，就会出现内存泄漏，严重时可能把堆撑爆。
- **1分钟展开**：
  - **先区分几个概念**：垃圾回收是自动释放不再使用内存的机制；内存泄漏是“本该释放，但因为仍然可达而没被回收”；内存溢出是可用内存不够，申请失败。泄漏长期积累，最终就可能引发溢出。
  - **V8 的判断标准是可达性**：GC 会从一组根对象出发，比如全局对象、当前调用栈、闭包里仍被引用的变量等，沿引用关系往下找。能到达的是存活对象，走不到的才是垃圾。

- 为什么引用计数不是主流：引用计数思路是“谁引用我几次”，引用数变成 0 就回收，但它解决不了循环引用问题，所以现代 JS 引擎不会把它作为核心回收方案。面试里要强调，V8 主要看可达性。
- 为什么要分新生代和老生代：大部分对象都“生得快、死得也快”，少数对象会长期存活。分代后可以针对不同生命周期选不同算法，减少整体回收成本。
- 新生代回收：新生代空间小、回收频繁，常见思路是 **Scavenge**，也可以理解成复制/疏散算法。它会把存活对象复制到另一块区域，原区域整体清空，所以适合“垃圾多、活对象少”的场景，速度快。对象经历多次回收后仍存活，或者新生代空间压力较大时，可能晋升到老生代；如果新生代回收后仍放不下，也会触发晋升甚至更重的 GC。
- 老生代回收：老生代对象更多、更大、存活率更高，如果还用复制算法，成本会很高，所以更常见的是标记清除和标记整理。标记清除先标记存活对象，再清掉未标记对象；标记整理会在清理后进一步移动存活对象，减少内存碎片。
- 常见泄漏场景：全局变量挂太多数据、事件监听没有移除、`setInterval/setTimeout` 没有清理、闭包持有大对象、缓存没有淘汰策略、`Map/Set` 长期累积引用，都会让对象一直保持可达，GC 也回收不掉。
- 怎么预防：减少不必要的全局变量，事件和定时器及时解绑/清理，缓存要有大小或时间限制，不再需要的引用及时置空；如果键对象应该随外部对象一起释放，可以优先考虑 `WeakMap / WeakSet`。
- 高频关键词：V8、垃圾回收、GC、可达性、根对象、分代式垃圾回收、新生代、老生代、**Scavenge**、复制算法、疏散、晋升、标记清除、标记整理、内存碎片、内存泄漏、内存溢出、引用计数、循环引用、全局变量、闭包、定时器、事件监听、缓存、`Map`、`Set`、`WeakMap`
- 易追问：V8 为什么不用引用计数做主方案？什么叫可达性分析？新生代为什么适合 **Scavenge**？对象什么时候会从新生代晋升到老生代？标记清除和标记整理有什么区别？内存泄漏和内存溢出有什么区别？前端开发里常见的泄漏场景有哪些？为什么 `WeakMap` 更适合某些缓存场景？
- 回答雷区：不要把“垃圾对象”说成“引用数为 0 的对象”，V8 的核心判断是可达性；不要把内存泄漏和内存溢出混为一谈，泄漏是该释放没释放，溢出是内存不够；不要说新生代和老生代只是“内存大小不同”，更关键是对象生命周期和回收策略不同；不要说有 GC 就不会泄漏，只要无用对象仍然可达，GC 也无能为力。

## 53. 正则表达式的常见用法

- 一句话结论：正则表达式本质上是一套字符串匹配规则，前端里最常见的场景是做格式校验、内容提取、批量替换和文本处理，关键是理解字符类、量词、边界和常用方法。

- 30秒回答：正则在前端里最常见的用法有四类。第一类是创建正则，可以用字面量 `/abc/`，也可以用 `new RegExp('abc')`；第二类是做格式校验，比如手机号 `^1[3-9]\d{9}$`、6 位验证码 `^\d{6}$`、邮箱 `^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$`；第三类是字符串处理，比如千分位分隔符 `/\\B(?=(\\d{3})+(?!\\d))/g`；第四类是配合字符串方法使用，比如 `match`、`exec`、`replace`。面试里不要只背表达式，更重要是能解释 `^`、`$`、`[]`、`\\d`、`{n}`、`?=` 这些符号是什么意思。
- 1分钟展开：
  - 两种创建方式：最常见的是字面量写法，比如 `/abc/`，适合模式固定的场景；另一种是构造函数 `new RegExp('abc')`，适合模式需要动态拼接时使用。面试里可以顺带提一句，构造函数写法要注意转义问题。
  - 常见校验场景：手机号 `^1[3-9]\d{9}$` 表示以 `1` 开头，第二位是 `3-9`，后面再跟 9 个数字，`^` 是开头，`$` 是结尾；6 位验证码 `^\d{6}$` 就是从头到尾正好 6 个数字；邮箱 `^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$` 的意思是 `@` 前后都要有内容，并且不能包含空白和 `@` 本身。
  - 常见符号怎么理解：`[]` 表示字符范围或字符集合，比如 `[3-9]`；`\\d` 表示数字；`{n}` 表示正好 `n` 次，`+` 表示一次或多次；`^` 和 `$` 通常表示整串匹配的开始和结束。把这些基础符号解释清楚，比死记表达式更重要。
  - 千分位分隔符：`/\\B(?=(\\d{3})+(?!\\d))/g` 常见于金额格式化。这里 `\\B` 表示非单词边界，`(?=...)` 是正向预查，表示当前位置后面必须满足某个条件，`(\\d{3})+` 表示后面是一组或多组三位数字，`(?!\\d)` 表示这组条件后面不能再跟数字。它常配合 `replace` 在匹配位置插入逗号。
  - 常用方法：`str.match(reg)` 常用来拿匹配结果；`reg.exec(str)` 常用来做逐次匹配，尤其是配合全局标记时；`str.replace(reg, newStr)` 常用来做替换，比如脱敏、格式化、批量清洗文本。实际开发里这三个方法出现频率很高。
  - 面试回答思路：可以按“创建方式 -> 常见校验 -> 特殊匹配 -> 常用方法”这个顺序讲，再挑一两个例子解释关键符号，回答会更像真正理解，而不是只背模板。
- 高频关键词：正则表达式、`regex`、字面量、`new RegExp()`、字符串匹配、格式校验、字符类、字符集合、`[3-9]`、`\\d`、`^`、`$`、`{n}`、`+`、邮箱校验、手机号校验、验证码、千分位分隔符、正向预查、`?=`、负向预查、`?!\\d`、`\\B`、`match`、`exec`、`replace`
- 易追问：正则字面量和 `new RegExp()` 有什么区别？`^` 和 `$` 分别表示什么？`\\d`、`[3-9]`、`{6}` 分别是什么意思？千分位分隔符为什么要用预查？`match` 和 `exec` 有什么区别？邮箱正则为什么常写成 `^[^\\s@]+` 这种形式？正则能不能完全校验所有合法邮箱？

- 回答雷区：不要把正则讲成“只会做表单校验”，它还常用于提取、替换和格式化；不要只报出表达式不解释元字符含义，面试官通常更看重理解；不要说邮箱正则能 100% 覆盖所有合法邮箱，业务里一般是做常见格式校验；不要忽略 `new RegExp()` 的转义问题，动态拼接时很容易写错。

## 54. Node.js 有哪些常用模块

- 一句话结论：Node.js 常用模块可以分成两类，一类是内置核心模块，负责路径、文件、网络、系统、流和加密等基础能力；另一类是第三方模块，负责快速搭建 Web 服务、请求转发、数据库操作和跨域处理中间件等业务能力。
- 30秒回答：如果面试问 Node.js 常用模块，我一般会分内置模块和第三方模块来答。内置模块里比较常见的有 `path` 处理路径，`fs` 读写文件，`http/https` 起服务，`os` 获取操作系统信息，`url` 做 URL 解析，`events` 处理事件发布订阅，`stream` 处理流式读写，`crypto` 做加密和摘要，`util` 放一些工具方法。第三方模块里常见的有 `Express` 用来搭 Web 服务，`axios` 发 HTTP 请求，`mongoose` 操作 MongoDB，`cors` 处理中间件跨域。面试里重点不是把名字背全，而是知道这些模块各自解决什么问题。
- 1分钟展开：
  - 先分内置模块和第三方模块：内置模块也叫核心模块，Node 安装后就能直接用，比如 `path`、`fs`、`http`；第三方模块一般通过 `npm` 安装，用来补充具体业务场景能力，比如 `Express`、`axios`、`mongoose`、`cors`。
  - 路径和文件类模块：`path` 主要做路径拼接、解析和规范化，比如 `join`、`resolve`；`fs` 是文件系统模块，用来读写文件、创建目录、删除文件、处理文件流，这两个模块在 Node 开发里非常常见。
  - 网络和服务类模块：`http` 和 `https` 用来创建服务器、处理请求响应，也能发起网络请求；`url` 常用来解析请求地址、查询参数和路径信息。虽然现在很多项目会用框架封装，但底层能力还是这些模块提供的。
  - 系统和工具类模块：`os` 用来获取 CPU、内存、平台、临时目录等系统信息；`util` 提供一些工具方法，比如格式化输出、回调风格转 Promise 等，属于比较偏辅助性的模块。
  - 事件和流模块：`events` 是 Node 事件机制的基础，很多模块底层都和事件驱动有关；`stream` 则是 Node 很重要的一块，适合处理大文件、网络传输、管道操作，优势是边读边处理，不用一次性把数据全放进内存。
  - 安全相关模块：`crypto` 常用于摘要、加密、解密、签名、随机字符串生成等，比如做 `hash`、Token 签名或者密码处理时都可能用到。

- 高频第三方模块：`Express` 常用于快速搭建 Web 服务和中间件体系；`axios` 常用于服务端请求其他接口；`mongoose` 是 Node 操作 MongoDB 的常见 ODM；`cors` 常用于处理跨域响应头。面试里如果你能顺带讲出应用场景，会比只列模块名更有说服力。
- 回答思路：最稳的方式是按“核心模块负责底层能力，第三方模块负责业务框架和生态扩展”来讲，再挑几个你实际用过的模块展开。
- 高频关键词：`Node.js`、核心模块、内置模块、第三方模块、`path`、`fs`、`http`、`https`、`os`、`url`、`events`、`stream`、`crypto`、`util`、文件系统、路径处理、创建服务器、URL 解析、事件驱动、流式处理、加密、`Express`、`axios`、`mongoose`、`cors`、`npm`
- 易追问：`path.resolve()` 和 `path.join()` 有什么区别？`fs` 常见 API 有哪些？`http` 和 `https` 模块能做什么？为什么 Node 里 `stream` 很重要？`events` 和发布订阅有什么关系？`crypto` 常见使用场景有哪些？`Express` 和原生 `http` 的区别是什么？`mongoose` 为什么不直接用 MongoDB 原生驱动？
- 回答雷区：不要把所有模块混在一起背，最好先区分内置模块和第三方模块；不要只列名字不说用途，面试官更关心你是否知道应用场景；不要把 `axios`、`Express` 说成 Node 内置模块，它们属于第三方生态；不要漏掉 `stream` 和 `events`，这两个模块很能体现 Node 的运行模型和设计特点。

## 55. 你有哪些优化性能的手段，如何衡量性能提升？

- 一句话结论：前端性能优化不能只靠单点技巧，通常要从资源体积、请求链路、构建产物、运行时渲染和监控指标几层一起做，最后再用核心指标验证优化是否真的带来了用户体验提升。
- 30秒回答：如果面试问性能优化，我一般会分四层来讲。第一层是资源层，比如 JS、CSS、图片压缩，减少包体积和请求数量，配合 CDN、强缓存、协商缓存提升加载效率；第二层是构建层，比如 Tree Shaking、代码拆分、按路由或组件动态加载，把首屏不需要的代码延后；第三层是应用运行层，比如虚拟列表、防抖节流、Web Worker、`requestAnimationFrame`、`requestIdleCallback`，减少主线程阻塞和无效渲染；第四层是指标衡量层，常看 `FCP`、`LCP`、`INP`、`TTFB`，也会结合首屏时间、资源体积、请求数和真实用户监控数据判断优化效果。
- 1分钟展开：
  - 资源层优化：先减少资源体积。JS 可以用 `Terser` 压缩，CSS 可以用 `CSSnano` 压缩，图片要从分辨率、尺寸、多套资源和格式上一起优化，比如优先考虑 `webP` 或 `AVIF`，根据场景选择 `JPG`、`PNG`。这里的目标是让首屏下载更轻，避免单次 HTTP 请求过大。

- **请求链路优化**：减少不必要的 HTTP 请求，比如公共资源合并、合理打包、雪碧图或图标方案优化；同时充分利用浏览器缓存、CDN 缓存、强缓存和协商缓存。客户端存储层面，常见还有 `localStorage`、`Cache Storage`、`IndexedDB`，分别适合持久化配置、请求缓存和更复杂的本地数据存储。
- **构建层优化**：构建工具会先建立依赖图，再通过 `Tree Shaking` 去掉无用代码、做代码压缩和代码拆分，最终生成更适合线上加载的产物。实际项目里常见的拆分方式包括公共组件拆分、第三方库拆分、路由级拆分和动态导入。
- **应用层优化**：首屏阶段会重点做组件动态导入、路由懒加载、预加载和预请求，把首屏必须资源和非必须资源区分开。列表场景用虚拟列表减少 DOM 数量；高频事件用防抖和节流；计算密集型任务可放到 `web worker`；动画相关更新尽量放在 `requestAnimationFrame`；低优先级任务可以考虑 `requestIdleCallback`。
- **渲染和交互优化的核心思路**：本质是减少主线程压力，减少不必要的 JS 执行、样式计算、布局和重绘，让页面更快可见、可交互、可响应。
- **如何衡量性能提升**：指标上通常会看 `TTFB`、`FCP`、`LCP`、`INP`。`TTFB` 反映服务端首字节返回时间，`FCP` 是首次内容绘制，`LCP` 更接近用户感知的主要内容加载完成时间，`INP` 反映交互响应质量。部分团队也会关注 `FSP`，但现在更推荐优先看 Core Web Vitals 相关指标。
- **怎么证明优化有效**：不能只说“感觉变快了”，而要对比优化前后的包体积、请求数量、首屏耗时、`LCP/INP` 数据、接口耗时和真实用户监控结果。如果是业务页面，还可以看跳出率、转化率或页面停留时长有没有变化。
- **回答思路**：面试里最稳的是按“资源优化 -> 构建优化 -> 运行时优化 -> 指标衡量”来讲，最后再补一句“我会先定位瓶颈，再针对性优化，再用指标验证”。
- **高频关键词**：性能优化、资源压缩、`Terser`、`CSSnano`、图片压缩、`WebP`、`AVIF`、`JPG`、`PNG`、减少 HTTP 请求、浏览器缓存、CDN 缓存、强缓存、协商缓存、`localStorage`、`Cache Storage`、`IndexedDB`、打包工具、依赖图、`Tree Shaking`、代码拆分、动态加载、路由懒加载、预加载、预请求、虚拟列表、防抖、节流、`web worker`、`requestAnimationFrame`、`requestIdleCallback`、`FCP`、`LCP`、`INP`、`FSP`、`TTFB`
- **易追问**：你怎么判断性能瓶颈在哪？`FCP`、`LCP`、`INP` 分别代表什么？为什么代码拆分能优化首屏？强缓存和协商缓存有什么区别？`localStorage`、`Cache Storage`、`IndexedDB` 分别适合什么场景？虚拟列表解决的核心问题是什么？`web worker` 适合放哪些任务？性能优化后你怎么证明真的有效？
- **回答雷区**：不要把性能优化讲成一堆零散技巧，最好按层次组织；不要只说用了懒加载、压缩、缓存，却不说明它们分别解决什么问题；不要只靠本地体感判断优化效果，必须用指标和数据对比验证；不要把 `FSP` 当成唯一核心指标，当前更稳妥的是优先结合 `LCP`、`INP`、`TTFB` 这些更贴近用户体验的指标。

## 56. 懒加载的方式有哪些？

- 一句话结论：懒加载的核心思路是“非当前必须的资源先不加载，等真正需要时再加载”，常见做法包括原生 `loading='lazy'`、`IntersectionObserver` 可见区监听和 `import()` 动态导入，而 SSR 更准确地说是首屏渲染优化手段，不等于懒加载本身。
- 30秒回答：前端常见的懒加载方式主要有三类。第一类是资源级懒加载，比如图片和 `iframe` 可以直接用原生 `loading='lazy'`；第二类是基于可视区域的懒加载，常用 `IntersectionObserver` 监听元素进入视口后再加载图片、列表数据或组件；第三类是代码级懒加载，比如 `import()` 动态导入组件、路由和第三方模块，实现按需加载。SSR 也常跟懒加载一起出现，因为它能优化首屏渲染，但严格来说 SSR 不是懒加载，而是服务端提前把页面渲染出来。
- 1分钟展开：
  - 什么叫懒加载：本质就是把“现在不需要的内容”延后，等到用户真正看到、点击到、滚动到，或者业务逻辑真正用到时再加载，这样能减少首屏资源体积和主线程压力。
  - 原生属性懒加载：最简单的是给图片或 `iframe` 加 `loading='lazy'`。这种方式接入成本低、浏览器原生支持，但可控性相对一般，适合常规图片列表场景。
  - `IntersectionObserver` 懒加载：通过监听元素是否进入视口，进入后再去设置真实图片地址、加载组件或拉取数据。它比传统滚动监听更高效，也更适合做图片懒加载、无限滚动和分段渲染。
  - `import()` 动态导入：这是代码层面的懒加载。常见于组件动态导入、路由懒加载和按需引入大型第三方库。构建工具会配合代码拆分，把这部分代码单独打成 `chunk`，在真正访问时再请求。
  - 组件和路由级懒加载：在 React、Vue 这类框架里，常见是页面级路由拆分、弹窗组件按需加载、图表库或编辑器延迟加载。它本质上还是建立在动态导入之上。
  - SSR 的边界：SSR 可以优化首屏可见内容的到达时间，让用户更快看到页面，但它不等于懒加载。更准确的说法是，SSR 和懒加载可以配合使用，比如首屏内容服务端渲染，非首屏模块客户端再动态加载。
  - 回答思路：面试里可以按“资源懒加载 -> 可视区懒加载 -> 代码懒加载 -> SSR 边界”这条线讲，结构会比较清楚。
- 高频关键词：懒加载、按需加载、`loading='lazy'`、图片懒加载、`iframe` 懒加载、`IntersectionObserver`、视口监听、可视区检测、动态导入、`import()`、代码拆分、`chunk`、组件懒加载、路由懒加载、按需引入、首屏优化、SSR、服务端渲染

- 易追问：`loading='lazy'` 和 `IntersectionObserver` 有什么区别？为什么 `IntersectionObserver` 比滚动事件更合适？`import()` 为什么能实现懒加载？路由懒加载底层是怎么做的？SSR 为什么不等于懒加载？懒加载会不会影响 SEO？懒加载和预加载有什么区别？
- 回答雷区：不要把 SSR 直接说成懒加载，它更准确是首屏渲染优化手段；不要只讲图片懒加载，代码懒加载同样很常见；不要把 `import()` 说成普通异步请求，它本质上和构建后的代码拆分强相关；不要忽略懒加载的边界，过度拆分也可能带来更多请求和切换抖动。

## 57. 常用的 Git 命令有哪些？

- 一句话结论：常用 Git 命令可以按 workflow 来记忆，主要包括初始化仓库、查看状态、暂存提交、分支切换、拉取同步、合并对比和远程仓库管理几类。
- 30秒回答：如果面试问常用 Git 命令，我一般按日常流程来答。先是仓库初始化和查看状态，比如 `git init`、`git clone`、`git status`、`git log`；然后是提交相关，比如 `git add`、`git commit`、`git diff`；再是远程同步，比如 `git fetch`、`git pull`、`git push`；分支相关有 `git branch`、`git checkout -b`、`git merge`、`git branch -d`；远程仓库管理常用 `git remote -v`、`git remote add origin`、`git remote set-url origin`。如果要回退或撤销，还会用到 `git reset` 和 `git checkout`，不过面试里最好顺带说明它们有一定风险，使用时要区分场景。
- 1分钟展开：
  - 初始化和查看类命令：`git init` 用来初始化本地仓库，`git clone` 用来从远程拉一个仓库下来；`git status` 看当前工作区和暂存区状态，`git log` 看提交历史，`git diff` 看代码差异。
  - 提交类命令：`git add` 是把修改放进暂存区，`git commit` 是把暂存内容提交到本地仓库。面试里可以把它理解成“工作区 -> 暂存区 -> 本地仓库”的过程。
  - 同步远程仓库：`git fetch` 只拉远程更新，不自动合并；`git pull` 通常可以理解成拉取并合并；`git push` 是把本地提交推到远程。这里 `fetch` 和 `pull` 的区别是很高频的追问点。
  - 分支管理：`git branch` 查看本地分支，`git branch -r` 查看远程分支，`git checkout -b xxx` 创建并切换到新分支，`git merge` 合并分支，`git branch -d` 删除已经合并的本地分支。
  - 远程仓库管理：`git remote` 查看远程仓库别名，`git remote -v` 看更详细的远程地址；`git remote add origin <url>` 用来新增远程地址，`git remote set-url origin <url>` 用来修改已有远程地址。

- 回退和切换：`git reset` 常用于回退提交或重置暂存区；`git checkout` 既可以切分支，也能切换到某个提交或恢复文件。虽然现在很多场景会拆分成 `git switch` 和 `git restore`，但面试里说 `checkout` 仍然是常见回答。
- 回答思路：最稳的方式不是背一长串命令，而是按“初始化 -> 开发提交 -> 分支协作 -> 远程同步 -> 回退处理”来组织。
- 高频关键词：Git、版本控制、`git init`、`git clone`、`git status`、`git log`、`git diff`、`git add`、`git commit`、`git push`、`git fetch`、`git pull`、`git reset`、`git checkout`、`git branch`、`git branch -r`、`git checkout -b`、`git merge`、`git branch -d`、`git remote`、`git remote -v`、`git remote add origin`、`git remote set-url origin`
- 易追问：`git fetch` 和 `git pull` 有什么区别？`git add` 和 `git commit` 分别做什么？`git reset` 常见用法有哪些？`git checkout -b` 做了哪两件事？`git branch -d` 和 `-D` 有什么区别？`git remote add origin` 和 `git remote set-url origin` 分别适合什么场景？为什么现在有人推荐用 `git switch` 和 `git restore`？
- 回答雷区：不要把命令只背成清单，最好按工作流来讲；不要把 `git fetch` 和 `git pull` 混为一谈，前者通常不自动合并；不要把 `git reset` 说成“随便回退都安全”，它可能改动历史或工作区；不要把 `git remote` 写成 `get remote`，正确命令是 `git remote`。

## 58. Map 跟 WeakMap 有什么区别？

- 一句话结论：`Map` 和 `WeakMap` 都是键值对结构，但 `Map` 对键是强引用、可遍历，适合通用存储；`WeakMap` 对键是弱引用、只能用对象作键且不可遍历，更适合做对象关联缓存和私有数据存储。
- 30秒回答：两者最核心的区别有三点。第一，`Map` 是强引用，如果把对象作为 `key` 放进去，即使外部把 `obj = null`，只要 `Map` 里还在引用，这个对象就不能被垃圾回收；`WeakMap` 是弱引用，外部对象没有其他引用后，就可以被 GC 回收。第二，`Map` 的 `key` 可以是任意类型，`WeakMap` 的 `key` 只能是对象。第三，`Map` 可以遍历、有 `size`，`WeakMap` 不能遍历，也没有 `size`，因为它里面的数据可能随时被垃圾回收机制清掉。
- 1分钟展开：
  - 强引用和弱引用的区别：`Map` 会强持有 `key`，只要映射还在，对象就还活着；`WeakMap` 不会阻止 `key` 对象被回收，所以更适合那些“生命周期应跟对象本身走”的场景。
  - `key` 的类型限制：`Map` 的 `key` 可以是字符串、数字、对象、函数等任意值；`WeakMap` 只能用对象作为 `key`，不能用基本类型。这一点面试里很容易被追问。

- 可遍历性不同：`Map` 支持 `keys()`、`values()`、`entries()`、`forEach()`，也有 `size`；`WeakMap` 没有这些能力。原因不是“官方没做”，而是因为它内部项可能被 GC 随时移除，如果允许遍历，语义会很不稳定。
- 典型使用场景：`Map` 更像增强版对象字典，适合通用缓存、映射表、去重辅助结构；`WeakMap` 常用于给 DOM 节点、实例对象挂关联数据，或做不会造成内存泄漏的缓存。
- 你给的例子怎么理解：如果 `const map = new Map(); let obj = {};`  
`map.set(obj, 1); obj = null;`，原对象仍然可能因为 `Map` 持有 key 而不被回收；而如果换成 `WeakMap`，外部引用断开后，这条映射就可能被 GC 清理。
- 回答思路：面试里最稳的是按“引用方式 -> key 类型 -> 是否可遍历 -> 使用场景”这条线讲。
- 高频关键词：`Map`、`WeakMap`、强引用、弱引用、垃圾回收、GC、对象 key、可遍历、`size`、键值对、缓存、私有数据、内存泄漏、对象关联数据
- 易追问：为什么 `WeakMap` 不能遍历？为什么 `WeakMap` 的 key 只能是对象？`Map` 和普通对象有什么区别？什么场景下更适合用 `WeakMap`？`WeakSet` 和 `Set` 的区别是什么？为什么 `WeakMap` 更适合做缓存？
- 回答雷区：不要只说“一个强引用一个弱引用”，最好补上可遍历性和 key 类型差异；不要说 `WeakMap` 会“立刻”删除数据，是否回收取决于 GC 时机；不要把 `WeakMap` 说成性能更高，它的核心价值是引用语义和内存管理边界。

## 59. 箭头函数和普通函数的区别？

- 一句话结论：箭头函数最大的特点不是“写法更短”，而是它没有自己的 `this`、`arguments`、`prototype`，也不能作为构造函数，所以更适合回调和需要绑定外层 `this` 的场景；普通函数则更通用、语义更完整。
- 30秒回答：箭头函数和普通函数最核心的区别有几个。第一，箭头函数没有自己的 `this`，它的 `this` 来自定义时外层作用域；普通函数的 `this` 取决于调用方式。第二，箭头函数不能作为构造函数，不能用 `new`，也没有 `prototype`。第三，箭头函数也没有自己的 `arguments` 对象。正因为这些特性，箭头函数更适合写简洁回调、定时器、数组方法和需要继承外层 `this` 的场景；如果你需要对象方法、构造函数或依赖动态 `this`，通常还是普通函数更合适。
- 1分钟展开：
  - `this` 绑定方式不同：普通函数的 `this` 是运行时决定的，谁调用它，`this` 通常就指向谁；箭头函数没有自己的 `this`，会直接捕获定义时外层作用域的 `this`，所以常用来避免回调里 `this` 丢失。

- 不能作为构造函数：箭头函数没有 `[[Construct]]` 能力，不能 `new`，也就不存在自己的 `prototype`。这就是为什么它不适合拿来定义类式构造逻辑。
- 没有自己的 `arguments`：箭头函数内部访问到的不是自己的 `arguments`，而是外层作用域的。如果想拿不定参数，通常更推荐显式使用剩余参数 `...args`。
- 语法更简洁：单表达式场景下，箭头函数可以直接隐式返回，比如数组 `map`、`filter`、`reduce` 的回调会写得更紧凑。但这只是表面差异，真正重要的是它的语义差异。
- 适用场景：事件回调、`Promise` 链、数组方法、高阶函数、`React/Vue` 里局部回调都很适合箭头函数；对象方法、原型方法、构造逻辑或需要动态绑定 `this` 的函数，更适合普通函数。
- 回答思路：可以先讲 `this`，再讲 `new` / `prototype` / `arguments`，最后补应用场景，面试官通常就会觉得你理解比较完整。
- 高频关键词：箭头函数、普通函数、`this`、词法作用域、`arguments`、`prototype`、构造函数、`new`、剩余参数、回调函数、隐式返回、动态 `this`
- 易追问：为什么箭头函数没有自己的 `this`？箭头函数为什么不能 `new`？箭头函数为什么没有 `prototype`？`arguments` 在箭头函数里怎么处理？对象方法为什么不建议用箭头函数？箭头函数和普通函数在事件回调里有什么区别？
- 回答雷区：不要把箭头函数的区别只讲成“写法更简洁”；不要说箭头函数的 `this` 永远指向 `window`，它取决于外层作用域；不要把“没有 `prototype`”和“不能作为构造函数”拆成毫无关系的两点，它们本质相关；不要在对象方法场景里盲目推荐箭头函数。

## 60. 正则表达式的尾缀

- 一句话结论：正则表达式尾缀本质上是修饰符，也叫 `flags`，用来改变匹配行为，前端最常见的有 `i`、`g`、`m`、`s`、`u`。
- 30秒回答：正则后面的尾缀其实就是修饰符。`i` 表示忽略大小写，`g` 表示全局匹配，`m` 表示多行匹配，让 `^` 和 `$` 能按行工作，`s` 表示让点号 `.` 可以匹配换行符，`u` 表示按 `Unicode` 模式解析。面试里不要只背字母，最好能顺带解释它们到底影响了什么匹配规则，比如 `g` 会影响多次匹配，`m` 影响的是边界行为，`s` 影响的是点号能力。
- 1分钟展开：
  - `i` 忽略大小写：加了 `i` 之后，`/abc/i` 可以匹配 `abc`、`ABC`、`Abc` 等不同大小写形式，常见于用户输入校验或搜索场景。
  - `g` 全局匹配：默认正则往往只拿到第一个匹配项，带 `g` 后会在整个字符串里继续找。它和 `match`、`exec` 的行为也有关联，所以这是非常高频的知识点。

- **m 多行匹配**：它不会让点号匹配换行，而是让 **^** 和 **\$** 不再只看整个字符串的开头和结尾，而是把每一行都当成边界。
- **s dotAll**：加了 **s** 之后，**.** 才能匹配换行符。很多人会把 **m** 和 **s** 搞混，面试里一定要区分清楚。
- **u Unicode 模式**：开启 Unicode 语义后，正则在处理更完整的 Unicode 字符时会更准确，比如一些特殊字符、码点匹配场景会依赖它。
- **怎么创建带修饰符的正则**：字面量写法可以直接写成 `/abc/ig`，构造函数写法则是 `new RegExp('abc', 'ig')`。
- **回答思路**：可以按“每个 flag 影响什么规则”来讲，而不是只背缩写全称。
- **高频关键词**：正则表达式、修饰符、flags、**i**、**g**、**m**、**s**、**u**、忽略大小写、全局匹配、多行匹配、dotAll、Unicode、边界、**^**、**\$**、**.**
- **易追问**：**g** 会影响哪些方法的返回结果？**m** 和 **s** 有什么区别？为什么 **.** 默认匹配不到换行？**u** 解决了什么问题？正则修饰符可以组合使用吗？`/abc/ig` 和 `new RegExp('abc', 'ig')` 有什么区别？
- **回答雷区**：不要把修饰符讲成“尾缀语法糖”，它们会真实改变匹配行为；不要混淆 **m** 和 **s**，一个影响边界，一个影响点号；不要说 **g** 只是“找全部”，它还会影响某些方法的迭代匹配行为；不要忽略 **u**，现在很多字符匹配场景都和它有关。

## 61. 前端规范的一些新特性

- **一句话结论**：近年前端规范的新特性，核心趋势是“语言能力更强、CSS 选择器更智能、原生交互能力更丰富”，像 `Object.groupBy()`、`Promise.withResolvers()`、CSS Nesting、`:has()`、`popover`、View Transition API 都体现了这种方向。
- **30秒回答**：如果面试问近年前端规范的一些新特性，我会从 JS、CSS 和浏览器交互 API 三层来讲。JS 里像 `Object.groupBy()` 可以更方便地按条件分组数组数据，`Promise.withResolvers()` 可以把 `resolve` 和 `reject` 单独拿出来；CSS 里有 CSS Nesting 支持嵌套写法，`:has()` 让父选择器场景更好处理；浏览器原生交互层面有 `popover` 和 View Transition API，前者简化弹层能力，后者让页面切换动画更原生。这类特性的共同点是减少对第三方库和大量样板代码的依赖。
- **1分钟展开**：
  - **Object.groupBy()**：它可以把数组按某个规则分组，返回一个按分组键组织起来的对象。以前这类逻辑往往要手写 `reduce`，现在表达更直接。
  - **Promise.withResolvers()**：它返回一个包含 `promise`、`resolve`、`reject` 的对象，适合那些需要先创建 Promise、再在外部时机触发完成的场景。可以把它理解成对“手动暴露 `resolve/reject`”这种模式的原生支持。

- **CSS Nesting**: 原生 CSS 现在也在逐步支持嵌套语法，能让样式层次更清晰，某些场景下减少对 Sass/Less 的依赖。但面试里也要知道，它的语法和预处理器并不是完全一样。
- **:has()**: 这是一个非常重要的关系型选择器，可以理解成“带条件的父级/祖先匹配能力”，让很多过去必须靠 JS 或 DOM class 切换才能做的样式判断，变成纯 CSS 就能处理。
- **popover**: 它是浏览器提供的原生弹出层能力，适合轻量弹层、菜单、提示层等场景。它让很多简单浮层不一定非要依赖整套组件库。
- **View Transition API**: 它让页面切换、局部视图切换的过渡动画更容易实现，尤其适合单页应用页面切换、列表到详情动画等场景。
- **回答思路**: 最稳的是按“JS 新能力 -> CSS 新能力 -> 浏览器原生交互 API”来讲，再补一句它们共同在减少样板代码和第三方依赖。
- **高频关键词**: 前端规范、新特性、`Object.groupBy()`、`Promise.withResolvers()`、CSS Nesting、`:has()`、`popover`、View Transition API、JavaScript、CSS、原生能力、分组、Promise 控制、嵌套样式、关系型选择器、页面转场
- **易追问**: `Object.groupBy()` 和 `reduce` 有什么区别？`Promise.withResolvers()` 适合什么场景？CSS Nesting 和 Sass 嵌套一样吗？`:has()` 为什么重要？`popover` 和手写弹层相比有什么优势？View Transition API 更适合什么类型的页面切换？
- **回答雷区**: 不要把这些特性只当成“语法糖”，很多都是原生能力增强；不要默认所有运行环境都完全支持，真实项目里还要关注兼容性；不要把 CSS Nesting 和 Sass/Less 完全画等号；不要只报 API 名，不解释它解决了什么开发痛点。

## 62. BigInt

- **一句话结论**: `BigInt` 是 JavaScript 用来表示超出 `Number` 安全整数范围的大整数类型，适合处理大整数计算，但它和普通 `Number` 不能随意混算。
- **30秒回答**: JavaScript 里的 `Number` 虽然能表示很大的值，但安全整数范围只有  $-(2^{53} - 1)$  到  $2^{53} - 1$ ，也就是 `Number.MAX_SAFE_INTEGER` 附近，再往上精度就可能出问题。`BigInt` 就是为了解决大整数精度问题而引入的，你可以用 `123n` 这种字面量创建，也可以用 `BigInt('123')` 创建。它适合处理 ID、大整数运算、金融或底层数据场景，但要注意 `BigInt` 不能和 `Number` 直接做算术运算。
- **1分钟展开**:
  - **为什么需要 BigInt**: 因为 JS 的 `Number` 基于双精度浮点数实现，大整数超过安全范围后，比较和运算都可能失真。比如两个不同的大整数，结果看起来可能会一样。

- 怎么创建：最常见的是在整数后面加 `n`，比如 `123n`；另一种是 `BigInt('123')` 或 `BigInt(123)`。面试里通常会顺带强调，它只能表示整数，不是高精度小数方案。
- 和 `Number` 的边界：`BigInt` 能解决的是“大整数精度”问题，不是所有数字问题。它和 `Number` 是不同类型，像 `1n + 1` 这种直接混算会报错，通常需要显式转换。
- 适用场景：超大 ID、加密算法相关整数、时间戳扩展场景、底层二进制数据处理、需要绝对整数精度的运算，都可能用到 `BigInt`。
- 使用限制：很多 `Math` 方法不能直接处理 `BigInt`；JSON 序列化也不能直接无脑用；同时它不适合带小数的金融计算，金融场景更常见还是整数分单位或专门高精度库。
- 回答思路：面试里可以按“为什么出现 -> 怎么创建 -> 解决什么问题 -> 有什么限制”来讲。
- 高频关键词：`BigInt`、大整数、`Number`、安全整数、`253 - 1`、`Number.MAX_SAFE_INTEGER`、精度丢失、`123n`、`BigInt('123')`、整数计算、类型转换、不能混算
- 易追问：为什么 `Number` 会有安全整数上限？`BigInt` 和 `Number` 能一起运算吗？`BigInt` 能表示小数吗？什么场景下适合用 `BigInt`？`BigInt` 和高精度小数库的区别是什么？为什么 `Math` 不能直接处理很多 `BigInt`？
- 回答雷区：不要把 `BigInt` 说成“更大的 `Number`”，它是独立类型；不要说它能解决所有精度问题，它主要解决整数精度问题；不要忽略 `BigInt` 和 `Number` 不能直接混算的限制；不要把它误用到小数金融计算场景。

## 63. TS 常用的内置工具类型

- 一句话结论：TypeScript 的内置工具类型本质上是在已有类型基础上做“增删改查式”的类型变换，最常见的是可选、必填、只读、挑选字段、排除字段、联合类型过滤和函数参数/返回值提取。
- 30秒回答：TS 里常用的工具类型可以按用途来记。对象字段变换类有 `Partial<T>`、`Required<T>`、`Readonly<T>`、`Pick<T, K>`、`Omit<T, K>`、`Record<K, T>`；联合类型处理类有 `Exclude<T, U>` 和 `Extract<T, U>`；函数类型提取类有 `Parameters<T>` 和 `ReturnType<T>`。它们的核心价值是复用已有类型，避免你每次都重新手写一套新类型，让类型系统更抽象、更可维护。
- 1分钟展开：

- 对象字段改造类：`Partial<T>` 会把所有属性变成可选；`Required<T>` 则相反，把所有属性变成必填；`Readonly<T>` 会把属性变成只读，适合配置对象或不可变数据约束。
- 字段选择类：`Pick<T, K>` 表示从一个类型中挑出部分字段；`Omit<T, K>` 表示从一个类型中排除某些字段。这两个在接口裁剪、表单类型、接口返回类型二次加工时非常常见。
- 映射生成类：`Record<K, T>` 可以根据一组 `key` 批量生成对象类型，比如把联合类型键映射成统一 `value` 类型，特别适合做字典表、配置表。
- 联合类型过滤类：`Exclude<T, U>` 表示从 `T` 中剔除能赋值给 `U` 的部分；`Extract<T, U>` 则是从 `T` 中提取出和 `U` 有交集的部分。它们本质上是联合类型的过滤器。
- 函数类型提取类：`Parameters<T>` 可以拿到函数参数元组类型；`ReturnType<T>` 可以拿到函数返回值类型。实际开发中经常用来复用服务函数、hooks、工具函数的类型签名。
- 为什么这些工具类型重要：它们能让类型定义围绕已有事实推导，而不是复制粘贴。代码改一处，相关类型也能跟着收敛，维护成本会更低。
- 回答思路：面试里不要只背名字，最好按“对象改造 -> 联合过滤 -> 函数提取”来讲，会更有结构。
- 高频关键词：TypeScript、工具类型、`Partial<T>`、`Required<T>`、`Readonly<T>`、`Pick<T, K>`、`Omit<T, K>`、`Record<K, T>`、`Exclude<T, U>`、`Extract<T, U>`、`Parameters<T>`、`ReturnType<T>`、类型变换、映射类型、条件类型
- 易追问：`Pick` 和 `Omit` 的区别是什么？`Partial` 和 `Required` 分别适合什么场景？`Record` 和普通对象类型有什么区别？`Exclude` 和 `Extract` 是怎么过滤联合类型的？`Parameters` 拿到的是什么类型？这些工具类型底层是怎么实现的？
- 回答雷区：不要把 `Required<T>` 说成 `Require<T>`，内置工具类型名是 `Required`；不要只列名字不说明用途；不要把 `Record<K, T>` 理解成运行时对象，它只是类型层面的映射；不要把函数提取类和对象字段变换类混在一起讲。

## 64. Webpack 与 Vite 的配置

- 一句话结论：Webpack 和 Vite 都能完成前端工程构建，但它们的配置重心不同，Webpack 更偏底层构建链路控制，Vite 更偏开发服务器、插件能力和少量关键构建选项。

- 30秒回答：Webpack 常见配置项有 `entry`、`output`、`module`、`plugins`、`mode`、`devServer`。`entry` 决定入口，`output` 决定产物输出位置和命名，`module` 里主要配 `loader`，`plugins` 负责扩展构建能力，`mode` 区分开发和生产模式，`devServer` 负责本地开发服务。Vite 配置相对轻一些，常见的是 `plugins`、`build`、`server`，也就是插件扩展、打包构建和开发服务器配置。面试里可以总结成一句话：Webpack 配置更全更细，Vite 配置更轻、更贴近现代前端开发体验。
- 1分钟展开：
  - **Webpack 的配置思路**：Webpack 是围绕“入口 -> 依赖图 -> loader 处理 -> plugin 扩展 -> 输出产物”这条链路来配置的，所以配置项更底层、更完整。
  - **Webpack 常见核心项**：`entry` 定义一个或多个入口；`output` 配置输出目录、文件名、公共路径；`module.rules` 负责告诉 Webpack 不同文件类型怎么交给不同 loader 处理；`plugins` 则做 HTML 生成、环境注入、代码压缩、CSS 提取等增强；`mode` 决定开发或生产优化策略；`devServer` 负责本地热更新、代理等开发体验。
  - **Vite 的配置思路**：Vite 默认给了很多现代前端能力，所以不需要像 Webpack 那样从零堆很多配置。常见配置更多是“在默认能力上做少量定制”。
  - **Vite 常见核心项**：`plugins` 用于框架支持和能力扩展；`server` 配开发服务器、端口、代理、HMR 等；`build` 配打包输出目录、chunk 策略、压缩方式、Rollup 选项等。
  - **两者差异怎么讲**：Webpack 配置更像搭建一条完整流水线，控制力很强；Vite 更像在现代默认配置上做增量调整，心智负担更低。
  - **实际回答技巧**：如果面试官追问配置，不要只背字段名，最好每个字段都顺带说它控制哪一段构建流程。
- 高频关键词：Webpack、Vite、配置项、`entry`、`output`、`module`、`loader`、`plugins`、`mode`、`devServer`、`build`、`server`、开发服务器、构建流程、打包配置
- 易追问：Webpack 的 `loader` 和 `plugin` 有什么区别？`entry` 和 `output` 分别决定什么？Vite 的 `build` 底层和 Rollup 是什么关系？为什么 Vite 的配置通常更少？`devServer` 和 `server` 都能做哪些事？什么时候你会更想用 Webpack？
- 回答雷区：不要把配置项只背成清单，最好解释它们对应哪一段流程；不要把 Vite 说成“没有打包配置”，它同样有 `build` 配置；不要把 Webpack 和 Vite 的配置复杂度简单理解成“一个好一个坏”，更多是设计理念不同。

## 65. Webpack 是如何打包的

- 一句话结论：Webpack 的打包本质是从入口出发，递归分析模块依赖，交给 loader 处理不同资源，再通过 plugin 扩展构建流程，最终把依赖图转成一个或多个可部署的静态资源文件。
- 30秒回答：Webpack 打包可以理解成四步。第一步，从一个或多个 **entry** 入口开始；第二步，解析代码里的依赖关系，递归构建模块依赖图；第三步，遇到不同类型文件时，通过 **loader** 把它们转换成 Webpack 能处理的模块；第四步，把整个依赖图按配置打成一个或多个 **bundle**，并输出到指定目录。过程中 **plugin** 会在不同生命周期节点介入，比如生成 HTML、压缩代码、抽离 CSS、注入环境变量等。
- 1分钟展开：
  - 从入口开始：Webpack 先读取配置里的 **entry**，把入口文件当成起点。一个项目可以有单入口，也可以有多入口。
  - 解析模块依赖：它会分析入口文件里的 **import**、**require** 等依赖声明，找到引用的其他模块，然后继续递归往下分析，最终形成一张完整的模块依赖图。
  - **loader** 处理非 JS 资源：Webpack 默认更擅长理解 JS 模块，但真实项目里还有 CSS、Less、图片、字体、TS、Vue/React 单文件组件等，所以要通过 **loader** 把这些资源转换成可被打包系统理解的模块。
  - **plugin** 扩展构建流程：plugin 不是处理单个文件，而是介入整个构建生命周期，做更广义的能力增强，比如清理目录、生成 HTML、代码压缩、提取 CSS、注入环境变量等。
  - 输出产物：依赖图构建完成后，Webpack 会根据 **splitChunks**、缓存策略、公共依赖提取等规则，把模块组织成一个或多个 **bundle**，输出到配置的目录里。
  - 为什么能做优化：因为它手里有整张依赖图，所以可以做代码拆分、Tree Shaking、公共模块提取、懒加载 **chunk** 划分等优化。
  - 回答思路：最清晰的说法就是“入口 -> 依赖图 -> loader 转换 -> plugin 扩展 -> 输出 bundle”。
- 高频关键词：Webpack、打包、入口、**entry**、依赖图、模块解析、**import**、**require**、**loader**、**plugin**、**bundle**、代码拆分、Tree Shaking、**splitChunks**、输出目录
- 易追问：Webpack 为什么需要依赖图？**loader** 和 **plugin** 的职责边界是什么？为什么 CSS/图片也能被 Webpack 打包？Tree Shaking 依赖什么前提？多入口和单入口有什么差异？Webpack 输出为什么可能不止一个 **bundle**？
- 回答雷区：不要把 Webpack 打包说成“把所有文件简单拼接起来”，核心是依赖图和模块系统；不要把 **loader** 和 **plugin** 混为一谈；不要忽略它对非 JS 资源的处理能力，这正是工程化价值的一部分；不要漏掉打包优化建立在依赖图之上。

## 66. Less 有哪些变量

- 一句话结论：Less 变量本质上是给样式里的颜色、尺寸、层级、间距等设计令牌起名字，让样式更统一、更可维护，尤其适合主题化和设计系统场景。
- 30秒回答：Less 变量最常见的写法是用 `@` 声明，比如 `@primary-color`、`@font-size-base`、`@spacing-md`。从分类上看，常见有主题色变量、文字色变量、背景色变量、边框和圆角变量、字号变量、间距变量、层级变量、布局尺寸变量。像你列的 `@primary-color`、`@text-color`、`@border-radius-base`、`@z-index-modal`、`@header-height`、`@sidebar-width` 都属于非常典型的设计系统变量。面试里重点不是背多少变量名，而是说明为什么要抽变量，以及它能解决主题统一和样式复用问题。
- 1分钟展开：
  - 变量的基本语法：Less 用 `@变量名: 值;` 来声明变量，比如 `@primary-color: #1677ff;`，后面可以在任意样式里复用这个值。
  - 颜色类变量：最常见的是主题色、成功色、警告色、危险色，以及正文、次级文字、禁用文字、背景色等。它们的意义是让视觉规范统一，而不是在项目里到处手写十六进制颜色。
  - 尺寸类变量：包括字体大小、边框圆角、间距、头部高度、侧边栏宽度等。像 `@font-size-base`、`@spacing-lg`、`@header-height` 这类变量，能让布局和排版更容易统一调整。
  - 层级类变量：比如 `@z-index-dropdown`、`@z-index-modal`，这种变量可以避免弹层、抽屉、对话框层级互相冲突。
  - 为什么要抽变量：当设计规范调整时，只要改少量变量定义，整套页面就能跟着变化，这比在很多组件里逐个改样式成本低得多。
  - 典型使用场景：主题换肤、组件库定制、后台系统统一设计语言、响应式布局的尺寸管理，都非常适合用 Less 变量。
  - 回答思路：可以按“语法 -> 变量分类 -> 为什么要用 -> 项目场景”来讲，而不是只念一串变量名。
- 高频关键词：Less、变量、`@primary-color`、`@success-color`、`@warning-color`、`@danger-color`、`@text-color`、`@bg-color`、`@border-color`、`@border-radius-base`、`@font-size-base`、`@spacing-md`、`@z-index-modal`、`@header-height`、设计令牌、主题化、样式复用
- 易追问：Less 变量和 CSS 自定义属性有什么区别？为什么设计系统里要抽主题变量？层级变量为什么重要？主题色和文字色变量通常怎么组织？Less 变量能不能参与运算？为什么很多组件库都暴露 Less 变量？

- 回答雷区：不要把 Less 变量理解成只能存颜色，它也能存尺寸、间距、层级等；不要只背变量名不解释分类意义；不要把 Less 变量和 CSS 运行时变量完全混为一谈，Less 变量主要发生在编译阶段；不要忽略它在主题定制和设计系统中的价值。

## 67. created 和 mounted 的区别

- 一句话结论：created 和 mounted 的核心区别在于，created 时组件实例和响应式数据已经就绪，但真实 DOM 还没挂载；mounted 则表示模板已经渲染并挂载到页面，可以安全操作 DOM。
- 30秒回答：如果是 Vue2，created 发生在实例创建完成后，这时候 data、methods、computed、watch 基本都可用了，但页面 DOM 还没挂载，所以适合发请求、初始化数据，不适合直接操作真实 DOM。mounted 发生在组件挂载完成后，DOM 已经渲染到页面上了，这时候适合做 DOM 操作、第三方库初始化、拿元素尺寸等。面试里还可以顺带说一句，Vue3 组合式 API 更常见的是 setup 和 onMounted，而不是继续强调 created。
- 1分钟展开：
  - Vue2 生命周期链路怎么理解：通常是 beforeCreate -> created -> beforeMount -> mounted -> beforeUpdate -> updated -> beforeDestroy -> destroyed。如果用了 keep-alive，还会有 activated / deactivated。
  - created 阶段有什么：组件实例已经建立，响应式数据、methods、computed、watch 等都已经可用，所以初始化状态、发接口请求、做数据准备都很常见。
  - created 阶段没有什么：这个阶段真实 DOM 还没有挂到页面上，所以不能依赖页面节点去拿宽高、调用 DOM API 或初始化依赖真实元素的第三方库。
  - mounted 阶段有什么：模板已经渲染完成并插入页面，\$el 可用，适合做 DOM 读写、图表库挂载、滚动位置处理、元素尺寸测量等。
  - Vue3 的对应关系：Vue3 里没有完全等价强调 created/mounted 的写法，组合式 API 里更常见的是 setup() 作为逻辑入口，再配合 onMounted()、onUpdated()、onBeforeUnmount()、onUnmounted()。如果是选项式 API，mounted 仍然存在。
  - 怎么回答更稳：你可以说“数据初始化更偏 created / setup，DOM 操作更偏 mounted / onMounted”，这会比单纯背生命周期更贴近实际开发。
  - 补充边界：beforeUpdate / updated 是数据更新阶段；Vue2 是 beforeDestroy / destroyed，Vue3 则是 beforeUnmount / unmounted，不要混用。

- 高频关键词：Vue、生命周期、`beforeCreate`、`created`、`beforeMount`、`mounted`、`beforeUpdate`、`updated`、`beforeDestroy`、`destroyed`、`beforeUnmount`、`unmounted`、`activated`、`deactivated`、`setup`、`onMounted`、DOM 挂载、数据初始化
- 易追问：为什么 `created` 可以发请求？为什么 `created` 不能操作 DOM？`mounted` 一定能拿到最终页面尺寸吗？Vue3 里 `created` 对应什么阶段？`setup` 和 `mounted` 的关系是什么？`activated` 和 `mounted` 有什么区别？`beforeDestroy` 和 `beforeUnmount` 是什么关系？
- 回答雷区：不要把 `created` 说成“数据加载完成”，更准确是实例和响应式能力就绪；不要把 `mounted` 说成“所有子异步内容都一定完成”，它主要表示当前组件已挂载；不要混淆 Vue2 和 Vue3 的销毁钩子命名；不要把 `setup`、`created`、`mounted` 混成同一阶段，它们各自职责不同。

## 68. 事件周期和事件委托

- 一句话结论：浏览器事件本质上遵循“捕获 -> 目标 -> 冒泡”的事件流，而事件委托就是利用冒泡机制，把多个子节点的事件统一交给祖先节点处理，从而减少监听器数量、提升动态节点管理和代码维护性。
- 30秒回答：前端事件流一般分三个阶段。第一是捕获阶段，事件从 `window/document` 一路往目标元素方向传递；第二是目标阶段，事件真正到达目标节点，此时 `event.target` 指向触发源；第三是冒泡阶段，事件再从目标节点向上冒泡到祖先节点。事件委托就是利用这个冒泡机制，把本来绑定在很多子节点上的事件，统一绑定到父节点或祖先节点上，通过判断 `event.target` 来决定具体处理谁。这样可以减少内存占用、减少重复绑定，也更适合动态新增节点场景。
- 1分钟展开：
  - 事件流怎么走：标准 DOM 事件流一般是捕获阶段、目标阶段、冒泡阶段。捕获是从外往内，冒泡是从内往外。面试里要区分 `event.target` 和 `event.currentTarget`，前者是谁触发了事件，后者是谁当前正在处理这个事件。
  - 事件委托是什么：本质上是把对子元素的处理逻辑委托给父元素。比如一个列表里有很多按钮，与其给每个按钮单独绑定点击事件，不如只给列表容器绑定一次，再根据点击源去分发逻辑。
  - 事件委托的优势：第一，减少大量重复监听器，节省内存；第二，动态新增子节点不需要重新绑事件；第三，逻辑收口到父节点，维护和统一修改更方便。
  - 阻止传播和阻止默认行为：`stopPropagation()` 用来阻止事件继续传播；`preventDefault()` 用来阻止浏览器默认行为，比如阻止链接跳转、表单提交。旧版兼容写法常见有 `window.event.cancelBubble = true` 和 `window.event.returnValue = false`。

- 事件绑定的常见方式：HTML 行内写法如 `onclick="..."`；DOM0 写法如 `el.onclick = fn`，这种方式同一事件通常只能保留一个处理函数；DOM2 写法是 `addEventListener(name, handler, options)`，可以绑定多个监听器，也能配置 `capture`、`once`、`passive` 等选项。
- 自定义事件：可以通过 `new CustomEvent('user-login', { detail: {...} })` 创建自定义事件，再用 `addEventListener` 监听，用 `dispatchEvent` 触发。这里要注意，`capture` 不是 `CustomEvent` 构造参数本身，而是监听时通过 `addEventListener` 的第三个参数去控制。
- Vue 里的事件补充：Vue 常见修饰符有 `.stop`、`.prevent`、`.self`、`.once`、`.capture`、`.passive`，本质上是对原生事件行为的语法封装。Vue3 组件通信里如果要声明自定义事件，组合式 API 常见写法是 `const emit = defineEmits<{ (e: 'change', id: number): void }>()`，然后通过 `emit('change', 1)` 触发。
- 回答思路：最稳的说法是“先讲事件流，再讲事件委托为什么成立，再补传播控制、绑定方式和框架里的映射关系”。
- 高频关键词：事件流、事件捕获、目标阶段、事件冒泡、`event.target`、`event.currentTarget`、事件委托、祖先节点、`stopPropagation()`、`preventDefault()`、`cancelBubble`、`returnValue`、`onclick`、`addEventListener`、`capture`、`once`、`passive`、`CustomEvent`、`dispatchEvent`、Vue 事件修饰符、`defineEmits`
- 易追问：`event.target` 和 `event.currentTarget` 有什么区别？事件委托为什么能成立？事件委托适合所有事件吗？`stopPropagation()` 和 `preventDefault()` 有什么区别？`addEventListener` 第三个参数能配什么？为什么 `passive` 对滚动性能有帮助？Vue 的 `.stop`、`.prevent` 本质上做了什么？
- 回答雷区：不要把事件流只说成“冒泡”，标准流程还有捕获和目标阶段；不要混淆 `target` 和 `currentTarget`；不要说事件委托适合所有事件，像某些不冒泡事件就要额外处理；不要把 `capture` 写到 `CustomEvent` 配置里，它属于监听阶段选项；不要把 Vue 修饰符当成独立事件机制，它本质还是对原生事件的封装。

## 69. CSS 中的 var 是什么？定义如何？

- 一句话结论：CSS 里的 `var()` 是用来读取 CSS 自定义属性的函数，常配合 `--变量名` 定义设计令牌，实现主题化、复用和运行时动态切换样式。

- 30秒回答: `var()` 不是单独定义变量的语法, 而是读取 CSS 变量的函数。真正定义变量时要写成 `--变量名: 值;`, 比如在 `:root` 里定义 `--primary-color: #000;`, 然后在具体样式里通过 `color: var(--primary-color);` 来使用。因为它是浏览器原生支持的运行时变量, 所以很适合做主题切换、暗黑模式、组件样式复用。面试里要注意语法别写错, `:root` 里要包在花括号里, 不是 `:root(--primary-color:#000)` 这种写法。
- 1分钟展开:
  - `var()` 是什么: `var()` 是 CSS 的一个函数, 用来读取自定义属性, 也就是平时说的 CSS 变量。它本身不是声明语法, 声明要靠 `--xxx`。
  - 怎么定义: 最常见是在根节点定义, 比如:  
`:root { --primary-color: #000; }`  
这样整个页面都可以访问这个变量。
  - 怎么使用: 在具体样式里写 `color: var(--primary-color);`, 也可以写默认值, 比如 `color: var(--primary-color, #333);`, 当前面的变量不存在时会用后备值。
  - 为什么常放在 `:root`: 因为 `:root` 接近全局作用域, 放在这里方便整个项目共享。如果变量只服务某个组件, 也可以定义在组件局部节点上。
  - 和预处理器变量的区别: Less/Sass 变量主要是编译时替换; CSS 自定义属性是运行时存在的, 可以配合 JS 动态修改, 也能参与主题切换和层级继承。
  - 典型场景: 主题色、字号、边距、圆角、暗黑模式、组件皮肤定制等, 都很适合用 CSS 变量来做。
  - 回答思路: 面试里可以按“定义语法 -> 使用方式 -> 为什么用 -> 和 Less/Sass 的区别”来讲。
- 高频关键词: CSS 变量、CSS 自定义属性、`var()`、`--primary-color`、`:root`、运行时变量、默认值、主题切换、暗黑模式、样式复用、设计令牌
- 易追问: `var()` 和 Less 变量有什么区别? 为什么常把变量定义在 `:root`? CSS 变量可以被继承吗? `var()` 的第二个参数有什么用? CSS 变量能不能被 JavaScript 修改? 什么时候更适合用 CSS 变量而不是 Sass/Less 变量?
- 回答雷区: 不要把 `var()` 说成“定义变量”的语法, 它是读取变量; 不要把 `:root` 写成函数调用形式, 正确写法是 `:root { --xx: value; }`; 不要把 CSS 变量和 Less/Sass 变量完全等同, 它们发生的时机不同; 不要忽略 `var()` 可以写默认值。

## 70. 在前端向后端传递参数时，如何通知后端 body 里面的数据类型？

- 一句话结论：前端通常通过请求头里的 `Content-Type` 告诉后端请求体是什么格式，后端再按对应协议去解析 body。
- 30秒回答：通知后端 body 数据类型，最核心就是设置 `Content-Type`。如果传 JSON，一般用 `application/json`；表单键值对常见是 `application/x-www-form-urlencoded`；文件上传通常是 `multipart/form-data`；纯文本则常用 `text/plain`。不同 `Content-Type` 会影响后端怎么解析请求体，也会影响前端该怎么组织数据。比如上传文件时通常要用 `FormData`，而且浏览器会自动补 boundary，这时候不要手动乱写 `multipart/form-data` 头。
- 1分钟展开：
  - `Content-Type` 是什么：它是 HTTP 请求头，告诉服务端“我 body 里的内容是什么格式”。服务端中间件通常就是根据这个头决定走哪种解析方式。
  - `application/json`：最常见的接口请求格式，body 通常是 JSON 字符串，适合结构化数据传输，比如新增、修改表单数据提交到接口。
  - `application/x-www-form-urlencoded`：更像传统表单提交格式，数据会被编码成 `key=value&key2=value2` 这种形式，后端也很常见。
  - `multipart/form-data`：常用于文件上传，因为它允许同时传文件和普通字段。真实项目里经常配合 `FormData` 使用。
  - `text/plain`：适合纯文本传输，比如简单文本内容、日志片段、某些特殊接口协议。你写的 `text-plain` 不准确，标准写法是 `text/plain`。
  - 为什么这件事重要：如果前端 body 内容和 `Content-Type` 对不上，后端就可能解析失败，出现拿不到参数、字段为空、文件上传异常等问题。
  - 回答思路：最稳的说法是“先讲 `Content-Type` 的作用，再按几种常见类型和场景举例”。
- 高频关键词：HTTP、请求头、`Content-Type`、`application/json`、`multipart/form-data`、`application/x-www-form-urlencoded`、`text/plain`、body、后端解析、`FormData`、boundary
- 易追问：为什么 JSON 要配 `application/json`？`multipart/form-data` 为什么常配合 `FormData`？上传文件时为什么不建议手动写 boundary？`application/json` 和 `application/x-www-form-urlencoded` 有什么区别？后端是怎么根据 `Content-Type` 解析 body 的？GET 请求为什么通常不强调 body？
- 回答雷区：不要把“通知后端数据类型”说成靠 query 参数或 URL，本题核心是请求头；不要把 `text/plain` 写错成 `text-plain`；不要在文件上传时机械手动设置 `multipart/form-data` 而忽略 boundary；不要只背 MIME 类型，不解释各自适用场景。

## 71. 时间复杂度和空间复杂度

- 一句话结论：时间复杂度描述的是输入规模变大时算法执行成本的增长趋势，空间复杂度描述的是算法运行时额外占用存储空间的增长趋势，两者都是评价算法效率的重要指标。
- 30秒回答：时间复杂度看的是“数据量变大后，执行次数怎么增长”，空间复杂度看的是“额外内存怎么增长”。面试里常用大 O 表示法，比如  $O(1)$ 、 $O(\log n)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 。如果按增长速度从低到高，一般是  $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$ 。要注意，大 O 看的是趋势，不是具体常数时间。
- 1分钟展开：
  - 时间复杂度是什么：它不是算代码跑了多少毫秒，而是描述当输入规模  $n$  增大时，算法执行次数大致怎么增长。所以它更适合比较算法级别的优劣，而不是直接替代性能测试。
  - 空间复杂度是什么：它关注的是算法为了完成计算，额外申请了多少辅助空间。这里强调的是“额外空间”，不是把输入数据本身都简单算进去。
  - 为什么用大 O 表示：大 O 会忽略低阶项和常数项，重点看主导增长趋势。比如  $O(2n + 3)$  一般记成  $O(n)$ ，因为当数据规模很大时，真正决定趋势的是  $n$  本身。
  - 常见复杂度大小规律：从好到差通常可以记成  $O(1)$ 、 $O(\log n)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 、 $O(n^3)$ 、 $O(2^n)$ 、 $O(n!)$ 。这里要注意， $n!$  的增长通常比  $2^n$  还快，所以顺序别说反了。
  - 怎么直观理解：遍历一次数组常见是  $O(n)$ ；双重循环很多时候是  $O(n^2)$ ；二分查找是  $O(\log n)$ ；递归暴力穷举、全排列这类问题很容易走到  $O(2^n)$  或  $O(n!)$ 。
  - 时间和空间的权衡：有些算法会用更多空间换更少时间，比如哈希表缓存；也有些算法为了省空间，会牺牲时间效率。所以两者常常要结合业务场景一起看。
  - 回答思路：面试里可以按“定义 -> 常见量级 -> 大小关系 -> 例子 -> 时间空间权衡”来讲。
- 高频关键词：时间复杂度、空间复杂度、大 O、 $O(1)$ 、 $O(\log n)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 、 $O(n^3)$ 、 $O(2^n)$ 、 $O(n!)$ 、增长趋势、额外空间、算法效率、时间换空间
- 易追问：为什么大 O 不看常数项？ $O(n \log n)$  为什么比  $O(n^2)$  更好？二分查找为什么是  $O(\log n)$ ？双重循环一定是  $O(n^2)$  吗？空间复杂度为什么强调额外空间？什么叫时间换空间？实际开发里为什么不能只看复杂度？

- 回答雷区：不要把时间复杂度理解成真实运行时间，它描述的是增长趋势；不要把空间复杂度理解成程序总内存占用，更准确是额外辅助空间；不要把  $2^n$  和  $n!$  的大小关系说反，通常是  $n!$  增长更快；不要只背定义，不举常见例子，面试里会显得很空。

## 72. WebSocket 和 MQTT 的区别

- 一句话结论：WebSocket 和 MQTT 不在同一层面上比较更准确，WebSocket 更像一个全双工实时通信通道，MQTT 则是一套基于发布/订阅模型的轻量消息协议；很多场景里 MQTT 甚至会跑在 WebSocket 之上。
- 30秒回答：如果面试问 WebSocket 和 MQTT 的区别，我会先说它们不是完全同类东西。WebSocket 解决的是浏览器和服务端如何建立一个持续的双向连接，让双方都能主动发消息；它本身不规定你的消息怎么组织、怎么分发，这些通常要靠业务自己定义。MQTT 则是一套消息协议，核心是 **topic**、发布者、订阅者和 **Broker** 中转分发。发布者把消息发到某个 **topic**，订阅者只订阅自己关心的 **topic**，不需要关心彼此。实际项目里，尤其是 Web 场景，MQTT 常常会承载在 WebSocket 之上。
- 1分钟展开：
  - 先区分层次：WebSocket 更偏传输通道或连接机制，重点是建立持久、全双工、低延迟通信；MQTT 更偏应用层消息协议，重点是消息如何组织、路由和分发。
  - **WebSocket** 的特点：建立连接后，客户端和服务端都可以主动发消息，非常适合实时聊天、实时通知、协同编辑、行情推送等场景。但 **WebSocket** 并不会替你定义“主题订阅”“消息等级”“谁该收到这条消息”，这些通常要你自己业务层设计。
  - **MQTT** 的特点：MQTT 天然发布/订阅模型，有 **Broker** 做消息中转，消息围绕 **topic** 进行路由。发布者发消息到某个主题，订阅者只接收自己订阅的主题，彼此解耦更强，尤其适合 IoT、设备通信、弱网、小包高频消息场景。
  - 消息分发能力差异：**WebSocket** 只是把消息送到连接对端，至于群发、按频道推送、按房间推送，通常要你自己实现；MQTT 则天然有 **topic** 和 **Broker** 机制，消息分发模型是协议自带的。
  - 资源和场景差异：MQTT 设计得更轻量，报文头更小，对低带宽和不稳定网络更友好；WebSocket 更通用，浏览器支持也更直接，所以在 Web 前端里更常见。
  - 两者如何组合：在浏览器环境里，MQTT 客户端往往不能直接像设备端那样走原始 TCP，这时常见做法就是“MQTT over WebSocket”，也就是用 **WebSocket** 作为传输层承载 MQTT 报文。
  - 怎么回答更稳：面试里最稳的说法不是简单对立，而是先讲“WebSocket 是通道，MQTT 是协议”，再补一句“MQTT 可以跑在 WebSocket 上”。

- 高频关键词：WebSocket、MQTT、全双工通信、长连接、实时通信、消息协议、发布订阅、topic、Broker、消息中转、解耦、IoT、低带宽、弱网、MQTT over WebSocket
- 易追问：WebSocket 为什么不自带发布订阅能力？MQTT 的 Broker 负责什么？为什么 MQTT 更适合 IoT 场景？MQTT 和 WebSocket 可以一起用吗？浏览器里为什么常见 MQTT over WebSocket？MQTT 的 topic 和 WebSocket 自己定义的房间路由有什么区别？
- 回答雷区：不要把 WebSocket 和 MQTT 说成完全同层协议，它们关注点不同；不要说 WebSocket 自带 topic 订阅机制，这通常是业务自己实现的；不要忽略 Broker 在 MQTT 里的核心作用；不要把“MQTT 跑在 WebSocket 上”理解成两者互相替代，更准确是协议和传输通道的组合关系。

## 73. 动态规划和贪心算法的区别

- 一句话结论：动态规划和贪心算法都在解决最优化问题，但动态规划会系统地比较并保存子问题结果，追求全局最优；贪心算法则在每一步直接做局部最优选择，前提是局部最优能够推出全局最优。
- 30秒回答：两者的核心区别在于决策方式。动态规划会把原问题拆成多个子问题，保存中间状态，避免重复计算，再通过状态转移推到全局最优；贪心算法则是每一步都选当前看起来最优的方案，不回头修改。动态规划通常适合背包、最短路径、最大子数组这类有重叠子问题和最优子结构的问题；贪心更适合活动选择、哈夫曼编码这类满足贪心选择性质的问题。简单说，DP 更稳但通常更重，贪心更快更简洁，但不是所有最优问题都能用。
- 1分钟展开：
  - 动态规划的核心思路：把问题拆成子问题，子问题之间往往有重叠，所以会把结果记录下来，比如用数组、表或记忆化缓存，避免重复求解。
  - 贪心的核心思路：每一步只看当前，直接做局部最优决策，希望最后能拼出全局最优。它不会像 DP 那样保存大量状态、全面比较多种路径。
  - 适用条件不同：动态规划通常依赖最优子结构和重叠子问题；贪心除了最优子结构外，还要求满足贪心选择性质，也就是“当前这一步选局部最优，不会破坏最终全局最优”。
  - 结果可靠性差异：动态规划通常是在枚举和比较后拿到全局最优；贪心只有在特定问题里才能保证全局最优，否则很容易得到局部最优但不是全局最优的结果。
  - 实现方式差异：DP 常见写法是递归 + 记忆化，或者自底向上的状态转移表；贪心通常写法更直接，按某种排序规则或优先选择规则一路往下做。
  - 复杂度感受：动态规划往往时间和空间都更高，因为它要保存状态；贪心通常更轻量，代码也更简洁，但前提是你能证明问题真的可以贪心。
  - 回答思路：面试里最稳的是按“决策方式 -> 适用条件 -> 是否保证全局最优 -> 典型例子”来讲。

- 高频关键词：动态规划、DP、贪心算法、最优子结构、重叠子问题、状态转移、记忆化、局部最优、全局最优、贪心选择性质、背包问题、最短路径、最大子数组、活动选择、哈夫曼编码
- 易追问：什么叫最优子结构？什么叫重叠子问题？为什么贪心不一定得到全局最优？什么问题更适合动态规划？什么叫贪心选择性质？背包问题为什么 0/1 背包通常用 DP、而部分背包可以用贪心？
- 回答雷区：不要把动态规划和贪心都简单说成“递归”；不要说贪心一定能得到最优解，它依赖问题性质；不要把“保存子问题结果”漏掉，这是 DP 的核心价值；不要把两者差异只讲成“一个快一个慢”，更关键是决策方式和正确性前提不同。

## 74. 数据库中，向量和索引的关系是什么？

- 一句话结论：向量是数据的表示形式，索引是为了加速查询的数据结构；在向量数据库或向量检索场景里，向量索引就是专门为高维向量相似度搜索设计的加速结构。
- 30秒回答：如果面试问向量和索引的关系，我会先区分两层。向量是把文本、图片、音频等数据编码成一组高维数字特征，用来表达语义或相似性；索引则是为了让查询更快。因为向量检索常常是高维相似度搜索，传统 B 树这类索引不适合，所以会使用专门的向量索引，比如 IVF、HNSW、LSH 这类近似最近邻结构。简单说，向量是“查什么”，索引是“怎么更快地查到”。
- 1分钟展开：
  - 向量是什么：在现代数据库或 AI 检索场景里，文本、图片、用户行为等数据常被编码成 **embedding** 向量。这个向量本质上是多维特征表示，距离越近通常表示语义越相似。
  - 索引是什么：索引是数据库为了加速查询建立的辅助结构。传统关系型数据库里常见的是 B+ 树、哈希索引；但这些结构主要适合精确查找、范围查找，不适合高维相似度搜索。
  - 为什么向量需要专门索引：因为向量搜索通常不是查“完全相等”，而是查“最相近的几个”。高维空间里直接全量扫描会非常慢，所以要用向量索引来缩小搜索范围，提高召回效率和响应速度。
  - 向量索引的作用：它本质是在高维空间里建立更适合近邻搜索的组织方式，让数据库能更快找到相似向量，而不是逐个暴力比对。
  - 常见向量索引思路：面试里可以提到近似最近邻（ANN）检索，常见结构有 HNSW、IVF、LSH 等。不同索引方案会在查询速度、内存占用、构建成本和召回率之间做权衡。
  - 应用场景：语义搜索、RAG、推荐系统、相似图片检索、问答匹配等场景，都会出现“先把数据转成向量，再通过向量索引加速查询”。

- **回答思路：**最稳的是按“向量是表示，索引是加速；向量检索本质是相似度搜索”这条线来讲。
- **高频关键词：**向量、**embedding**、高维特征、索引、向量索引、相似度搜索、最近邻搜索、ANN、HNSW、IVF、LSH、数据库、语义检索、RAG、推荐系统
- **易追问：**为什么 B+ 树不适合向量检索？什么叫近似最近邻？向量索引为什么能加速搜索？向量索引和普通数据库索引有什么区别？向量维度越高会带来什么问题？为什么向量检索常常是近似而不是绝对精确？
- **回答雷区：**不要把向量和索引混成同一个概念，一个是数据表示，一个是查询加速结构；不要把向量索引说成“只是一种普通索引”，它服务的是高维相似度搜索；不要泛泛只说 KD 树，现代向量数据库更常见的面试关键词是 ANN、HNSW、IVF；不要忽略向量检索通常关注的是相似度而不是精确匹配。

## 75. SQL 和 NoSQL 数据库的区别

- **一句话结论：**SQL 和 NoSQL 的核心区别不只是“有没有 SQL 语法”，而是数据模型、事务能力、扩展方式和适用场景不同，前者更强调结构化关系和强一致性，后者更强调灵活建模和水平扩展。
- **30秒回答：**SQL 数据库像 MySQL、PostgreSQL，基于关系模型，数据主要存在表里，适合结构稳定、关系复杂、事务要求高的场景，通常支持标准 SQL 和较强的 ACID 事务。NoSQL 则是非关系型数据库的统称，包含键值、文档、列族、图数据库等，比如 MongoDB、Redis、Cassandra。它们更强调灵活 schema、分布式扩展和海量数据处理，特别适合高并发、半结构化或非结构化数据场景。简单说，SQL 更适合强关系和事务型业务，NoSQL 更适合灵活扩展和特定模型场景。
- **1分钟展开：**
  - **数据模型不同：**SQL 数据库以表、行、列和关系为核心；NoSQL 则可能是文档、键值、列族、图等不同形态，数据组织方式更灵活。
  - **Schema 灵活性不同：**SQL 往往有相对明确的表结构和约束，字段设计更稳定；NoSQL 往往对 schema 更宽松，适合需求变化快、字段不固定的场景。
  - **事务和一致性：**SQL 数据库通常更强调 ACID 事务和强一致性，适合订单、支付、库存这类事务敏感业务；NoSQL 往往更强调可扩展性和高可用，部分系统会在一致性上做权衡。
  - **扩展方式不同：**传统 SQL 常见是纵向扩展或复杂分库分表；很多 NoSQL 从设计上更偏向水平扩展，更容易做分布式部署。
  - **查询能力差异：**SQL 的关系查询、连接查询、聚合查询通常更成熟；NoSQL 往往针对特定模型做了优化，但跨实体复杂关系查询可能不如关系型数据库自然。

- 应用场景：用户订单、财务系统、ERP 这类强事务业务常见 SQL；缓存、日志、内容管理、推荐、会话存储、海量文档型数据则常见 NoSQL。
- 回答思路：面试里不要只讲“一个表格一个非表格”，最好按“模型、事务、扩展、场景”来讲。
- 高频关键词：SQL、NoSQL、关系型数据库、非关系型数据库、MySQL、PostgreSQL、MongoDB、Cassandra、Redis、表结构、schema、ACID、事务、强一致性、水平扩展、文档数据库、键值数据库、列族数据库、图数据库
- 易追问：为什么 SQL 更适合事务型业务？NoSQL 为什么更容易水平扩展？MongoDB 属于哪类 NoSQL？Redis 算不算 NoSQL？NoSQL 就一定没有事务吗？关系型数据库为什么做复杂关联查询更自然？什么时候会同时用 SQL 和 NoSQL？
- 回答雷区：不要把 SQL 和 NoSQL 的区别讲成“一个能写 SQL，一个不能”，核心是模型和场景；不要把 NoSQL 一概说成“不支持事务”，不同产品能力不同；不要把 SQL 说成一定不能扩展、NoSQL 一定更好，关键看业务需求；不要忽略 NoSQL 其实是很多类型数据库的总称。

## 76. 多个 Web Worker 之间如何通信？

- 一句话结论：多个 Web Worker 通常不能像 DOM 节点那样直接互相访问，常见通信方式是通过主线程中转 `postMessage()`，或借助 `MessageChannel`、`BroadcastChannel`、`SharedWorker` 等机制；传大数据时还可以用 `Transferable` 或共享内存降低拷贝成本。
- 30秒回答：最常见的方式是主线程中转。也就是 Worker A 通过 `postMessage()` 把消息发给主线程，再由主线程转发给 Worker B。默认 `postMessage()` 走的是结构化克隆，不是简单引用传递，所以大对象会有拷贝成本。如果要减少这类开销，可以传 `Transferable`，比如 `ArrayBuffer`，把所有权直接转移过去；更进一步还可以用 `SharedArrayBuffer` 做共享内存。除了主线程中转，还可以用 `MessageChannel` 建立专门通道，或者在支持场景下用 `BroadcastChannel` 做广播通信。
- 1分钟展开：
  - 默认通信方式：Worker 和主线程之间最基础的通信 API 是 `postMessage()` / `onmessage`。多个 Worker 最常见的组织方式，就是由主线程充当消息中枢。
  - 为什么不是直接共享对象：默认消息传递走的是结构化克隆算法，会复制数据，而不是简单共享 JS 对象引用。所以你说的“值传递、深拷贝”方向是对的，但更准确地说是结构化克隆。
  - `Transferable` 的作用：像 `ArrayBuffer` 这类可转移对象，可以通过 `transferable list` 把所有权从一个上下文直接移交到另一个上下文，避免完整拷贝，提高性能。

- 共享内存方案：如果确实需要多个线程访问同一块数据，还可以用 `SharedArrayBuffer`。这不再是转移所有权，而是共享底层内存，但也会带来同步控制复杂度。
- 其他通信方式：`MessageChannel` 适合建立点对点消息通道；`BroadcastChannel` 适合多个上下文广播；`SharedWorker` 则适合多个页面或脚本共享同一个 `Worker` 实例。
- 应用场景：音视频处理、图像处理、大量计算、并行任务调度、数据分片处理时，多个 `Worker` 通信会比较常见。
- 回答思路：面试里可以按“默认 `postMessage` -> 结构化克隆 -> `Transferable` / `SharedArrayBuffer` 优化 -> 其他通道机制”来讲。
- 高频关键词：`Web Worker`、多线程、`postMessage()`、主线程中转、结构化克隆、深拷贝、`Transferable`、`ArrayBuffer`、`SharedArrayBuffer`、`MessageChannel`、`BroadcastChannel`、`SharedWorker`、并行计算
- 易追问：`postMessage()` 默认是引用传递还是拷贝？什么是结构化克隆？`ArrayBuffer` 为什么适合 `Transferable`？`SharedArrayBuffer` 和 `Transferable` 有什么区别？多个 `Worker` 能不能直接互相通信？`BroadcastChannel` 适合什么场景？为什么共享内存会更复杂？
- 回答雷区：不要把 `postMessage()` 说成普通对象引用共享，默认不是；不要把 `Transferable` 和共享内存混为一谈，一个是转移所有权，一个是共享底层内存；不要只说“多个 `Worker` 直接传”，更常见的是主线程中转；不要忽略 `MessageChannel`、`BroadcastChannel` 这些更完整的答案补充。

## 77. BOM 和 DOM 的区别

- 一句话结论：DOM 是文档对象模型，重点是把 HTML/XML 文档抽象成可操作的节点树；BOM 是浏览器对象模型，重点是浏览器窗口及其相关能力，比如地址栏、历史记录、屏幕信息等。
- 30秒回答：简单说，DOM 管的是页面内容，BOM 管的是浏览器环境。DOM 的核心入口通常是 `document`，我们平时用 `getElementById`、`querySelector`、修改节点、操作属性，本质都在操作 DOM。BOM 的核心入口通常是 `window`，像 `location`、`history`、`navigator`、`screen` 这些都属于 BOM 范畴。因为 `document` 本身是挂在 `window` 上的，所以很多人会把两者混在一起，但面试里最好区分：DOM 偏文档结构，BOM 偏浏览器宿主环境。
- 1分钟展开：
  - **DOM 是什么**：DOM 把页面文档解析成一棵对象树，里面有元素节点、文本节点、属性节点等。前端日常的节点查找、增删改、事件绑定，大多都和 DOM 有关。

- **BOM 是什么：**BOM 更偏浏览器提供的宿主环境对象模型，不是单纯页面内容本身。它让 JS 可以访问窗口、地址、历史、屏幕、定时器等浏览器能力。
- **常见 BOM 对象：**`window` 是顶层对象；`location` 管 URL 和跳转；`history` 管前进后退；`navigator` 管浏览器和设备信息；`screen` 管屏幕分辨率等。你写的 `document` 虽然挂在 `window` 上，但严格说它属于 DOM 的核心对象。
- **常见 DOM 操作：**`document.getElementById()`、`querySelector()`、`createElement()`、`appendChild()`、修改 `className`、`style`、`textContent` 等，都是典型 DOM 操作。
- **两者的关系：**DOM 和 BOM 并不是互斥的两个系统，而是浏览器环境下 JS 接触页面和浏览器能力的两套对象体系。BOM 提供宿主环境，DOM 提供文档结构访问能力。
- **为什么容易混淆：**因为平时写代码经常从 `window.document` 进入，导致很多人把 `document` 也当成 BOM 本体。更准确地说，`document` 是 DOM，`window` 是 BOM 的顶层入口。
- **回答思路：**面试里最稳的说法就是“DOM 管页面，BOM 管浏览器；`document` 偏 DOM，`window` 偏 BOM”。
- **高频关键词：**BOM、DOM、浏览器对象模型、文档对象模型、`window`、`document`、`location`、`navigator`、`history`、`screen`、DOM 树、节点操作、宿主环境
- **易追问：**为什么 `document` 不等于 BOM？`window` 和 `document` 是什么关系？`location`、`history`、`navigator` 分别做什么？DOM 操作和 BOM 操作各举几个例子？为什么说 BOM 是浏览器宿主环境提供的能力？
- **回答雷区：**不要把 `document` 直接归到 BOM，本质上它属于 DOM；不要把 BOM 讲成标准化非常完整统一的规范，它更多是浏览器宿主环境对象集合；不要只背对象名，不说明“页面内容”和“浏览器环境”这两个维度的区别。

## 78. 怎么进行两个页面通信？

- **一句话结论：**两个页面通信要先看它们的关系和同源条件，常见方案包括路由参数和全局状态、`localStorage/sessionStorage`、`storage` 事件、`BroadcastChannel`、`postMessage`，以及需要实时同步时通过 `WebSocket` 或后端中转。
- **30秒回答：**如果是单页应用内部页面通信，最常见的是路由参数、全局状态管理和浏览器存储；如果是多个同源标签页之间通信，可以用 `storage` 事件或者 `BroadcastChannel`；如果是父子窗口、`iframe` 或 `window.open` 打开的页面，最常用的是 `postMessage`，而且它支持跨域安全通信；如果是多端或更实时的同步，就可以走 `WebSocket` 或者让后端做中转。面试里最值得重点掌握的是 `postMessage`、`storage`、`BroadcastChannel` 和状态管理这几类。

- 1分钟展开：
  - 先看是不是 **SPA** 内部页面：如果其实只是同一个单页应用里的路由切换，那本质上不是两个独立浏览器上下文，最常见做法就是路由参数、查询参数、状态管理（如 **Pinia**、**Vuex**、**Redux**）或本地存储。
  - 同源多标签页通信：多个标签页如果是同源，可以借助 **localStorage** + **storage** 事件通信。一个页面写入数据，其他页面会收到 **storage** 事件。更直接的方式是 **BroadcastChannel**，它天然就是为同源多个上下文广播消息设计的。
  - 父子窗口 / **iframe** 通信：如果是父页面和 **iframe**，或者通过 **window.open** 打开的窗口，最常见的是 **postMessage**。它既支持同源也支持跨域场景，但要做好 **origin** 校验，避免安全问题。
  - 为什么 **postMessage** 重要：因为它是浏览器原生支持的跨上下文消息机制，适合页面嵌套、微前端、第三方页面嵌入、**OAuth** 登录回调等场景。
  - 需要实时同步怎么办：如果不只是两个页面互发一次消息，而是需要更实时、更复杂的多端同步，比如聊天室、协同编辑、通知中心，那通常会引入 **WebSocket**，或者通过后端做消息中转。
  - 存储方案的边界：**sessionStorage** 更偏单标签页会话隔离；**localStorage** 能跨同源标签页感知变化；但它们更适合轻量级状态同步，不适合高频实时通信。
  - 回答思路：面试里最稳的方式是按“SPA 内部 -> 同源多标签页 -> 父子窗口/iframe -> 实时跨端”分类来讲。
- 高频关键词：页面通信、路由参数、全局状态管理、**localStorage**、**sessionStorage**、**storage** 事件、**BroadcastChannel**、**postMessage**、**iframe**、**window.open**、同源通信、跨域通信、**WebSocket**、后端中转
- 易追问：**storage** 事件什么时候会触发？**BroadcastChannel** 和 **storage** 有什么区别？**postMessage** 为什么支持跨域？为什么要校验 **origin**？**sessionStorage** 和 **localStorage** 在多页面通信里有什么边界？什么时候更适合 **WebSocket** 而不是浏览器原生页面通信？
- 回答雷区：不要把所有页面通信都说成用 **localStorage**，要先区分场景；不要忽略 **postMessage** 的安全校验；不要把 SPA 内部路由切换和真正两个浏览器上下文混为一谈；不要把 **sessionStorage** 讲成天然适合跨标签页通信，它通常更偏单会话隔离。

## 79. 原型链和类介绍和区别

- 一句话结论：JavaScript 的继承本质建立在原型链之上，`class` 只是基于原型和构造函数的一层更清晰的语法糖；原型链强调底层对象委托关系，类语法强调更接近传统面向对象的书写方式和语义约束。
- 30秒回答：原型链可以理解成对象通过 `[[Prototype]]` 一层层向上查找属性和方法的链路。创建对象时，如果当前对象没有某个属性，就会顺着它的原型继续找，直到 `Object.prototype`。`class` 本质上还是建立在构造函数和原型机制上的语法糖，但写法更像传统类，比如可以直接写 `constructor`、实例方法、`extends`、`super`。两者区别在于：原型链更偏底层机制，类更偏语法表达；类方法本质仍然挂在 `prototype` 上，但类提供了更规范的继承写法，也避免了一些手动改原型时容易出错的问题。
- 1分钟展开：
  - 什么是原型链：每个对象内部都有一个 `[[Prototype]]` 指针，通常可以通过 `__proto__` 观察到。当访问某个属性时，JS 会先在对象本身找，找不到就沿着 `[[Prototype]]` 往上找，这条链就叫原型链。
  - `prototype` 和 `[[Prototype]]` 的区别：`prototype` 通常是函数对象上的一个属性，用来给实例提供共享方法；`[[Prototype]]` 则是对象内部的原型引用。很多人会把这两个概念混在一起，面试里要分清。
  - 原型方式创建和继承：可以通过构造函数 + `new`，也可以通过 `Object.create()` 让一个对象直接以另一个对象为原型。原型继承的好处是方法共享，但如果把引用类型属性直接挂到共享原型或父实例上，确实可能出现一个实例修改影响其他实例的问题。
  - 类是什么：`class` 是 ES6 提供的语法糖，本质还是构造函数、原型和原型链。类的方法默认定义在 `prototype` 上，实例属性在构造过程中初始化。
  - 类继承怎么理解：`class A extends B` 背后依然和原型链有关。实例这边有一条 `实例 -> 子类.prototype -> 父类.prototype` 的链；类本身作为函数对象也有一条 `子类 -> 父类` 的静态继承链，所以你提到的“两条链”方向是对的。
  - 两者差异：原型写法更底层、更灵活，但心智负担大；类写法更直观、更适合表达继承关系，并提供 `super`、类字段等现代语法。底层机制没变，但开发体验和可读性提高了。
  - 回答思路：面试里最稳的是“先讲原型链是什么，再讲 `class` 是语法糖，最后讲 `prototype`、`[[Prototype]]` 和继承关系”。
- 高频关键词：原型链、`[[Prototype]]`、`__proto__`、`prototype`、构造函数、`Object.create()`、对象委托、属性查找、`class`、语法糖、`extends`、`super`、实例方法、继承、两条链

- 易追问：`prototype` 和 `__proto__` 有什么区别？`Object.create()` 做了什么？为什么说 `class` 是语法糖？类方法最终放在哪里？为什么修改原型上的引用类型可能影响多个实例？`extends` 背后发生了什么？实例链和静态继承链分别是什么？
- 回答雷区：不要把原型链说成“对象的 `prototype` 属性一层层链接”，更准确是对象通过 `[[Prototype]]` 向上查找；不要把 `class` 说成全新继承模型，它底层仍然基于原型；不要混淆 `prototype`、`__proto__` 和 `[[Prototype]]`；不要忽略类继承里既有实例原型链，也有构造函数自身的静态继承链。

## 80. Vue3 的编译流程

- 一句话结论：Vue3 的编译流程本质上是把模板先解析成 AST，再在 AST 上做语法转换和性能优化，最后生成 `render` 函数；运行时再由 `render` 产出虚拟 DOM，并通过 `patch/diff` 把变化高效同步到真实 DOM。
- 30秒回答：Vue3 编译大致分三步。第一步是 `parse`，把 `template` 解析成 AST 语法树；第二步是 `transform`，遍历 AST，把插值、`v-if`、`v-for` 这类模板语法转成更底层的渲染表达，并顺手做静态提升、动态节点标记等优化；第三步是 `codegen`，把处理后的 AST 生成为 `render` 函数代码。运行时执行 `render` 会得到虚拟 DOM，然后再通过 `patch` 过程做 `diff`，把真正变化的部分更新到页面。面试里可以把它理解成“编译时尽量多做优化，运行时尽量少做无效更新”。
- 1分钟展开：
  - `parse` 阶段：Vue 会先把模板字符串解析成 AST，也就是一棵描述节点结构、属性、指令、文本和表达式的语法树。这个阶段的目标是“把模板变成可分析的数据结构”。
  - `transform` 阶段：这是 Vue3 编译优化的关键阶段。它会遍历 AST，把插值表达式、`v-if`、`v-for`、事件绑定、属性绑定等模板语法转成更接近 `render` 调用的结构。
  - `transform` 里的重要优化：一类是静态提升，也就是把完全不变的节点提前提出来复用，避免每次 `render` 重新创建；另一类是动态标记，也就是给真正会变化的节点打上 `PatchFlag`，运行时 `diff` 时可以更精准地更新。
  - 静态提升怎么理解：如果某个节点内容完全静态，Vue 会把它提到渲染函数外部或更高层作用域里，只生成一次，后续 `render` 直接复用，而不是每次重新构造整棵虚拟节点树。
  - `codegen` 阶段：经过转换和优化后的 AST，最终会被生成为 `render` 函数字符串或等价代码。这个 `render` 函数就是组件运行时真正执行的渲染逻辑。
  - 运行时阶段：`render` 执行后会产出虚拟 DOM，Vue 再通过 `patch` 过程把新旧 `vnode` 对比，决定哪些节点复用、更新、移动或删除，最后同步到真实 DOM。

- **虚拟 DOM 和 patch 的意义**：虚拟 DOM 不是“完全不操作 DOM”，而是让框架先在 JS 层描述视图变化，再尽量把真实 DOM 更新压缩到最小。Vue3 在 diff 上也做了很多优化，比如静态提升、PatchFlags、Block Tree，以及列表场景下更高效的 keyed diff。
- **Vue2 和 Vue3 diff 的回答边界**：面试里可以说两者本质都是同层节点比较、父节点相同再比较子节点、节点类型不同则替换子树；但 Vue3 因为编译期标记更多动态信息，运行时可以比 Vue2 更少做无效遍历。
- **回答思路**：最稳的说法就是“`parse -> transform -> codegen` 是编译阶段，`render -> virtual DOM -> patch` 是运行阶段”，再补一句 Vue3 把很多优化前置到了编译期。
- **高频关键词**：Vue3、编译流程、`parse`、AST、`transform`、`codegen`、render 函数、模板编译、插值表达式、`v-if`、`v-for`、静态提升、动态标记、PatchFlag、虚拟 DOM、patch、diff、Block Tree、keyed diff
- **易追问**：AST 是什么？为什么 `transform` 阶段最重要？静态提升解决了什么问题？什么是 PatchFlag？Vue3 为什么比 Vue2 diff 更高效？render 函数最终产出的是什么？虚拟 DOM 到真实 DOM 中间经历了什么？如果节点类型不同，patch 会怎么处理？
- **回答雷区**：不要把编译流程和运行流程混成一段，`parse/transform/codegen` 是编译阶段，`render/patch` 是运行阶段；不要把虚拟 DOM 讲成“完全不操作 DOM”，它只是减少无效 DOM 操作；不要把静态提升说成“缓存整个 render 函数”，更准确是提取静态节点复用；不要只背“Vue3 diff 更快”，最好点出编译期标记和运行时精准更新这层原因。

## 81. CSS 的继承属性有哪些

- **一句话结论**：CSS 里最容易继承的是文字、字体、排版相关属性，而盒模型、尺寸、边框、背景、定位布局类属性通常不会自动继承。
- **30秒回答**：面试里可以先记一个规律，不用死背所有属性。一般和“文字显示”强相关的属性大多会继承，比如 `color`、`font-size`、`font-family`、`font-weight`、`font-style`、`line-height`、`text-align` 这类；列表相关如 `list-style` 也常继承。反过来，和盒子模型、尺寸、边框、背景、定位布局相关的属性大多不继承，比如 `width`、`height`、`margin`、`padding`、`border`、`background`、`display`、`position`。开发里更重要的是理解这个规律，而不是机械背表。
- **1分钟展开**：
  - **继承是什么**：子元素在没有显式设置某个属性时，可能会沿用父元素的计算结果，这就是继承。它主要发生在那些和文本展示强相关的属性上。

- 典型可继承属性：`color`、`font` 及其拆分属性（如 `font-size`、`font-family`、`font-weight`、`font-style`）、`line-height`、`text-align`、`visibility`、`cursor`、`list-style` 系列等，都是高频可继承属性。
- 典型不可继承属性：`width`、`height`、`margin`、`padding`、`border`、`background`、`display`、`position`、`top/left/right/bottom`、`float`、`clear`、`z-index` 等，通常都不会因为父元素设置了就自动传给子元素。
- 为什么会这样：因为文字类属性天然更适合“父元素定规则，子元素沿用”；但盒子尺寸、边框、背景、定位如果自动继承，页面布局会非常难控制。
- 开发里的常见现象：很多人会发现父元素设置了 `color`、`font-size`，子元素会跟着变；但设置 `background-color` 或 `border` 时，子元素不会自动同步，这就是继承规则在起作用。
- 补充边界：有些属性本来不继承，但可以通过 `inherit` 强制继承；也可以用 `initial`、`unset`、`revert` 这类值控制最终表现。
- 回答思路：面试里最稳的是“先讲规律，再举可继承和不可继承的典型例子，再补一个 `inherit` 边界”。
- 高频关键词：CSS 继承、可继承属性、不可继承属性、文字属性、字体属性、排版属性、`color`、`font-size`、`font-family`、`line-height`、`list-style`、`width`、`height`、`margin`、`padding`、`border`、`background`、`display`、`position`、`inherit`
- 易追问：为什么文字类属性更容易继承？`background` 为什么默认不继承？`inherit`、`initial`、`unset` 分别是什么意思？`line-height` 为什么适合继承？有哪些属性看起来像排版但容易被忽略？开发里怎么快速判断一个属性是否会继承？
- 回答雷区：不要把“能继承”和“所有子元素都一定一样”混为一谈，子元素显式声明后会覆盖；不要死背零散属性却说不出规律；不要说所有文本相关属性都继承，最好举典型例子更稳；不要忽略 `inherit` 这种可以手动强制继承的边界。

## 82. 选择器优先级

- 一句话结论：CSS 选择器优先级本质是比较一组权重值，通常是行内样式高于 ID，ID 高于类/属性/伪类，类/属性/伪类高于标签/伪元素；`!important` 会进一步提升声明优先级，但它和选择器权重不是同一层概念。
- 30秒回答：面试里可以先给出常见顺序：`!important` 声明优先，其次通常是行内样式，再往下是 ID 选择器、类/属性/伪类选择器、标签/伪元素选择器，通配符权重最低。更准确地说，CSS 会按类似 `(a, b, c, d)` 的权重比较：行内样式一档，ID 一档，类/属性/伪类一档，标签/伪元素一档，前面的位数越大优先级越高。像并集选择器 `h1, h2`，不会叠加权重，而是各自独立。

计算；后代、子代、相邻兄弟这些组合关系本身不加权，真正起作用的是里面出现了哪些选择器。

- 1分钟展开：
  - 优先级怎么比较：常见理解方式是四元组权重。通常可以记成：行内样式最高一档，然后是 ID，接着是类/属性/伪类，最后是标签/伪元素。比较时从前往后逐位比，前面位数大就直接赢。
  - 常见顺序：`!important` 声明 > 行内样式 > `#id` > `.class` / `[attr]` / `:hover` > `div` / `::before` > `*`。这里要注意，`!important` 不是选择器类型，而是声明级别提升。
  - 并集和组合选择器的边界：并集选择器 `A, B` 是分别计算优先级，不会把 A 和 B 的权重加在一起。后代选择器、子代选择器 `>`、相邻兄弟 `+`、通用兄弟 `~` 这些组合符本身不增加优先级，关键还是组合里的每一部分选择器类型。
  - 为什么有时后写的会生效：如果两个规则优先级一样，最终会看源码顺序，后面声明的会覆盖前面的，这叫层叠规则，不是优先级更高。
  - 外部样式和优先级不是一回事：外部样式表、内部样式、行内样式属于样式来源和书写位置的区别，其中行内样式确实权重高，但“外部样式”本身不是一个单独的优先级层级。
  - 开发里的典型场景：很多样式覆盖失败，不是因为类名不对，而是因为对方规则里有更高权重的 ID、行内样式或 `!important`。
  - 回答思路：面试里最稳的是“先讲权重规则，再讲 `!important` 和层叠顺序边界，再补组合选择器怎么计算”。
- 高频关键词：CSS 优先级、选择器优先级、`specificity`、`!important`、行内样式、ID 选择器、类选择器、属性选择器、伪类、标签选择器、伪元素、通配符、层叠顺序、四元组
- 易追问：`!important` 和优先级是什么关系？并集选择器会叠加优先级吗？后代选择器 `div` `.box` 的优先级怎么算？为什么同样的选择器后写的会生效？伪类和伪元素分别算哪一档？外部样式为什么不等于低优先级？
- 回答雷区：不要把 `!important`、行内样式、外部样式、选择器优先级混成一条简单链，它们不完全是同一个维度；不要说组合符本身会增加权重，真正加权的是选择器本体；不要把并集选择器当成权重累加；不要只背“谁大谁小”，最好能说出四元组比较思路。

## 83. JS 有哪些数据类型

- 一句话结论：JavaScript 数据类型可以先分成基本类型和引用类型，基本类型有 `number`、`string`、`boolean`、`null`、`undefined`、`symbol`、`bigint`，引用类型最常见统称为 `object`。

- 30秒回答：JS 里常见的基本类型有 7 个：`number`、`string`、`boolean`、`null`、`undefined`、`symbol`、`bigint`。除了这些之外，其余像对象、数组、函数、日期、正则，本质上都属于引用类型，统一可以归到 `Object` 体系里。面试里如果继续追问类型判断，最基础的是 `typeof`，但它有坑，比如 `typeof null` 是 `object`；判断数组可以用 `Array.isArray()`，判断实例关系常用 `instanceof`，更稳妥的通用方式是 `Object.prototype.toString.call(value)`。如果面试官顺带问 TypeScript，则可以再补 `enum`、`void`、`any`、`unknown`、`never`、`interface` 这些是 TS 类型系统里的概念，不是 JS 运行时原生数据类型。
- 1分钟展开：
  - 基本类型有哪些：`number` 表示数值，`string` 表示字符串，`boolean` 表示布尔值，`undefined` 表示未定义，`null` 表示空值，`symbol` 用来创建唯一标识，`bigint` 用来表示超大整数。
  - 引用类型怎么理解：对象、数组、函数、日期、正则等在语言层面都可以看作对象类型。也就是说，数组和函数虽然平时单独讲很多，但底层仍然在 `Object` 这个大类里。
  - 栈和堆怎么讲更稳：面试里常见说法是“基本类型值更接近直接存储，引用类型变量保存的是对象引用，对象实体通常在堆内存里”。但这更偏引擎实现层面的简化管理，不要把它说成 ECMAScript 规范里写死的存储规则。
  - `typeof` 的边界：`typeof` 很方便，但有经典坑，比如 `typeof null` 返回 `object`；`typeof function() {}` 返回 `function`，这是对象里的一个特殊情况。
  - `instanceof` 的边界：它更适合判断对象实例和原型链关系，比如数组、函数、自定义类实例；但对基本类型通常不适合作为主判断方法。
  - 更稳的通用判断方式：`Object.prototype.toString.call(value)` 能区分更多内建对象类型，比如 `[object Array]`、`[object Date]`、`[object RegExp]`。
  - TS 类型的边界：你列的 `enum`、`void`、`any`、`unknown`、`never`、`typeof`、`interface` 属于 TypeScript 类型系统能力，不是 JS 运行时原生数据类型，面试里最好区分这两个层面。
  - 回答思路：最稳的说法是“先分基本类型和引用类型，再讲类型判断，再补 TS 和 JS 的边界”。
- 高频关键词：JavaScript、数据类型、基本类型、引用类型、`number`、`string`、`boolean`、`null`、`undefined`、`symbol`、`bigint`、`Object`、`typeof`、`instanceof`、`Object.prototype.toString.call()`、`Array.isArray()`、TypeScript

- 易追问：`typeof null` 为什么是 `object`？数组属于什么类型？函数属于基本类型还是引用类型？`instanceof` 为什么能判断对象？`Array.isArray()` 和 `instanceof Array` 有什么区别？JS 数据类型和 TS 类型系统有什么区别？`symbol` 和 `bigint` 分别解决什么问题？
- 回答雷区：不要把数组、函数单独讲成和 `Object` 完全并列的数据大类；不要把 TS 类型系统能力说成 JS 原生数据类型；不要机械背“基本类型在栈、引用类型在堆”而不说明这是简化理解；不要漏掉 `symbol` 和 `bigint` 这两个 ES6+ 后的重要类型。

## 84. 数组常用方法，是否改变原数组

- 一句话结论：数组方法最常见的考点不是名字本身，而是要分清哪些会原地修改原数组，哪些只是返回新结果不改原数组。
- 30秒回答：会改变原数组的常见方法有 `push`、`pop`、`shift`、`unshift`、`splice`、`sort`、`reverse`；不会改变原数组的常见方法有 `slice`、`map`、`filter`、`reduce`。`forEach` 一般不返回新数组，本身也不会直接改原数组，但如果你在回调里手动改数组元素，那结果还是会变。面试里最容易写错的是 `splice` 和 `slice`，前者改原数组，后者不改。
- 1分钟展开：
  - 会改原数组的方法：`push` 末尾新增，`pop` 末尾删除，`shift` 头部删除，`unshift` 头部新增，`splice` 按索引插入/删除/替换，`sort` 排序，`reverse` 反转，这些都会直接修改原数组。
  - 不改原数组的方法：`slice` 截取并返回新数组，`map` 映射得到新数组，`filter` 过滤得到新数组，`reduce` 聚合出一个结果值，通常不直接改原数组。
  - `forEach` 的边界：`forEach` 本身返回 `undefined`，不是为了生成新数组。它默认不帮你改原数组，但如果你在回调里主动修改原数组元素或外部状态，它也可以造成副作用。
  - `splice` 和 `slice` 区别：这是高频追问。`splice` 更像手术刀，直接在原数组上动刀；`slice` 更像复制切片，拿走一段副本，不动原数组。
  - 排序方法的边界：`sort` 会改原数组，而且默认按字符串编码顺序排序，所以数字排序常要自己传比较函数，比如 `(a, b) => a - b`。
  - 为什么这个题常考：因为它和响应式更新、不可变数据、React/Vue 状态管理都有关。你如果误把会改原数组的方法直接用在状态里，很容易引发 bug 或渲染问题。
  - 回答思路：最稳的方式是“先分改不改原数组，再挑 `splice/slice`、`sort`、`forEach` 这些高频边界单独解释”。

- 高频关键词：数组方法、修改原数组、不修改原数组、`push`、`pop`、`shift`、`unshift`、`splice`、`sort`、`reverse`、`slice`、`map`、`forEach`、`reduce`、`filter`、副作用、不可变数据
- 易追问：`splice` 和 `slice` 有什么区别？`sort` 会不会改原数组？为什么 `forEach` 不适合做返回值链式处理？`map` 和 `forEach` 的核心区别是什么？`reduce` 为什么通常不改原数组？为什么在状态管理里要特别注意会改原数组的方法？
- 回答雷区：不要把 `splice` 拼成 `splite`；不要说 `forEach` 一定不改原数组，更准确是它本身不直接改，但回调里可以造成副作用；不要把 `sort` 当成返回新数组的方法，它会改原数组；不要只背清单，最好顺手补 `splice/slice` 和 `sort` 的边界。

## 85. 异步队列和后端并发优化

- 一句话结论：前端里的异步队列本质上是一种“削峰填谷”的调度手段，常见做法是基于 `Promise` 把大量异步任务排队或限流执行，从而避免请求瞬时打满后端；而 `Promise` 本身则提供了统一的异步状态管理和链式编排能力。
- 30秒回答：如果后端瞬时并发很大，前端可以通过异步队列把请求、上传、重试任务排队执行，或者限制并发数，比如一次只放行 3 个、5 个任务，等前面的完成后再继续，这样可以避免请求雪崩、减轻服务压力。实现这种队列时，通常会基于 `Promise` 做任务包装，因为 `Promise` 有三种状态、状态不可逆、支持链式调用、错误统一处理，而且回调是异步执行的。要注意，`Promise/A+` 里的 `thenable` 规则主要解决链式调用和结果传递问题，但 `Promise` 本身并不原生支持取消。
- 1分钟展开：
  - **`Promise` 的核心特点**：`Promise` 有 `pending`、`fulfilled`、`rejected` 三种状态，而且一旦从 `pending` 变成成功或失败就不可逆。它符合 `Promise/A+` 的 `then` 规范，可以链式调用，也能通过 `catch/then` 统一处理错误。
  - **`thenable` 是什么**：`Promise/A+` 更关注的是“只要对象有 `then` 方法，就可以按 `thenable` 规则吸收其结果”，这也是为什么 `Promise` 链能把异步值层层传递下去。
  - **回调为什么是异步执行**：`Promise` 的 `then/catch/finally` 回调会放到微任务队列里异步执行，所以不会在当前同步代码栈里立刻触发。
  - **为什么需要异步队列**：如果前端一次性发出大量请求、上传或批处理任务，后端可能在瞬时被压垮，出现接口超时、限流、排队堆积甚至雪崩。前端排队或限流能起到“削峰”作用。
  - **异步队列怎么做**：常见思路是维护一个待执行任务队列，再设置一个最大并发数。每次只取前几个任务执行，任务完成后再从队列里补位。这样既保留异步吞吐，又不会无限并发。

- **适用场景：**大批量文件上传、批量接口请求、图片预处理、导入导出轮询、重试队列、消息发送等场景都很常见。
- **补充边界：**前端异步队列只是缓解手段，不是根治方案。真正的并发治理还要配合后端限流、熔断、重试策略、消息队列、削峰填谷等服务端设计。
- **回答思路：**面试里最稳的是“先讲 **Promise** 的异步编排能力，再讲为什么要做前端队列限流，最后补和后端并发治理的配合关系”。
- **高频关键词：**异步队列、**Promise**、**Promise/A+**、**thenable**、三种状态、状态不可逆、链式调用、错误统一处理、微任务、不可取消、并发控制、限流、削峰填谷、请求雪崩、上传队列、任务调度
- **易追问：****Promise/A+** 规范主要约束什么？为什么 **Promise** 回调是异步执行的？前端怎么做并发限制？异步队列和 **Promise.all** 有什么区别？为什么说 **Promise** 不能原生取消？前端限流和后端限流分别解决什么问题？什么场景适合做上传队列？
- **回答雷区：**不要把 **Promise** 说成“就是异步队列”，它更准确是异步结果抽象和链式编排工具；不要把 **thenable** 简化成“普通对象”，关键是它有 **then** 方法；不要说 **Promise** 天然支持取消，标准 **Promise** 并没有这个能力；不要把前端限流当成后端并发治理的全部，它只是协同优化的一环。

## 86. HTTPS 加密原理和 CA 证书流程

- **一句话结论：**HTTPS 的核心不是“全程都用证书公钥直接加密”，而是通过 **TLS** 握手先完成身份校验和密钥协商，再用对称加密传输数据；CA 证书体系的作用则是证明服务器公钥确实属于这个域名对应的服务器。
- **30秒回答：**HTTPS 建立连接时，客户端先发 **ClientHello**，服务端返回 **ServerHello** 和数字证书。客户端会先校证书是否合法，比如证书链、域名、有效期、签名是否可信；验证通过后，双方再通过密钥交换协商出会话密钥，后续真正的数据传输主要用这个对称密钥加密，因为它效率更高。CA 证书流程本质上是：服务器生成密钥对和 CSR，提交给 CA；CA 审核后，用自己的私钥对证书内容做数字签名；客户端收到证书后，用内置的 CA 公钥去验签，确认这个证书没有被篡改，而且确实来自受信任 CA。
- **1分钟展开：**
  - **为什么 HTTPS 不直接全程非对称加密：**非对称加密安全但开销大，如果所有数据都用公私钥加密，性能会很差。所以 HTTPS 通常先用握手完成身份验证和密钥协商，再用对称加密传输业务数据。
  - **TLS 握手大致流程：**客户端发 **ClientHello**，带上支持的 TLS 版本、加密套件、随机数等；服务端返回 **ServerHello**，选择参数并发送证书。之后双方完成密钥交换和握手确认，最终得到同一个会话密钥。

- **数字证书里有什么：**证书通常包含服务器公钥、域名信息、颁发者、有效期、签名等内容。客户端最关心的是：这个公钥是不是可信 CA 背书的，且是不是当前访问域名的合法证书。
- **CA 证书签发流程：**服务器先生成公私钥和 CSR（证书签名请求），把 CSR 提交给 CA；CA 审核域名或企业身份后，生成证书，并用 CA 的私钥给证书签名。这里准确说法是“签名”，不是简单说“用私钥加密整个证书”。
- **客户端怎么校验：**客户端会根据本地信任链找到 CA 公钥，验证证书签名是否正确，同时检查证书链、域名、有效期、吊销状态等。如果这些检查失败，浏览器就会报证书不安全。
- **密钥协商怎么理解：**在更现代的 TLS（如 TLS 1.3）里，更多是通过密钥交换算法协商出共享密钥，而不是简单“客户端拿服务器公钥把最终会话密钥直接加密过去”。这是面试里比较稳的说法。
- **后续数据怎么传：**握手完成后，双方用协商出的对称密钥做加密通信。这样既保证安全，也兼顾性能。
- **回答思路：**最稳的是“握手流程 -> 证书校验 -> 密钥协商 -> 对称加密传输”，再补一句 CA 的本质作用是建立公钥信任链。
- **高频关键词：**HTTPS、TLS、SSL、`ClientHello`、`ServerHello`、数字证书、CA、CSR、证书签名、证书链、公钥、私钥、验签、密钥协商、对称加密、非对称加密、会话密钥、域名校验、TLS 1.3
- **易追问：**为什么 HTTPS 还要用对称加密？CA 的私钥和公钥分别做什么？证书里包含哪些内容？浏览器如何验证证书合法性？为什么会出现证书不受信任？TLS 1.2 和 TLS 1.3 在密钥协商上有什么方向性变化？如果证书校验失败会发生什么？
- **回答雷区：**不要把 HTTPS 讲成“公钥加密所有数据”，真正业务数据通常走对称加密；不要把 CA 签名说成“直接私钥加密整个证书内容”，更准确是对证书摘要进行数字签名；不要把“客户端用服务器公钥加密会话密钥”讲成唯一通用流程，现代 TLS 更强调密钥协商；不要忽略客户端除了验签，还会校证书链、域名和有效期。

## 87. Vue 路由鉴权

- **一句话结论：**Vue 路由鉴权本质上是利用路由守卫在页面跳转前后拦截导航，根据登录态、角色权限或路由元信息决定是否放行、重定向或终止跳转。
- **30秒回答：**做 Vue 路由鉴权，最常见的是在全局前置守卫 `beforeEach` 里判断用户有没有登录、有没有权限，再结合路由 `meta` 信息决定要不要放行。路由级守卫常见是 `beforeEnter`，更适合某个页面自己的特殊限制；全局解析守卫 `beforeResolve` 一般在所有组件和路由守卫都通过之后触发；跳转完成后会进 `afterEach`。如果是组件离开前拦截，比如表单未保存提示，通常会用 `beforeRouteLeave`。面试里可以概括成一句：登录态判断多用 `beforeEach`，页面离开确认多用 `beforeRouteLeave`，埋点收尾多用 `afterEach`。

- 1分钟展开：
  - 为什么路由鉴权常用守卫：因为路由守卫发生在导航过程里，最适合做“还没进页面前就先判断”的逻辑，比如是否登录、是否有角色权限、是否需要重定向到登录页。
  - `beforeEach` 的时机：它是全局前置守卫，每次路由切换几乎都会先经过它，所以最常见的登录鉴权都放在这里，比如检测 `token`、用户信息、白名单页面等。
  - `beforeEnter` 的时机：这是路由独享守卫，只对某个具体路由生效，适合某个页面的特殊访问限制，不会像全局守卫那样影响所有页面。
  - `beforeResolve` 的时机：它会在所有组件内守卫、异步组件解析等都完成后、导航最终确认前触发。适合做“最后一道确认”，比如某些依赖异步数据准备完成后的准入判断。
  - `beforeRouteLeave` 的时机：这是组件离开前守卫，常见场景是“表单还没保存，确定离开吗”。它不主要用来做登录鉴权，但在导航控制题里经常一起考。
  - `afterEach` 的时机：导航确认完成后触发，不能中断路由，通常适合做埋点、页面标题更新、日志记录等收尾动作。
  - 鉴权常见实现思路：登录后把 `token` 或会话状态存起来，路由配置里用 `meta.requiresAuth`、`meta.roles` 之类字段标记权限要求，在 `beforeEach` 里统一判断，不满足就 `next('/login')` 或返回指定页面。
  - 回答思路：面试里最稳的是“先讲全局守卫做登录鉴权，再讲路由独享守卫、组件离开守卫和后置守卫的触发时机与职责”。
- 高频关键词：Vue Router、路由鉴权、路由守卫、`beforeEach`、`beforeEnter`、`beforeResolve`、`beforeRouteLeave`、`afterEach`、登录态、权限控制、`meta.requiresAuth`、重定向、导航拦截、白名单
- 易追问：`beforeEach` 和 `beforeEnter` 有什么区别？`beforeResolve` 为什么比 `beforeEach` 更晚？`afterEach` 为什么不能拦截路由？`beforeRouteLeave` 适合什么场景？路由权限一般放在 `meta` 里还是硬编码？如果用户刷新页面，登录态怎么恢复？前端路由鉴权为什么不能代替后端鉴权？
- 回答雷区：不要把所有守卫都讲成“做登录判断”，它们的职责不同；不要说 `afterEach` 能阻止跳转，它只做收尾；不要忽略前端鉴权只是用户体验层，真正权限校验仍要靠后端；不要把守卫时机顺序讲乱，面试里至少要把 `beforeEach`、`beforeEnter`、`beforeResolve`、`afterEach` 区分开。

## 88. Webpack 无法加载 png 图该怎么办？

- 一句话结论：Webpack 加载不了 png，通常不是“图片坏了”，而是资源处理链没配好，排查时优先看资源模块或 loader 配置、引用路径、输出目录和缓存问题。
- 30秒回答：如果 Webpack 无法加载 png，我一般按四步排查。第一，看资源处理配置是否完整，老版本常看 `file-loader`、`url-loader`，新版本 Webpack 5 更常见的是 `asset/resource` 或 `asset`；第二，看图片引用路径对不对，是相对路径、别名还是 `public` 路径出了问题；第三，看打包后的输出目录里有没有生成图片文件，以及 HTML/CSS/JS 里引用地址是不是正确；第四，清理缓存重新构建，排除缓存脏数据。面试里可以总结成一句：先查 loader 或资源模块，再查路径和产物，再查缓存。
- 1分钟展开：
  - 先看资源处理配置：Webpack 本身不会天然理解所有静态资源，所以要确认 png 是否被正确规则匹配。老项目常见是 `file-loader`、`url-loader`；Webpack 5 里很多场景已经改用内置的 Asset Modules。
  - Webpack 5 的新思路：如果是新项目，优先检查 `type: 'asset/resource'`、`asset/inline`、`asset` 这类配置，而不是只盯着旧版 loader。面试里提到这一点会更稳。
  - 路径问题很常见：比如 `import img from './a.png'` 路径写错、别名没配对、CSS 里的 `url()` 相对路径基准不对、`publicPath` 配置不对，都可能导致打包后图片 404。
  - 看产物有没有真正输出：有时构建过程没报错，但输出目录里根本没生成图片文件，或者图片生成了但引用地址不对，所以要同时检查“有没有产物”和“产物地址是否正确”。
  - 清理缓存再构建：缓存不一致、热更新残留、依赖升级后旧缓存没清干净，也会导致资源加载异常，所以可以删除构建缓存重新打包验证。
  - 还可以排查什么：比如 loader 安装版本和 Webpack 版本是否匹配、规则匹配顺序有没有问题、图片是否被错误排除、大小写路径在不同系统下是否一致等。
  - 回答思路：最稳的说法是“配置链 -> 路径 -> 产物 -> 缓存”四步排查法，再补一句 Webpack 5 更推荐 Asset Modules。
- 高频关键词：Webpack、png、静态资源、`file-loader`、`url-loader`、Asset Modules、`asset/resource`、路径问题、`publicPath`、输出目录、构建缓存、重新构建、图片 404
- 易追问：Webpack 5 为什么很多场景不再用 `file-loader`？`url-loader` 和 `file-loader` 有什么区别？`asset/resource` 和 `asset/inline` 有什么区别？为什么路径没错但图片还是 404？`publicPath` 会影响什么？CSS 里的 `url()` 路径和 JS 里的 `import` 路径有什么区别？

- 回答雷区：不要把所有问题都归结成“没装 loader”，Webpack 5 可能根本不需要旧版 loader；不要只查源码路径，不看打包后的输出和引用地址；不要忽略缓存和 `publicPath` 这种高频隐患；不要把“无法加载图片”简单说成浏览器问题，更多是构建链和资源路径问题。

## 89. OpenLayers 如何获取经纬度坐标？

- 一句话结论：OpenLayers 里点击地图拿到的 `event.coordinate` 默认通常是地图当前投影坐标，常见 Web 墨卡托是 `EPSG:3857`，如果要拿经纬度，需要再转换成 `EPSG:4326`；而像素坐标则可以通过 `map.getPixelFromCoordinate()` 获取。
- 30秒回答：在 OpenLayers 里获取地图点击坐标，最常见就是监听点击事件后拿 `event.coordinate`。但这个值默认一般不是经纬度，而是地图视图使用的投影坐标，很多项目默认是 `EPSG:3857`。如果面试官问经纬度，稳妥答法是再用 `transform(coordinate, 'EPSG:3857', 'EPSG:4326')` 转成经纬度；如果要拿屏幕像素位置，就用 `map.getPixelFromCoordinate(coordinate)`。
- 1分钟展开：
  - `event.coordinate` 是什么：它表示当前点击点在地图坐标系里的坐标，不是浏览器 DOM 像素点，也不一定直接就是经纬度。
  - 为什么常见要做坐标转换：很多 Web 地图底层用的是 Web Mercator，也就是 `EPSG:3857`，它更适合地图渲染和瓦片计算；但业务里经常需要的是地理坐标，经纬度一般对应 `EPSG:4326`。
  - 怎么转成经纬度：常见写法是 `transform(coordinate, 'EPSG:3857', 'EPSG:4326')`。转换后得到的通常是 `[lon, lat]`，也就是先经度后纬度，这个顺序别说反。
  - 像素坐标怎么拿：如果需求是做弹窗定位、浮层偏移、命中检测或者和页面元素联动，就不是要经纬度，而是要像素坐标，这时用 `map.getPixelFromCoordinate(coordinate)` 更合适。
  - 两个坐标体系别混：地图坐标是地理空间里的位置，像素坐标是当前屏幕渲染后的点位。缩放、拖拽之后，同一个地理坐标对应的像素位置会变化。
  - 回答思路：面试里最稳的是“先说 `event.coordinate` 先拿地图坐标，再补默认投影可能是 `EPSG:3857`，如果要经纬度就转 `EPSG:4326`，如果要屏幕点位就拿 pixel”。
- 高频关键词：OpenLayers、`event.coordinate`、坐标获取、地图点击、投影坐标、`EPSG:3857`、`EPSG:4326`、`transform`、经纬度、Web Mercator、像素坐标、`map.getPixelFromCoordinate()`

- 易追问：为什么 `event.coordinate` 默认不是经纬度？`EPSG:3857` 和 `EPSG:4326` 有什么区别？转换后的坐标顺序是 `[lat, lon]` 还是 `[lon, lat]`？什么时候需要像素坐标而不是经纬度？`getPixelFromCoordinate()` 和 `getCoordinateFromPixel()` 有什么区别？
- 回答雷区：不要把 `event.coordinate` 直接说成“就是经纬度”；不要把 `EPSG:3857` 和 `EPSG:4326` 混为一谈；不要把经纬度顺序说反，通常是先经度后纬度；不要把地图坐标和屏幕像素坐标当成同一种数据。

## 90. `Object.assign(target.prototype, source.prototype)` 实现了什么功能？

- 一句话结论：`Object.assign(target.prototype, source.prototype)` 的本质是把 `source.prototype` 上可枚举的自有属性浅拷贝到 `target.prototype` 上，常用于“混入方法”，但它不是原型链继承，也不等价于 `new`。
- 30秒回答：这句代码最核心的作用，是把 `source.prototype` 上的方法合并到 `target.prototype`，让 `target` 的实例也能访问这些方法。它做的是浅拷贝，且只拷贝可枚举自有属性，所以不会改 `target.prototype.__proto__`，也不会建立 `target` 和 `source` 之间的原型链关系。面试里要强调，它更像 `mixin`，而不是继承；和 `new` 的区别也很大，`new` 会基于原型链创建实例并执行构造函数，这句代码不会。
- 1分钟展开：
  - 它到底复制了什么：`Object.assign` 会把 `source.prototype` 上可枚举的自有属性，按键值复制到 `target.prototype` 上。如果同名属性已存在，后者会被覆盖。
  - 为什么说不是继承：这句代码不会让 `target.prototype.__proto__` 指向 `source.prototype`，所以不会形成真正的原型链继承。它只是把方法“拷贝过去”，不是把原型关系“连起来”。
  - 为什么说是浅拷贝：如果属性值本身是对象，复制的是引用，不会递归深拷贝；而且它复制的是属性值，不会完整保留属性描述符语义。
  - 和 `new` 的区别：`new Source()` 会创建一个新对象，把这个对象的原型指向 `Source.prototype`，然后执行构造函数逻辑；而 `Object.assign(target.prototype, source.prototype)` 既不会创建实例，也不会调用 `source` 构造函数。
  - 常见用途：它更适合做方法复用、原型方法混入、给已有类补能力，比如把一组工具方法挂到另一个构造函数的原型上。

- 边界要点: `constructor` 通常是不可枚举的, 所以很多时候不会被 `Object.assign` 拷过去; 如果你想做真正继承, 更常见的是 `target.prototype = Object.create(source.prototype)` 再修正 `constructor`。
- 回答思路: 面试里最稳的是“先定性为浅拷贝原型方法, 再强调不改原型链、不调用构造函数、不是 `new`, 最后补一句它更像 `mixin` 而不是真继承”。
- 高频关键词: `Object.assign`、`prototype`、浅拷贝、可枚举属性、自有属性、原型方法、方法混入、`mixin`、原型链、`__proto__`、继承、`new`、`Object.create`
- 易追问: 它会不会修改 `target.prototype.__proto__`? 为什么说它不是继承? `constructor` 会不会一起复制过去? 如果同名方法冲突怎么办? 它和 `Object.create(source.prototype)` 有什么区别? 它和 `new Source()` 的核心差异是什么?
- 回答雷区: 不要说成“把 `source.prototype` 挂到了 `target.prototype` 上”, 更准确是复制属性; 不要把它当作真正继承方案, 它不会改原型链; 不要忽略 `Object.assign` 只做浅拷贝且只拷贝可枚举自有属性; 不要把它和 `new` 混谈, `new` 会创建实例并执行构造函数, 这句代码不会。

## 91. `new Object()` 和 `Object.create(null)` 创建的对象有什么本质区别? 为什么做字典时常推荐后者?

- 一句话结论: `new Object()` 或 `{}` 创建的是带 `Object.prototype` 的普通对象, 而 `Object.create(null)` 创建的是没有原型链的纯字典对象; 当它被拿来存用户输入键名时, 后者更能避免原型链属性干扰和键名冲突。
- 30秒回答: `new Object()` 和 `{}` 本质上都会得到一个普通对象, 它的原型指向 `Object.prototype`, 所以天然能访问 `toString`、`hasOwnProperty`、`constructor` 这类方法。 `Object.create(null)` 不一样, 它创建出来的对象没有原型, 是一个“干净字典”。做 Map/Hash 字典时, 推荐它的原因是可以避免键名和原型链上的属性冲突, 比如用户传入 `toString`、`constructor`、`__proto__` 这类 key 时, 更不容易引发覆盖、判断失真或遍历歧义。但也要注意, 它没有 `hasOwnProperty` 这些原型方法, 判断自有属性通常要用 `Object.hasOwn()` 或 `Object.prototype.hasOwnProperty.call()`。
- 1分钟展开:
  - 本质区别在原型链: `new Object()` 和 `{}` 的原型一般都是 `Object.prototype`, 所以会继承一批默认方法; `Object.create(null)` 的原型是 `null`, 没有这条原型链。
  - 为什么说它更像纯字典: 因为它只存你自己塞进去的键值, 不会混入 `toString`、`valueOf`、`constructor` 这些继承属性, 结构更“干净”。

- 做字典时为什么容易踩坑：如果用普通对象做 key-value 存储，业务 key 可能和原型链属性重名，比如 `toString`、`hasOwnProperty`、`constructor`，这会让读取、判断、自定义遍历甚至安全处理出现歧义。
- `Object.create(null)` 的直接收益：它避免了原型链污染，也减少了和内置属性同名的风险，尤其适合做配置表、缓存表、计数器、按字符串分组的字典对象。
- 它的代价也要会讲：因为没有原型，所以不能直接调用 `obj.hasOwnProperty(key)`、`obj.toString()` 这类方法。面试里如果讲到这一点会更完整。
- 属性判断怎么写更稳：现代写法优先用 `Object.hasOwn(obj, key)`；如果要兼容旧环境，可以写 `Object.prototype.hasOwnProperty.call(obj, key)`。
- 它不等于真正的 `Map`：即使用 `Object.create(null)`，键本质上仍主要是字符串或 `Symbol`。如果场景需要任意类型键、稳定的插入顺序、内建的 `size`、更清晰的增删查语义，面试里应该补一句“更推荐直接用 `Map`”。
- 回答思路：最稳的答法是“先讲原型链差异，再讲为什么做字典更安全，最后补使用边界和 `Map` 的适用场景”。
- 高频关键词：`new Object()`、`{}`、`Object.create(null)`、原型链、`Object.prototype`、纯字典、键名冲突、原型污染、`toString`、`constructor`、`__proto__`、`Object.hasOwn()`、`hasOwnProperty.call()`、`Map`、Hash 字典
- 易追问：`Object.create(null)` 为什么拿不到 `hasOwnProperty`？`{}` 和 `new Object()` 有什么区别？为什么说它更适合做字典而不一定适合做普通对象？它和 `Map` 到底怎么选？`for...in` 遍历时两者有什么感知差异？为什么 `__proto__` 这类 key 会成为风险点？
- 回答雷区：不要把 `Object.create(null)` 说成“性能一定更高”，核心价值是结构更纯净、冲突更少；不要忽略它没有原型方法这个副作用；不要把它直接等同于 `Map`，两者的键类型能力和 API 设计并不一样；不要只说“防止覆盖 `toString`”，更完整的说法是避免原型链属性干扰和字典语义污染。

## 92. JavaScript 垃圾回收机制和内存泄露问题

- 一句话结论：垃圾回收本质上是引擎自动识别“已经不可再访问”的对象并回收内存，而内存泄露不是“内存不够了”，而是本该释放的对象因为仍然可达而一直无法被回收，长期堆积后才可能引发卡顿甚至 OOM。
- 30秒回答：JavaScript 的内存生命周期可以概括为分配、使用、释放。释放这一步大多不是手动做，而是靠垃圾回收器根据“可达性”判断对象是否还在使用中，也就是从根对象出发，比如全局对象、当前调用栈里的局部变量、闭包里仍被引用的数据，能不能继续访问到某个对象。现代引擎核心不是靠引用计数，而是以标记清除、标记整理和分代回收为主。新生代对

象回收频繁，新生代对象回收成本更高。内存泄露常见在全局变量、闭包、定时器、事件监听、游离 DOM、以及 `Map/Set` 强引用没清理这些场景。GC 本身会消耗时间，所以现代引擎会配合增量、并行、并发等方式降低卡顿。

- 1分钟展开：

- 什么是垃圾回收：就是自动识别程序里已经不再需要的内存并释放，避免无效对象长期占用堆内存。JS 开发者通常不直接 `free` 内存，而是依赖引擎做这件事。
- 内存生命周期怎么讲：对象先被创建并分配内存，然后在业务逻辑里被读写使用，最后在不再被引用时等待 GC 回收。面试里用“分配 -> 使用 -> 释放”这条线最稳。
- 核心判断标准是可达性：GC 会从一组根对象出发向外查找引用链，常见根包括全局对象、执行栈中的局部变量、函数调用链仍持有的引用等。一个对象如果从这些根出发已经不可达，才有机会被回收。
- 常见算法怎么区分：引用计数好理解，但解决不了循环引用问题；现代主流更常见的是标记清除，先标记可达对象，再清掉未标记对象；如果要减少内存碎片，还会配合标记整理，把存活对象挪到一起。
- 分代回收思想很重要：大多数对象“生得快死得也快”，所以新生代会更频繁回收，新生代回收次数更少但成本更高。以 V8 为例，可以把它理解成对象先在新生代分配，存活多轮 GC 后再晋升到老年代；你提到的 `nursery -> intermediate -> old generation` 可以作为 V8 语境下的回答补充。
- 新生代和老年代怎么答：新生代空间更小、回收更频繁，常见会提到 `From/To` 半区和 `Scavenge` 思路；对象经历几轮存活后会晋升。老年代更常配合标记清除、标记整理；当老年代存活对象增长到某个启发式阈值以上时，可能触发 Major GC。
- 什么叫内存泄露：不是说程序立刻崩了，也不是单纯内存占用高，而是“本来业务上已经没用了，但因为仍被引用而保持可达”，导致 GC 无法回收。
- 常见泄露场景：无意挂到全局上的变量、闭包里长期持有大对象、`setInterval`/定时器没清、事件监听没移除、DOM 已删但 JS 还引用着、`Map/Set` 持续缓存对象却不清理，都是高频考点。
- GC 对性能的影响：GC 会抢占时间，尤其大对象多、老年代回收时更明显，所以浏览器和 V8 会通过增量回收、并行回收、并发回收等方式降低主线程停顿，但不能简单说“GC 完全不阻塞页面”。
- 回答思路：最稳的是“生命周期 -> 可达性 -> 算法 -> 分代 -> 泄露定义 -> 常见场景 -> 性能影响”。

- 高频关键词：垃圾回收、GC、内存生命周期、分配、使用、释放、可达性、根对象、引用计数、循环引用、标记清除、标记整理、分代回收、新生代、老生代、**From/To**、Scavenge、Major GC、V8、内存泄露、游离 DOM、**Map**、**Set**
- 易追问：为什么引用计数解决不了循环引用？标记清除和标记整理有什么区别？什么算根对象？为什么闭包可能导致内存泄露？**Map** 和 **WeakMap** 在泄露问题上有什么差异？什么是游离 DOM？Minor GC 和 Major GC 有什么区别？前端怎么排查内存泄露？
- 回答雷区：不要把“不可达”说成“立刻回收”，GC 触发有时机和策略；不要把内存泄露等同于内存溢出，泄露是原因，OOM 可能是结果；不要把引用计数讲成现代 JS 引擎的唯一主方案，它更多是帮助理解历史思路；不要说 GC 完全不占主线程，更准确是现代引擎会尽量通过增量、并行、并发方式减少停顿。

## 93. 怎么根据 DOM 树和 CSSOM 树生成渲染树？

- 一句话结论：渲染树本质上是“DOM 中需要渲染的节点 + 这些节点最终计算后的样式”，生成时会过滤掉不参与渲染的节点，比如 **display: none**，但像 **visibility: hidden** 这种节点仍会保留并参与布局。
- 30秒回答：浏览器先把 HTML 解析成 DOM，把 CSS 解析成 CSSOM，这两个过程通常可以并行推进。等合并时，会遍历 DOM，过滤掉不需要渲染的节点，然后给保留下来的节点做样式计算，也就是根据 CSSOM、继承、层叠规则、优先级算出最终样式。这样得到的“可见结构 + 计算样式”就是渲染树。渲染树不是完整 DOM 树，后面浏览器再基于它做布局、绘制和合成。要特别区分：**display: none** 不进渲染树，**visibility: hidden** 会进渲染树并参与布局，只是不显示。
- 1分钟展开：
  - 先构建两棵树：HTML 被解析成 DOM，CSS 被解析成 CSSOM。可以理解成结构信息和样式信息分别准备好，后面再合并；但如果 CSSOM 没准备好，浏览器没法完成最终样式计算，所以首屏渲染会被样式表影响。
  - 合并前先做节点过滤：不是所有 DOM 节点都会进入渲染树。像 **display: none** 的节点及其子树不会生成渲染对象，一些纯元数据节点也不会参与页面可视渲染，所以 DOM 树和渲染树不是一一对应关系。
  - 再做样式计算：浏览器会给每个需要渲染的 DOM 节点匹配 CSS 规则，结合继承、层叠、优先级，得到最终的 **computed style**。可以把 CSSOM 理解成样式规则来源，而不是直接一条规则对应一个渲染树节点。
  - 生成渲染树：把“需要渲染的节点”和“这些节点的最终计算样式”绑定起来，就得到 **render tree**。它描述的是页面最终要怎么显示，而不是源码里原始的结构。

- 渲染树之后还没结束：后面还会继续做 Layout、Paint、Composite。也就是先根据渲染树算几何位置和大小，再绘制像素，最后把不同图层合成到屏幕上。
- 高频边界要会讲：`display: none` 直接不进渲染树，也不参与布局；`visibility: hidden` 会保留在渲染树里，也会参与布局，只是最终不显示。这是面试里最常被追问的区别。
- 高频关键词：DOM、CSSOM、渲染树、Render Tree、HTML 解析、CSS 解析、并行构建、样式计算、computed style、层叠、继承、优先级、节点过滤、`display: none`、`visibility: hidden`、Layout、Paint、Composite
- 易追问：DOM 树和渲染树为什么不是一一对应？CSS 会不会阻塞 DOM 解析？CSSOM 为什么会影响首屏渲染？`display: none` 和 `visibility: hidden` 为什么表现不同？渲染树生成后为什么还要做布局和绘制？回流和重绘分别发生在哪一步？
- 回答雷区：不要把渲染树说成“完整 DOM 树加样式”，它会过滤部分节点；不要说“每条 CSS 规则都会变成一个渲染树节点”，更准确是节点会去匹配规则并计算最终样式；不要把 `visibility: hidden` 说成不在渲染树里，它会参与布局；也不要忽略渲染树后面还有布局、绘制、合成三个阶段。

## 94. 什么是 DNS 劫持？有哪些常见类型？

- 一句话结论：DNS 劫持本质上是在域名解析阶段篡改结果，让用户输入正确域名却被导向错误 IP 或恶意站点；常见位置有本地 DNS 劫持、路由器 DNS 劫持和上游 DNS 服务器劫持。
- 30秒回答：DNS 劫持就是攻击者不改你输入的域名，而是改“域名解析到哪个 IP”这一步。这样用户访问 `a.com`，实际可能被带到钓鱼站、广告页或者伪装站点。常见有三类：第一，本地 DNS 劫持，比如改 `hosts`、改本机 DNS 配置、恶意软件污染本地解析；第二，路由器 DNS 劫持，攻击者改路由器 DNS 设置，整个局域网设备都会受影响；第三，上游 DNS 服务器劫持，也就是递归 DNS、运营商 DNS 或权威 DNS 侧被篡改。它和普通 HTTP 重定向不一样，问题发生在真正发请求之前的解析阶段。
- 1分钟展开：
  - 先说本质：用户访问网站前，浏览器和系统要先把域名解析成 IP。DNS 劫持就是在这条解析链路上做手脚，让“看起来访问的是对的域名”，但实际连到的是错误目标。
  - 本地 DNS 劫持：最常见的是 `hosts` 文件被改、本机 DNS 配置被恶意修改、恶意软件拦截解析或污染本地 DNS 缓存。这类问题通常只影响当前设备。
  - 路由器 DNS 劫持：如果家用或办公路由器后台被入侵，攻击者可以直接把 DNS 服务器地址改掉。这样同一路由器下的所有终端，访问相同域名时都可能被解析到错误地址。

- **上游 DNS 服务器劫持：**比如运营商 DNS、公共递归 DNS、企业内部 DNS，甚至权威 DNS 被篡改或配置被接管。它的影响范围通常更大，可能不是一台设备出问题，而是一批用户一起中招。
- **用户和前端侧怎么感知：**表象通常是访问正确域名却跳到假站、广告页、异常登录页，也可能出现证书报错、静态资源加载到陌生域名、接口请求被导到错误服务。HTTPS 能提高安全性，但不能阻止“被导向错误地址”这件事；它更多是让用户更容易看到证书异常，降低内容被悄悄伪装成功的概率。
- **面试里顺手补一句防护：**高频说法是用 HTTPS/HSTS、保护路由器后台口令、使用可信 DNS、必要时用 DoH/DoT、关键域名配 DNSSEC，并区分故障点是在本机、路由器还是上游 DNS。
- **高频关键词：**DNS 劫持、DNS Hijacking、域名解析、解析结果篡改、本地 DNS、`hosts`、本机 DNS 配置、路由器 DNS、递归 DNS、运营商 DNS、公共 DNS、权威 DNS、钓鱼网站、证书报错、HTTPS、HSTS、DoH、DNSSEC
- **易追问：**DNS 劫持和 DNS 污染、DNS 缓存投毒有什么区别？为什么说它不是普通重定向？HTTPS 能不能完全防住 DNS 劫持？如何排查是本地问题、路由器问题还是上游 DNS 问题？为什么同一个 Wi-Fi 下多台设备同时跳错站？`hosts` 文件为什么也算本地劫持场景？
- **回答雷区：**不要把 DNS 劫持讲成 HTTP 302 或前端路由跳转，它发生在解析阶段；不要只说“访问到恶意网站”，广告注入、假登录页、流量导流也很常见；不要说 HTTPS 能彻底消灭 DNS 劫持，更准确是它能增强校验和告警，但不能阻止解析结果先被改掉；也不要本地、路由器、上游 DNS 三个层级混在一起说。

## 95. 后端接口没开发好时，前端怎么测试和推进开发？

- **一句话结论：**后端接口没就绪时，前端不会被动等人，而是基于接口契约用 Mock 数据、本地微型服务和统一请求层切换把页面、交互和异常流先跑通，等真实接口接入时尽量做到低成本替换。
- **30秒回答：**我的做法一般分四层。第一，用 `Mock.js`、本地 JSON 或请求拦截先模拟接口返回，保证页面开发能继续，并把成功、空数据、权限不足、参数错误、服务异常这些场景一起覆盖掉；第二，如果要更接近真实联调，我会用 `Express`、`Koa` 或 `json-server` 起一个本地微型服务，把路由、参数、状态码和响应结构都模拟出来；第三，把 `mock` 和真实接口的切换放在统一请求层，用环境变量控制，不让业务代码到处改；第四，整个过程以接口文档和数据契约为中心，避免 `mock` 跟真实接口脱节。
- **1分钟展开：**
  - **先基于契约，不是随便造数据：**核心不是“写一份假数据先糊页面”，而是以前后端约定的 URL、请求参数、状态码、返回结构为中心推进开发，这样后面联调时才不会大面积返工。

- **Mock 先解决开发阻塞**：常见方式有 Mock.js、本地 JSON 文件、devServer/Vite 请求拦截、独立 mock 模块。它适合先把页面渲染、交互逻辑、空态和错误提示跑通。
- **不要只模拟成功态**：我做 mock 时不会只写 200 成功，而是会统一返回结构，并覆盖空列表、401/403 权限不足、400 参数错误、500 服务异常、超时或异常 message 等情况。这样前端的异常处理不会等联调时才暴露。
- **复杂场景用本地微型服务更真实**：如果页面依赖分页、筛选、搜索、表单提交、登录态或不同状态流转，我会用 Express、Koa 或 json-server 起本地服务，模拟真实的接口路由、查询参数、状态码和动态返回，比单纯写静态 JSON 更接近联调环境。
- **按业务场景覆盖边界值**：列表页会模拟分页、总数、状态字段、空页和边界页；表单会模拟校验失败、重复数据、幂等冲突；登录场景会模拟 token、用户信息、权限字段和过期态。这些都属于面试里比较加分的“工程意识”。
- **统一请求层做切换**：把 mock/real 的切换收口到请求封装层或网关配置层，通过环境变量或配置开关控制，业务组件只关心调接口，不关心当前是 mock 还是真实后端。这样切到真实接口时改动最小。
- **回答思路**：最稳的顺序是“契约驱动 -> Mock 跑通 -> 本地服务逼真模拟 -> 统一请求层切换 -> 覆盖异常和边界场景”。
- **高频关键词**：Mock 数据、Mock.js、本地 JSON、请求拦截、json-server、Express、Koa、本地微型服务、接口契约、契约驱动开发、统一请求层、环境变量切换、成功态、空数据、权限不足、参数错误、服务异常、分页、表单校验、token、权限字段
- **易追问**：为什么不直接在组件里写死假数据？Mock 和本地微型服务怎么选？为什么要把切换放在请求层？如何避免 mock 和真实接口不一致？列表页和表单页分别要重点模拟哪些异常？联调时发现接口字段和契约不一致怎么办？
- **回答雷区**：不要把“接口没好”理解成只能等后端，前端完全可以先基于契约推进；不要只做成功态 mock，不覆盖异常流；不要把 mock 数据散落在页面组件里，后续切换真实接口会很痛；也不要脱离接口协议自己随意设计字段，否则联调时最容易全部推倒重来。

## 96. 如何判断一个空对象？

- **一句话结论**：如果面试语境指“没有自有可枚举属性的普通对象”，最常见也最稳的写法是 `Object.keys(obj).length === 0`；但要先明确“空”的定义，因为 Symbol、不可枚举属性、原型链属性和 `JSON.stringify` 都会影响判断结果。
- **30秒回答**：判断空对象，最常见是 `Object.keys(obj).length === 0`，因为它拿的是对象自身可枚举的字符串键；`Object.entries(obj).length === 0` 也可以，本质类似。`for...in` 也能判断，但要配合 `hasOwnProperty` 或 `Object.hasOwnProperty()`，否则会把原型链上的属性也算进去。`for...of` 不能直接遍历普通对象，必须先配合 `Object.keys()` 或

`Object.entries()`。`JSON.stringify(obj) === '{}'` 不推荐当通用方案，因为 `undefined`、函数、`Symbol` 会被忽略，容易误判。要是面试官继续追问更严格的判断，我会补一句：如果连 `Symbol` 键也算，就用 `Reflect.ownKeys(obj).length === 0`。

- 1分钟展开：

- 先定义什么叫“空”：大多数面试默认指“这个对象没有自有可枚举属性”，这时 `Object.keys(obj).length === 0` 就够用了；但如果题目要求更严格，比如连 `Symbol` 键、不可枚举属性也要算，那判断方式要升级。
  - `Object.keys()` 是最常见答案：它返回对象自身所有可枚举的字符串键，所以写法简单、语义清晰。面试里如果没有特殊边界要求，优先答这个最稳。
  - `Object.entries()` 本质一样：`Object.entries(obj).length === 0` 也能判断，因为它拿的是自身可枚举的键值对，只是比 `keys` 多组了一层值，通常没有必要专门为了判空用它。
  - `for...in` 能用但要小心原型链：`for...in` 会遍历可枚举属性，但不仅包含自身属性，还可能拿到原型链上的可枚举属性。所以如果要用，通常要写 `for (const key in obj) { if (Object.hasOwn(obj, key)) return false }`。
  - `for...of` 不能直接遍历普通对象：普通对象默认不是 `iterable`，直接 `for...of obj` 会报错。它只能配合 `Object.keys(obj)`、`Object.values(obj)` 或 `Object.entries(obj)` 使用，所以它本身不是“判断空对象”的直接方案。
  - `JSON.stringify()` 不可靠：比如 `{ a: undefined }`、`{ a: function() {} }`、带 `Symbol` 键的对象，序列化后都可能变成 `'{}'`；另外循环引用还会直接报错。所以这更像临时观察手段，不适合作为标准答案。
  - 更严格的版本怎么答：如果你想判断“完全没有自有键”，包括字符串键、`Symbol` 键和不可枚举属性，可以用 `Reflect.ownKeys(obj).length === 0`。这句在高阶追问里比较加分。
  - 再补一个边界：如果题目限定“普通对象”，很多人会补 `obj && obj.constructor === Object`；但像 `Object.create(null)` 创建的对象没有 `constructor`，所以更稳的表达是先问清楚对象类型范围。
- 高频关键词：空对象、`Object.keys()`、`Object.entries()`、`for...in`、`for...of`、`JSON.stringify()`、`Reflect.ownKeys()`、可枚举属性、自有属性、原型链、`Object.hasOwn()`、`hasOwnProperty`、`Symbol`、不可枚举属性、plain object
  - 易追问：`Object.keys()` 和 `Object.entries()` 有什么区别？为什么 `for...in` 可能误判？为什么 `for...of` 不能直接遍历对象？为什么 `JSON.stringify()` 不可靠？如果对象有 `Symbol` 键怎么办？`Object.create(null)` 创建的对象该怎么判断？空对象和“没有可枚举属性的对象”是同一个概念吗？

- 回答雷区：不要把 `JSON.stringify(obj) === '{}'` 说成通用标准，它会漏掉 `undefined`、函数、`Symbol` 等场景；不要直接用 `for...in` 却不排除原型链属性；不要说 `for...of` 可以直接遍历普通对象；也不要忽略“空对象”到底是指没有可枚举键，还是完全没有任何自有键，这两个定义并不完全一样。

## 97. 前端怎么解决网络延迟的问题？

- 一句话结论：前端解决网络延迟，本质上不是只靠一个点优化，而是同时减少请求次数、减小资源体积、缩短关键链路等待，并利用缓存、CDN 和更高效的传输协议把首屏和交互请求尽量提前、尽量变少、尽量变快。
- 30秒回答：我一般从四个方向做。第一，减少 HTTP 请求，比如合并资源、代码分割、合理缓存、把静态资源放 CDN，避免每次都从源站拉；第二，减小传输体积，比如 gzip/Brotli 压缩、图片优化、按需加载；第三，优化加载时机，比如懒加载、预加载、预请求，只让当前必须的资源先走；第四，优化链路本身，比如用 HTTP/2 或 HTTP/3 提升多路复用效率，减少不必要的重定向，降低 DNS、TCP、TLS 这些前置握手带来的等待时间。
- 1分钟展开：
  - 先减少请求数量：请求越多，排队、握手和往返等待就越多。前端常见手段是资源缓存、代码分割、接口聚合、减少重复请求、把公共静态资源放 CDN，让用户优先命中就近节点。
  - 缓存和资源存储很关键：强缓存、协商缓存、Service Worker、本地存储型缓存都能减少重复下载。很多网络延迟问题，本质上不是“网慢”，而是资源每次都重新拉。
  - 减小资源体积：开启 gzip 或 Brotli，压缩 JS/CSS/HTML；图片用 WebP/AVIF、雪碧图或响应式图片；无用代码和大依赖尽量裁掉。体积越小，传输时间越短，尤其弱网下收益更明显。
  - 异步加载要讲清场景：懒加载适合图片、长列表、非首屏模块；预加载适合“马上就要用”的关键资源；预请求适合“下一步大概率会用”的资源。核心是把资源按优先级分层，而不是一进页面全加载。
  - 协议优化能减少链路损耗：HTTP/2 的多路复用、头部压缩、连接复用能减少并发请求阻塞；HTTP/3 基于 QUIC，弱网和丢包场景下通常更稳。虽然这些更多是服务端和基础设施能力，但前端要知道为什么它们能降低等待。
  - 减少重定向和无效链路：一次重定向往往意味着多一次 DNS、TCP、TLS 或请求往返，首屏会被明显拖慢。所以线上要尽量避免 `http -> https -> www` 这种多跳链路，接口域名和静态资源域名也要收敛。
  - 回答思路：最稳的顺序是“减少请求 -> 缓存/CDN -> 压缩与图片优化 -> 异步加载 -> HTTP/2/3 与减少重定向”，这样既有前端工程手段，也有网络链路意识。

- 高频关键词：网络延迟、HTTP 请求、资源缓存、协商缓存、强缓存、Service Worker、资源存储、代码分割、CDN、gzip、Brotli、图片优化、懒加载、预加载、预请求、HTTP/2、多路复用、HTTP/3、QUIC、减少重定向、关键渲染路径
- 易追问：缓存为什么能降低延迟？代码分割为什么既能优化也可能带来额外请求？懒加载、预加载、预请求怎么区分？HTTP/2 为什么能提升并发加载效率？HTTP/3 相比 HTTP/2 解决了什么问题？为什么重定向会拖慢首屏？CDN 除了加速还有什么价值？
- 回答雷区：不要把“解决网络延迟”只答成上 CDN，它只是其中一环；不要只说压缩资源，不提请求数量和链路等待；不要把懒加载、预加载、预请求混为一谈，它们的时机和目的不同；也不要忽略重定向、DNS、TCP、TLS 这些前置耗时，很多慢不在业务代码本身，而在请求真正发出去之前。

## 98. 在异步加载中怎么确保用户界面的一致性？

- 一句话结论：异步加载里的 UI 一致性，核心是让用户始终看到“当前真实状态”，也就是有明确的加载反馈、旧请求不会覆盖新结果、失败后界面可恢复，避免出现数据和交互状态错位的脏 UI。
- 30秒回答：我一般从四个点保证一致性。第一，所有异步操作都要有明确的交互状态，比如 loading、skeleton、禁用按钮或提交中提示，避免用户不知道系统在做什么；第二，要防止竞态条件，比如搜索切换、分页切换、重复点击时，保证只有最后一次请求结果生效，常见做法是请求打标记、比较 requestId，或者直接取消旧请求；第三，要有统一的错误处理和回滚策略，失败时恢复按钮、保留旧数据或给出兜底提示；第四，状态更新要收口，避免局部成功、局部失败后页面显示出互相矛盾的信息。
- 1分钟展开：
  - 先把交互状态做完整：最基础的是 loading、empty、success、error 四类状态要明确。比如列表页加载时显示骨架屏或 loading，占位稳定；提交按钮进入 pending 时禁用重复点击，防止用户连续触发多次请求。
  - 竞态条件是高频问题：像搜索框连续输入、筛选条件快速切换、分页快速点击时，前一个请求可能比后一个更晚返回，导致旧数据覆盖新状态。常见处理是“只认最后一次请求”，比如用请求序号、时间戳、requestId 或 AbortController 取消旧请求。
  - 错误处理不能只弹一个 toast：接口失败后，除了提示错误，还要把 UI 状态恢复正确，比如解除 loading、恢复按钮可点击、必要时保留上一份稳定数据，而不是让页面停在半加载半成功的状态。
  - 避免脏 UI 和状态错位：比如筛选条件已经切到 B，但列表还是 A 的结果；或者保存失败了，界面却已经展示成保存成功。这类问题通常要通过状态单一来源、更新时机统一、乐观更新可回滚来解决。

- 是否保留旧数据要看场景：列表刷新时，很多场景会保留旧数据并显示顶部 loading，避免白屏闪烁；但如果是用户切换了实体对象，比如详情页从 A 切到 B，就要及时清理旧内容或加过渡态，避免 A 的数据短暂冒充 B。
- 工程上要统一收口：请求层或状态管理层最好统一封装 loading、error、cancel、retry 逻辑，组件只消费结果，避免每个页面各自写一套，最后行为不一致。
- 回答思路：最稳的顺序是“加载态展示 -> 防竞态 -> 错误恢复 -> 避免脏 UI -> 统一状态管理”，这样面试官能听到你既懂体验，也懂异步状态控制。
- 高频关键词：异步加载、UI 一致性、loading、skeleton、pending、竞态条件、requestId、最后一次请求生效、AbortController、取消请求、错误处理、回滚、重试、脏 UI、状态错位、单一数据源、乐观更新、状态管理
- 易追问：什么是竞态条件？为什么前一个请求会覆盖后一个请求？AbortController 和 requestId 方案怎么选？什么时候该保留旧数据，什么时候该清空？乐观更新失败后怎么回滚？为什么只弹错误提示还不够？列表页和表单页在一致性处理上有什么区别？
- 回答雷区：不要把 UI 一致性只理解成“加一个 loading”；不要忽略旧请求晚返回覆盖新结果的问题；不要只做错误提示，不恢复交互状态；也不要让多个组件各自维护一份状态副本，最后出现显示结果、筛选条件和真实请求参数对不上的情况。

## 99. 引入虚拟 DOM 的好处是什么？为什么不直接操作 DOM？

- 一句话结论：引入虚拟 DOM 的核心价值，不是简单追求“比直接操作 DOM 更快”，而是通过状态驱动、增量更新和渲染抽象，减少不必要的真实 DOM 操作，提升开发效率，并为跨平台渲染提供统一基础。
- 30秒回答：虚拟 DOM 可以理解成真实 DOM 的一层 JavaScript 抽象。组件状态变化时，框架先生成新的虚拟 DOM，再和旧的做 diff，最后只把必要的变更同步到真实 DOM。这样做的好处有三个：第一，减少频繁、零散的真实 DOM 操作，尤其能配合批量更新降低重排重绘成本；第二，开发模式从“手动改 DOM”变成“声明式写 UI”，代码更好维护；第三，UI 描述和平台渲染解耦，同一套组件思想可以适配 Web、Native、SSR 等不同渲染目标。面试里还要补一句：虚拟 DOM 不是所有场景都比直接操作 DOM 快，小而确定的单点更新，手写 DOM 甚至可能更快，但框架更看重整体可维护性和稳定收益。
- 1分钟展开：
  - 为什么不直接操作 DOM：真实 DOM 操作通常伴随浏览器布局、重绘、回流等成本，手动操作一多，更新逻辑容易分散在各处，代码很快变成命令式拼装，维护成本高，也更容易出现状态和界面不一致。
  - 虚拟 DOM 的性能价值是什么：它不是“JS diff 一定比 DOM 快”，而是先在内存里比较差异，再把多个状态变化收敛成更少的真实 DOM patch，减少无意义的直接操作。性能收益更多来自“批量、增量、可预测更新”，而不是虚拟 DOM 这四个字本身。

- **声明式开发提升效率**：开发者只需要描述“当前状态下页面应该长什么样”，而不用自己手写增删改查 DOM。这样业务关注点从操作节点，转成管理状态和数据流，组件化也更自然。
- **更容易做统一调度**：有了虚拟 DOM 这层抽象，框架可以批量更新、异步调度、优先级处理，避免每次状态变化都立刻、零散地打到真实 DOM 上。React 这类框架的调度能力就是建立在这种抽象之上的。
- **跨平台兼容性更强**：虚拟 DOM 或更广义的虚拟节点树，本质上是平台无关的 UI 描述。底层 **renderer** 可以把同一套组件思想渲染成浏览器 DOM、原生控件、服务端字符串，甚至其他宿主环境。
- **边界要会讲**：如果只是极少量、非常确定的 DOM 改动，直接操作 DOM 可能更轻；而像 Vue 3 这类框架还会结合编译优化、静态提升、Patch Flags，不是单靠运行时 diff 硬算。所以不要把“虚拟 DOM = 一定最快”说死。
- **回答思路**：最稳的是“先讲抽象层作用，再讲性能优化，再讲开发效率，最后补跨平台和边界条件”。
- **高频关键词**：虚拟 DOM、Virtual DOM、真实 DOM、diff、patch、增量更新、批量更新、状态驱动、声明式 UI、组件化、重排、重绘、回流、调度、renderer、跨平台、SSR、Patch Flags
- **易追问**：虚拟 DOM 为什么不等于一定更快？diff 的核心思路是什么？直接操作 DOM 在什么场景可能更快？虚拟 DOM 和声明式开发是什么关系？跨平台为什么需要一层虚拟节点抽象？Vue 3 为什么说不完全依赖传统虚拟 DOM 优化？React Fiber 和虚拟 DOM 是什么关系？
- **回答雷区**：不要把虚拟 DOM 说成“比真实 DOM 多了一层所以一定更慢”或“所以一定更快”，两种绝对化说法都不对；不要把性能收益简单归因成 diff 本身，更关键的是减少不必要的真实 DOM 操作和统一调度；不要忽略它在开发模式和跨平台上的价值；也不要将虚拟 DOM 和真实 DOM 混成一回事，前者是内存中的 UI 描述，后者才是浏览器真正渲染的节点。

## 100. 怎么确定前端项目方案的可行性？

- **一句话结论**：判断一个前端方案是否可行，不能只看“技术上能不能写出来”，而是要同时评估业务必要性、技术实现路径、时间投入、性能兼容、维护成本、风险和测试验证，最后选出在当前约束下最稳的方案。
- **30秒回答**：我判断方案可行性，一般会按七步走。第一，先看业务是否真的需要，比如实时监控大屏到底是秒级刷新、分钟级刷新，还是人工刷新就够；第二，看技术能不能实现，比如数据推送到底选 WebSocket、SSE 还是轮询；第三，估算周期和人力，判断是不是能在当前排期内落地；第四，评估性能和兼容性，比如大屏数据量大、推送中断、弱网或浏览器限制时有没有降级方案；第五，看架构是否可维护，后续能不能复用；第六，提前识别风险点和极端场景；第七，一定要做验证测试，别把“理论可行”当成“实际可上线”。
- **1分钟展开**：

- **先看业务可行性：**不是所有需求都值得上复杂方案。以实时监控大屏为例，先要问清楚“实时”到底是什么级别，是 1 秒级、5 秒级还是分钟级；如果业务只需要分钟级，可能根本没必要上持续推送链路。
- **再看技术可行性：**明确前后端协作方式、数据来源和交互模式。比如大屏是单向推送，SSE 可能更简单；如果需要双向通信、心跳保活、客户端主动控制，WebSocket 更合适；如果环境受限，还要考虑轮询或长轮询作为兜底。
- **时间和投入要算清楚：**不仅看开发时间，还要算联调、压测、验收、异常处理和部署成本。有些方案技术上很高级，但当前人力和周期压根撑不住，那在项目管理意义上也不算真正可行。
- **性能和兼容性必须提前评估：**实时大屏很容易遇到高频推送、图表频繁重绘、内存增长、浏览器长时间运行卡顿等问题。还要想清楚断线重连、消息积压、数据丢失、浏览器兼容以及弱网情况下怎么办，是否有降级到轮询、降低刷新频率或只更新局部模块的方案。
- **可维护性和可执行性：**方案不能只满足这次开发，还要看模块拆分是否清晰、数据流是否统一、图表组件和连接层是否可复用、后续别人接手是否容易理解。如果一个方案只能靠个人经验硬撑，长期维护风险很高。
- **风险分析要具体：**比如后端推送能力不稳定、接口协议频繁变化、消息峰值过高、某些图表库渲染性能不足、展示设备分辨率不一致、长时间运行后内存泄漏，这些都要在设计阶段先列出来。
- **最后一定做测试验证：**至少要做功能测试、异常测试、断网重连测试、长时间运行稳定性测试、性能压测和兼容性测试。很多方案 PPT 上看着没问题，真正压一压、跑一晚就暴露问题了。
- **回答思路：**最稳的是“业务必要性 -> 技术路径 -> 时间投入 -> 性能兼容 -> 可维护性 -> 风险 -> 测试验证”，这样能体现你不是只会写代码，而是在做工程决策。
- **高频关键词：**方案可行性、业务可行性、技术可行性、时间评估、人力投入、实时监控大屏、秒级刷新、WebSocket、SSE、轮询、性能评估、兼容性、降级方案、断线重连、可维护性、复用性、风险分析、压测、稳定性测试
- **易追问：**什么时候该选 WebSocket，什么时候选 SSE？如果实时性要求没那么高，为什么不直接轮询？如何判断大屏项目的性能瓶颈在前端还是后端？如果推送中断怎么办？降级方案怎么设计？如何评估一个方案“能做”和“值得做”的区别？测试阶段最应该先压哪类问题？
- **回答雷区：**不要把“可行性”只理解成技术实现；不要一上来就选最复杂、最先进的技术，而不看业务是否真的需要；不要忽略排期、人力和维护成本；也不要没有验证就拍板，方案如果没经过异常场景和压力测试，很多时候只是“看起来可行”。

## 101.Canvas、SVG、WebGL 的区别是什么？

- 一句话结论：三者本质区别在渲染模型和适用场景上，**Canvas** 更适合高频二维绘制，**SVG** 更适合可交互、分辨率独立的矢量图形，**WebGL** 则更适合基于 GPU 的高性能 2D/3D 渲染和复杂图形场景。
- 30秒回答：**Canvas** 是一个画布，JS 每次直接往像素层绘制，属于即时绘制模式，适合游戏、粒子效果、频繁刷新的图表，但交互通常要自己通过坐标命中判断。**SVG** 是基于 XML 的矢量图，图形本身就是 DOM 节点，天然适合事件交互、CSS 动画和缩放不失真，适合图标、流程图、少量可交互图形。**WebGL** 是浏览器里的图形 API，底层调用 GPU，适合 3D、海量点线面和高性能可视化。简单记就是：二维高频绘制看 **Canvas**，矢量交互看 **SVG**，复杂高性能图形看 **WebGL**。
- 1分钟展开：
  - **Canvas** 是什么：**Canvas** 是 HTML 提供的画布元素，常见是 2D 上下文。你调用 API 画出来的是像素结果，不会为每个图形保留独立 DOM 节点，所以重绘通常靠自己控制。
  - **Canvas** 的优缺点：优点是绘制频繁、对象多、动画多时效率比较高，适合游戏、实时图表、粒子动画；缺点是图形不是 DOM，事件交互、拖拽命中、局部更新和无障碍支持都要自己额外处理。
  - **SVG** 是什么：**SVG** 是二维矢量图形标准，每个图形元素都可以是一个可操作节点，所以天然支持事件、样式和动画，而且缩放不会失真。
  - **SVG** 的优缺点：它很适合图标、地图、流程图、编辑器这类“元素数量相对可控但交互要求高”的场景；但如果节点非常多、频繁更新，DOM 数量会膨胀，性能压力往往比 **Canvas** 更大。
  - **WebGL** 是什么：**WebGL** 是 JavaScript 图形 API，本质上是让浏览器使用 GPU 做渲染。它不是专门只做 3D，也能做高性能 2D，但更适合大规模图形计算、3D 场景、GIS、工业可视化和复杂游戏。
  - **WebGL** 的特点：性能上限高，但开发复杂度也高，通常要处理着色器、缓冲区、矩阵、坐标变换等底层概念，所以很多项目会配合 **Three.js**、**Babylon.js** 这类封装库使用。
  - 怎么选型最稳：图标和可交互矢量图优先 **SVG**；大量点位、高频刷新、粒子效果和普通 2D 游戏优先 **Canvas**；3D 场景、海量数据可视化、地图引擎或对 GPU 加速要求很高的场景优先 **WebGL**。
- 高频关键词：**Canvas**、**SVG**、**WebGL**、即时模式、保留模式、像素绘制、矢量图、DOM 节点、分辨率独立、事件交互、坐标命中、GPU、2D、3D、实时图表、游戏、数据可视化、**Three.js**
- 易追问：**Canvas** 为什么交互通常要自己做命中检测？为什么 **SVG** 缩放不失真？什么时候 **Canvas** 比 **SVG** 更合适？**WebGL** 为什么性能更高？**WebGL** 能不能做 2D？**Three.js** 和 **WebGL** 是什么关系？如果做一个实时监控大屏图表，三者分别适合什么场景？

- 回答雷区：不要把 Canvas 说成矢量图，它本质是像素绘制；不要把 SVG 说成“性能一定最好”，节点多且频繁更新时它反而容易吃力；不要把 WebGL 只理解成“做 3D 的”，它也能做高性能 2D；也不要忽略三者的交互模型差异，Canvas 没有天然的图形节点概念，SVG 则有。

## 102. 设置验证码的作用是什么？

- 一句话结论：验证码的核心作用是做人机校验，拦截脚本、爬虫和恶意程序的批量操作，降低暴力破解、恶意注册、短信轰炸、刷活动和垃圾提交等风险。
- 30秒回答：验证码本质上不是为了“让用户多做一步”，而是为了区分真人和自动化程序，给高风险操作加一道门槛。它常用在登录、注册、找回密码、短信发送、评论发布、活动参与这些场景，主要作用有四个：第一，防止机器人批量攻击，比如暴力登录、刷票、刷接口；第二，提高系统安全性，减少撞库和批量试错；第三，降低服务器被恶意请求消耗的压力；第四，防止表单灌水、垃圾评论和广告提交。常见验证码有图形验证码、滑动验证码、点选验证码、数学题验证码和音频验证码。
- 1分钟展开：
  - 先说本质：验证码是风控体系里最常见的一种人机验证手段，目的不是单独解决所有安全问题，而是给自动化攻击增加成本，让“批量化”这件事变难。
  - 防止恶意机器人攻击：高频场景包括注册账号、登录暴力破解、刷票、刷评论、刷秒杀、刷活动名额等。验证码通过增加真人交互步骤，降低脚本重复请求的成功率。
  - 提高系统安全性：在登录、注册、找回密码、短信验证码发送等敏感流程中，验证码能限制黑客批量尝试用户名密码组合、撞库、短信轰炸等行为，减少用户和平台资产受损的概率。
  - 降低服务器压力：恶意爬虫和脚本如果持续刷请求，不只是安全问题，也会浪费计算资源和带宽。验证码可以提前过滤一部分非正常流量，减少无效请求对系统的冲击。
  - 防止恶意提交表单：留言板、论坛、评论区、反馈表单、注册页这些地方，如果没有验证码，很容易被脚本灌水、发垃圾广告或批量提交垃圾数据。
  - 常见验证码类型：图形验证码适合基础识别；滑动验证码和点选验证码更偏行为验证；数学题验证码实现简单；音频验证码适合视障用户，是可访问性补充。不同业务会根据安全要求和用户体验选不同方案。
  - 边界要会讲：验证码不是万能的，识别技术和打码平台也可能绕过它，所以实际项目里通常会和限流、设备指纹、IP 风控、登录失败次数限制、行为分析一起使用，而不是单靠验证码顶所有风险。
- 高频关键词：验证码、人机校验、风控、恶意机器人、爬虫、暴力破解、恶意注册、撞库、短信轰炸、刷票、刷活动、垃圾提交、图形验证码、滑动验证码、点选验证码、数学题验证码、音频验证码、限流、行为验证

- 易追问：验证码为什么不能单独解决安全问题？什么时候该加验证码，什么时候不该加？滑动验证码和图形验证码怎么选？短信发送为什么也要做验证码保护？验证码会不会影响转化率？除了验证码还有哪些常见风控手段？
- 回答雷区：不要把验证码说成“绝对安全”，它只是风控的一环；不要忽略用户体验，如果所有操作都强制验证码，转化率可能会受影响；不要只说防登录爆破，注册、评论、表单、活动、短信接口同样是高风险点；也不要把验证码类型只记成一种，面试里至少要能说出图形、滑动、点选、音频这些常见形式。

## 内容沉淀区

后续你每发一个问题或核心要点，我都按下面格式直接追加到这里。

### 1. 弱网检测如何做？

- 一句话结论：弱网检测不能只靠 `navigator.onLine`，更稳的做法是把离线事件、主动探测、耗时统计和业务降级结合起来，区分“断网”“网络抖动”和“真正的弱网”。
- 30秒回答：前端做弱网检测一般分三层。第一层用 `navigator.onLine` 配合 `online`、`offline` 事件，快速感知网络有没有断开，但它只能判断是否离线，不能准确代表网速。第二层做主动探测，比如定时 `fetch` 一个很小的数据包或静态资源，用 `performance.now()` 计算请求耗时，再结合超时和失败率判断当前网络质量。第三层是根据检测结果做降级，比如减少轮询频率、延长重试间隔、降低资源质量或提示用户当前网络较差。调试时可以直接用 DevTools 的 Network 面板模拟 `Slow 3G` 或 `Offline` 验证方案。
- 1分钟展开：
  - 被动检测适合发现断网：`navigator.onLine` 和 `online`、`offline` 事件适合第一时间感知连接状态变化，但它不等于“网络好”。设备连着局域网、Wi-Fi 没断，不代表外网一定可用，所以它更适合做断网提示，不适合单独做弱网判断。
  - 主动探测更接近真实网络质量：可以周期性请求一个体积很小、禁缓存或带时间戳的资源，记录请求耗时、超时次数和失败率。比如把 RTT、连续失败次数作为指标，再按阈值分成正常、一般、弱网、断网几档。
  - 不要只看单次结果：一次请求变慢，可能是服务端抖动，也可能是 DNS、链路波动，不一定就是弱网。更稳的是连续多次采样，看平均耗时、超时率、失败率，降低误判概率。
  - 检测完要配合降级策略：弱网下不能只是弹个 `toast`。更实际的是降低图片清晰度、暂停大资源预加载、减少并发请求、拉长轮询间隔、增加超时重试、上传走断点续传，让页面还能勉强可用。

- 兼容性补充要会说：部分浏览器支持 `navigator.connection`，能拿到 `effectiveType`、`rtt`、`downlink` 这些信息，但兼容性一般，所以面试里要强调它只能做辅助信号，不能作为唯一判断依据。
- 验证方式要落地：开发和联调阶段可以用 Chrome DevTools 的 Network 面板模拟 `Fast 3G`、`Slow 3G`、`Offline`，确认超时、重试、提示和降级逻辑是不是按预期生效。
- 高频关键词：弱网检测、断网检测、`navigator.onLine`、`online`、`offline`、主动探测、`fetch`、小数据包、`performance.now()`、RTT、超时、失败率、降级策略、重试、轮询、`navigator.connection`、`effectiveType`、DevTools、`Slow 3G`、`Offline`
- 易追问：`navigator.onLine` 为什么不能准确判断弱网？主动探测为什么要避免缓存？弱网和断网怎么区分？阈值应该怎么定？`navigator.connection` 在生产里能不能直接依赖？弱网场景下常见的前端降级策略有哪些？如果是 WebSocket 长连接，怎么判断连接质量？
- 回答雷区：不要把 `navigator.onLine` 说成完整的弱网检测方案，它更接近断网感知；不要只测一次请求就直接判定弱网，容易把服务端抖动误判成网络问题；不要只做提示不做降级和重试；也不要忽略缓存影响，探测资源如果被缓存命中，耗时数据就不可信。

## 2. JS 数组中哪些方法会修改原数组？

- 一句话结论：数组方法里，像 `push`、`pop`、`shift`、`unshift`、`splice`、`sort`、`reverse` 这类会直接修改原数组；而 `map`、`filter`、`slice`、`concat` 这类通常返回新数组，不会改原数组本身。
- 30秒回答：判断数组方法会不会改自身，最简单的思路是看它是不是“在原地操作”。常见会修改原数组的方法有 `push`、`pop`、`shift`、`unshift`、`splice`、`sort`、`reverse`，补充还有 `fill`、`copyWithin`。不会修改原数组的方法有 `map`、`filter`、`reduce`、`slice`、`concat`、`indexOf`、`includes`、`join` 等。`forEach` 本身不会返回新数组，也不会自动修改原数组，但你可以在回调里手动改元素，所以面试里最好说清楚“方法本身不改，代码写法可能改”。
- 1分钟展开：
  - 会修改原数组的方法：`push` 尾部新增，`pop` 尾部删除，`shift` 头部删除，`unshift` 头部新增，`splice` 可以插入、删除、替换，`sort` 原地排序，`reverse` 原地反转，这几个是最常考的。
  - 容易漏掉的两个：`fill` 会用指定值填充原数组的某一段，`copyWithin` 会把数组内部某段内容复制到另一位置，它们也都是直接改原数组。

- 不会修改原数组的方法：`map`、`filter`、`slice`、`concat` 会返回新数组；`reduce` 返回累计结果；`indexOf`、`includes` 返回查找结果；`join` 返回字符串。这些都不会直接改原数组结构。
- `forEach` 要单独说明：`forEach` 不会像 `map` 一样返回新数组，方法本身也不负责修改原数组，但如果你在回调里写 `arr[index] = ...`，那是你手动改了原数组，不是 `forEach` 自带的返回行为。
- `sort` 和 `reverse` 是高频陷阱：很多人以为它们会返回新数组，其实它们返回的还是同一个数组引用，只是已经被原地改掉了，所以在 `React`、`Vue` 这类依赖状态不可变更的场景里要特别小心。
- 面试加分点：如果不想改原数组，可以用 `slice()`、展开运算符 `[...]` 先拷贝一份再 `sort`，或者在新规范里用 `toSorted()`、`toReversed()`、`toSpliced()` 这些不改原数组的新方法。
- 高频关键词：原数组修改、可变方法、不可变方法、`push`、`pop`、`shift`、`unshift`、`splice`、`sort`、`reverse`、`fill`、`copyWithin`、`map`、`filter`、`reduce`、`slice`、`concat`、`forEach`、不可变更新、`toSorted()`
- 易追问：`splice` 和 `slice` 有什么区别？为什么 `sort` 会引发状态更新问题？`forEach` 到底算不算修改原数组？`map` 和 `forEach` 的区别是什么？哪些新数组方法是了解决不可变更新问题引入的？如果数组元素是对象，`slice` 之后算不算深拷贝？
- 回答雷区：不要把 `splice` 写成 `split` 或 `spite`，前者是数组方法，后者是字符串方法；不要把 `sort`、`reverse` 误说成返回新数组；不要把 `forEach` 简单归类成“一定修改”或“一定不修改”，要说明是“方法本身不改，但回调里可以手动改”；也不要忽略 `fill`、`copyWithin` 这两个也会原地修改数组的方法。

### 3. 前端如何快速获取页面 URL 的 query 参数？

- 一句话结论：前端获取 URL query 参数，原生场景最推荐用 `URLSearchParams`，简单直接、可读性好；如果在 `Vue` 这类路由框架里，优先用 `route.query`，只有兼容老代码时才考虑手动解析 `location.search`。
- 30秒回答：最常用的原生方式是 `new URLSearchParams(window.location.search).get('paramName')`，它能直接拿到查询参数，写法最简洁。传统方式也可以先拿 `window.location.search`，去掉前面的 `?` 后再手动拆分，但可维护性差一些。如果项目用了 `Vue Router`，一般直接用 `route.query.xxx` 或 `route.query['xxx']`，因为路由已经帮你把参数解析好了。面试里最好补一句：`URLSearchParams` 适合绝大多数现代浏览器场景，框架项目优先走路由对象，不要自己重复解析。

- 1分钟展开：
  - 原生首选 `URLSearchParams`：比如 `new URLSearchParams(window.location.search).get('id')`，优点是语义清晰、代码短，还支持 `getAll`、`has`、`entries` 等方法，适合快速读取 query 参数。
  - 手动解析是基础兜底思路：`window.location.search` 能拿到类似 `?id=1&name=tom` 的字符串，再通过 `substring(1)`、`split('&')`、`split('=')` 去拆。这个方式能体现你理解 query 的本质，但代码更啰嗦，也要自己处理解码、空值和边界。
  - 框架项目优先用路由对象：在 Vue Router 里一般直接用 `route.query`，例如 `route.query.id`。这样写的好处是和当前路由状态保持一致，也不用手动依赖 `window.location`。
  - 多值和编码问题要会讲：query 里可能出现同名参数、多值参数和中文编码，所以要注意 `decodeURIComponent` 这类处理。`URLSearchParams` 在这方面比手写解析稳很多。
  - 参数类型要注意：query 参数本质上大多是字符串，比如 `?page=1` 读出来通常是 `'1'`，如果业务需要数字或布尔值，要自己做类型转换，不能默认它就是 `number`。
  - 面试回答顺序建议：先讲原生推荐方案 `URLSearchParams`，再补手动解析体现基础，最后补框架写法 `route.query`，这样答案完整又贴近实际开发。
- 高频关键词：URL query、查询参数、`window.location.search`、`URLSearchParams`、`get()`、`getAll()`、`has()`、手动解析、`substring(1)`、`split('&')`、`split('=')`、`route.query`、Vue Router、参数解码、类型转换
- 易追问：`URLSearchParams` 和手动解析相比优势是什么？如果 query 里有中文或特殊字符怎么办？多个同名参数怎么取？为什么 `route.query` 更适合 Vue 项目？query 参数为什么默认是字符串？hash 参数和 query 参数有什么区别？
- 回答雷区：不要把 `URLSearchParams` 写成 `URLsearchParams`，`S` 必须大写；不要直接认为 query 参数拿出来就是数字或布尔值；不要在框架项目里明明有 `route.query` 还去重复手动拆 `location.search`；也不要忽略参数编码和多值参数的处理，真实项目里这两个问题很常见。

## 4. 前端如何处理一个页面有多个主题色可供选择的场景？

- 一句话结论：多主题色场景最稳的做法，是把颜色抽成 CSS 变量放在 `:root`，再通过 `data-theme` 之类的属性切换不同主题变量值，组件样式统一用 `var()` 取值，并配合 `localStorage` 持久化用户选择。
- 30秒回答：前端做多主题，一般不会在组件里写死颜色，而是先在 `:root` 定义一套通用设计变量，比如主色、背景色、文字色、边框色。然后给 `html` 或 `body` 挂 `data-theme="dark"`、`data-theme="blue"` 这类属性，在不同主题选择器下覆盖变量值，页面里的样式统一通过 `var(--color-primary)` 这类方式引用。这样切主题时只改根节点属性，不用重刷整套样式。浏览器原生控件还可以配合 `color-scheme` 做深浅模式适配，最后再用 `localStorage` 记住用户上次选的主题。
- 1分钟展开：
  - 先抽象设计令牌：比如在 `:root` 里定义 `--color-primary`、`--bg-color`、`--text-color`、`--border-color`。这样业务组件只依赖语义化变量，不直接依赖具体颜色值。
  - 用属性选择器切换主题：常见做法是给根节点加 `data-theme="light"`、`data-theme="dark"`、`data-theme="green"`，然后写 `[data-theme='dark'] { --bg-color: #111; ... }` 这种覆盖规则。相比写成 `[data-dark]`、`[data-light]`，`data-theme` 更适合扩展到多个主题。
  - 组件统一通过 `var()` 消费变量：例如按钮背景、文字颜色、阴影、边框都从变量取值。这样主题切换时只改变量，不需要逐个组件改 `class`，也能减少样式耦合。
  - 浏览器默认控件可配合 `color-scheme`：比如表单控件、滚动条、输入框等浏览器原生样式，在深色主题下可以通过 `color-scheme: dark` 或 `light dark` 让浏览器也跟着调整，减少原生控件和业务样式割裂。
  - 用户选择要做持久化：切换主题后把主题名存进 `localStorage`，页面初始化时优先读取并恢复，这样刷新页面不会丢。若用户没选过，也可以结合 `prefers-color-scheme` 先跟随系统主题。
  - 多主题场景的工程注意点：不要只改主色而忽略背景、边框、`hover`、禁用态、阴影和图表色板；图片、图标、插画也要考虑主题适配；如果是 SSR 项目，还要尽量在首屏渲染前就确定主题，避免闪屏。
  - 回答顺序建议：最稳的是“CSS 变量抽象 -> 属性切换主题 -> `var()` 使用 -> `color-scheme` 补充 -> `localStorage` 持久化 -> 系统主题和闪屏边界”。

- 高频关键词：多主题、主题切换、CSS 变量、`:root`、设计令牌、`var()`、`data-theme`、属性选择器、深色模式、浅色模式、`color-scheme`、`localStorage`、`prefers-color-scheme`、主题持久化、主题闪屏、设计系统
- 易追问：为什么推荐 CSS 变量而不是切换整套 class？`data-theme` 和 `[data-dark]` 这种写法怎么选？`color-scheme` 具体解决什么问题？如何避免刷新页面时主题闪屏？系统主题和用户手动选择冲突时怎么处理？如果有图表、图片和第三方组件，主题适配怎么做？
- 回答雷区：不要把多主题理解成只切一个主色，背景、文字、边框、状态色和图表色板也要统一设计；不要在组件里到处写死颜色值，否则后面很难维护；不要把 `localStorage` 写成 `localstory`；也不要忽略浏览器原生控件和首屏闪屏问题，实际项目里这两个问题很常见。

## 5. 如何计算页面白屏时间？

- 一句话结论：白屏时间本质上是“从开始加载页面，到用户第一次看到内容”的时间，最稳的口径通常是 `navigationStart` 到首个可见内容出现的时间；现代浏览器里更推荐直接看 FCP，或者结合 `PerformanceObserver` 和业务埋点来统计。
- 30秒回答：白屏时间关注的是“用户什么时候不再看到空白页”。起点一般可以理解成浏览器开始导航的时间，老写法常用 `performance.timing.navigationStart`。终点如果只是粗略估算，有人会看 `responseStart` 或 `DOMContentLoaded`，但这两个都不等于真正的白屏结束，前者只是收到首字节，后者是 DOM 解析完成。更标准的方案是直接看 FCP，也就是首次内容绘制时间；如果业务想统计“首屏某个核心模块真正可见”的时间，可以用 `PerformanceObserver` 监听 `paint`，或者用 `performance.mark` / `performance.measure` 给首屏关键节点做埋点。
- 1分钟展开：
  - 先讲白屏时间的定义：白屏时间不是页面完全加载完成时间，而是用户第一次感知到页面内容的时间。所以核心公式更接近“首个可见内容出现时间 - 开始导航时间”。
  - 开始时间怎么取：老项目里通常用 `performance.timing.navigationStart` 作为起点，你给的这个口径是常见说法。现代接口里也可以理解成从本次 navigation 开始计算，很多性能指标本身就是相对导航开始的偏移量。
  - `responseStart` 是什么：`responseStart` 表示浏览器收到服务端首字节响应的的时间，它更适合衡量后端响应和网络传输首包到达，不代表页面已经可见，所以只能作为粗略参考，不能直接当白屏结束时间。
  - `DOMContentLoaded` 能不能用：`DOMContentLoaded` 表示 DOM 解析完成，通常比 `load` 更早，但它依然不等于“用户看到内容”。有的页面 DOM 已经解析完，但首屏内容还没真正渲染出来；也有的页面骨架屏早就出现了，`DOMContentLoaded` 反而偏晚。

- 为什么直接用 **FCP** 更标准：FCP 是首次内容绘制时间，代表浏览器第一次把文本、图片、非空白 canvas 或 SVG 这类内容真正画到屏幕上，所以比 **responseStart**、**DOMContentLoaded** 更接近白屏结束的用户视角。
- 实际项目怎么监控：可以用 `new PerformanceObserver(({list}) => {})` 监听 **paint** 条目，取 **first-contentful-paint**；如果你要统计“首屏主模块”“首个业务卡片”“骨架屏结束”这类更贴业务的时刻，就在对应时机用 `performance.mark()` 打点，再用 `performance.measure()` 算区间。
- 面试里最稳的回答顺序：先说定义是“从 URL 加载到用户看到第一屏内容”，再说起点常见取 **navigationStart**，然后补充 **responseStart**、**DOMContentLoaded** 只是粗略参考，真正更推荐 FCP 或业务埋点，这样答案就比较完整。
- 高频关键词：白屏时间、首屏渲染、用户可感知内容、**navigationStart**、**responseStart**、**DOMContentLoadedEventStart**、FCP、**first-contentful-paint**、**PerformanceObserver**、**paint**、`performance.mark()`、`performance.measure()`、性能埋点、首字节时间、业务打点
- 易追问：**responseStart** 为什么不等于白屏结束？**DOMContentLoaded** 和 FCP 有什么区别？FCP 为什么更接近白屏时间？SPA 路由切换怎么统计白屏？如果页面先展示骨架屏，白屏时间该怎么算？业务埋点和浏览器标准性能指标应该怎么配合？
- 回答雷区：不要把 **responseStart**、**DOMContentLoaded** 或 **loadedEventStart** 直接等同于白屏时间，它们都只是不同阶段的时间点；不要只背 API 名字，不解释“用户什么时候第一次看到内容”这个本质；不要忽略 FCP 已经是更主流的标准指标；也不要吧浏览器通用指标和业务埋点混为一谈，前者偏标准化，后者偏业务感知。

## 6. 页面白屏是如何排查的？

- 一句话结论：排查页面白屏，最有效的思路不是盲目看报错，而是先区分“持续白屏”还是“短暂白屏”，再分别从资源加载、JS 执行、页面挂载、路由逻辑和性能耗时几个方向定位。
- 30秒回答：我排查白屏一般先分两类。第一类是持续白屏，也就是页面一直出不来，这时候优先看 Network 里有没有核心资源 404 或加载失败，再看 Console 有没有 JS 报错把渲染链路打断，再去 Elements 看根节点是不是根本没挂载，最后排查路由守卫、鉴权逻辑或 loading 状态是不是把页面一直卡住。第二类是短暂白屏，也就是页面最终能出来，但中间空白时间很长，这时候重点看首屏资源是不是太大、Main 线程有没有长任务、异步数据是不是拖慢了渲染，再结合 Performance 面板分析请求耗时、JS 执行时间和图片解码时间。
- 1分钟展开：
  - 先分清是持续白屏还是短暂白屏：持续白屏说明渲染流程可能根本没走通；短暂白屏说明最终能渲染，只是某个阶段耗时过长。这个分类很重要，因为前者偏功能故障，后者偏性能问题。

- **持续白屏先查资源加载：**打开 **Network** 看 **HTML**、**JS**、**CSS**、接口这些关键资源是否请求成功，重点关注 **404**、**500**、跨域失败、打包产物路径错误、静态资源 **CDN** 失效等问题。入口 **JS** 没加载到，页面通常就是纯白。
- **再查 JS 是否报错阻塞：**看 **Console** 有没有运行时报错，比如语法错误、空指针、模块加载失败、环境变量缺失、第三方 **SDK** 初始化异常。首屏初始化代码一旦抛错，后续挂载流程可能直接中断。
- **确认页面有没有真正挂载：**去 **Elements** 看根节点，比如 **#app**、**#root**，有没有内容渲染进去。如果只有空壳节点，说明框架初始化或渲染没完成；如果连根节点都不存在，就要反查模板、入口文件和挂载逻辑。
- **排查路由和状态逻辑：**很多白屏不是资源问题，而是路由守卫没有放行、重定向死循环、权限判断一直 **pending**、**loading** 状态没有结束，导致页面看起来像白屏但本质是逻辑卡死。
- **短暂白屏重点看首屏性能：**如果页面最终能出来，就去 **Performance** 面板看 **Main** 线程有没有长任务，**JS** 解析、执行、**hydrate** 是否过重；再看 **Network** 是否有慢接口、超大图片、首屏资源过多；必要时看图片解码、渲染和布局阶段是不是耗时过高。
- **异步请求和渲染节奏也要查：**首屏内容如果强依赖接口返回，接口慢就会直接拉长白屏时间。要确认有没有可以提前渲染骨架屏、并行请求、缓存兜底或服务端预取，否则用户会一直等数据。
- **回答顺序建议：**最稳的表达是“先分类 -> 持续白屏查资源、报错、挂载、路由 -> 短暂白屏查资源体积、长任务、异步接口、图片解码和 **Performance**”。这样既体现排障能力，也体现性能分析能力。
- **高频关键词：**白屏排查、持续白屏、短暂白屏、**Network**、**404**、入口资源加载失败、**Console** 报错、**JS** 执行异常、**Elements**、根节点挂载、**#app**、**#root**、路由守卫、重定向死循环、**loading** 卡住、**Performance**、**Main** 线程、长任务、异步请求、图片解码、首屏性能
- **易追问：**为什么要先区分持续白屏和短暂白屏？入口 **JS 404** 会导致什么现象？如何判断是挂载失败还是样式问题？路由守卫为什么会造成白屏？**Performance** 面板里最应该先看什么？如果首屏依赖异步接口，怎么降低白屏感知？骨架屏能不能解决所有白屏问题？
- **回答雷区：**不要一上来就只盯着 **Console**，看不到资源、挂载和路由问题；不要把持续白屏和短暂白屏混成一类，它们的定位路径不一样；不要只说“资源太大”，却不提 **Main** 线程长任务、异步接口和图片解码这些真实瓶颈；也不要忽略根节点是否真正挂载，很多时候页面白屏不是没请求到数据，而是压根没渲染起来。

## 7. 如何实现一个页面不可选中、不可复制？

- 一句话结论：页面“不可选中、不可复制”通常要分两层做，第一层用 CSS 的 `user-select: none` 禁止文本被选中，第二层用 JS 监听 `copy` 事件并 `preventDefault()` 阻止复制行为；但这只能提高复制成本，不能当成真正的安全防护。
- 30秒回答：如果只是让页面内容不方便被复制，最常见做法是先在样式层加 `user-select: none`，让用户没法直接框选文本；再在行为层监听 `copy` 事件，执行 `e.preventDefault()` 阻止复制。这个监听通常可以挂在 `document` 或根容器上，并且用捕获阶段提前拦截，这样即使子节点触发复制，也能被父层统一阻止。面试里最好补一句：这只是前端层面的限制，用户依然可能通过 DevTools、源码、接口返回或截图拿到内容，所以它更偏交互限制，不是安全方案。
- 1分钟展开：
  - **CSS 层做“不可选中”**：最直接的是 `user-select: none`，这样文本通常不能被鼠标选中，能先挡掉最常见的拖拽复制场景。实际项目里可以只给敏感区域加，而不是全站一刀切，避免影响正常交互。
  - **JS 层做“不可复制”**：监听 `copy` 事件，比如 `document.addEventListener('copy', (e) => e.preventDefault(), true)`。这里的关键是 `copy` 事件本身可取消，调用 `preventDefault()` 后浏览器默认复制行为会被拦截。
  - **为什么这里能体现事件捕获的使用场景**：把监听挂在父节点或 `document` 上，并开启捕获阶段，可以在事件到达具体子节点之前就统一拦截，避免每个子元素都单独绑一次复制事件。这就是事件捕获在“全局拦截某类行为”里的典型用法。
  - **常见补充限制**：有些项目还会顺手拦截 `cut`、`contextmenu`、`selectstart` 或快捷键组合，但这些都只是继续提高操作成本，体验代价也会更高，所以要慎用。
  - **工程上要讲边界**：如果页面上有输入框、代码块、表单内容或需要用户复制的邀请码，一刀切禁掉复制会直接伤害可用性。更合理的是只限制特定内容区域，保留必要功能。
  - **安全边界一定要会说**：前端渲染到浏览器里的内容，本质上已经发给用户了，所以“不可复制”并不能真正防泄露。用户仍然可以看源码、抓接口、用 DevTools 拿 DOM、截图 OCR，甚至直接从接口响应里取数据。
  - **回答顺序建议**：先讲 `user-select: none`，再讲 `copy` 事件 + `preventDefault()`，然后补一个“捕获阶段统一拦截”的加分点，最后补充它不是绝对防护，这样答案最完整。

- 高频关键词：不可复制、不可选中、`user-select: none`、`copy` 事件、`preventDefault()`、事件捕获、事件拦截、`document.addEventListener()`、捕获阶段、`selectstart`、`contextmenu`、快捷键拦截、交互限制、安全边界、内容防护
- 易追问：为什么 `copy` 事件可以被拦截？捕获阶段和冒泡阶段这里有什么区别？为什么把监听挂在父节点更合适？`user-select: none` 和 JS 拦截分别解决什么问题？为什么说这不是真正的安全方案？如果页面里有输入框或邀请码，怎么避免误伤正常复制？
- 回答雷区：不要把 `user-select: none` 说成“彻底禁止复制”，它只是不让用户方便选中；不要只写 JS 拦截而不提 `preventDefault()` 是关键；不要忽略事件捕获这个场景里的价值；也不要前端防复制吹成安全能力，只要内容已经到浏览器，理论上就总有办法被拿到。

## 8. 前端如何进行代码质量检测？

- 一句话结论：前端代码质量检测不是只配一个 `ESLint`，而是要把静态检查、类型约束、自动化测试、提交门禁、CI 和线上监控串成一条闭环，分别兜住规范问题、潜在 Bug、功能回归和线上异常。
- 30秒回答：我一般把前端代码质量分成五层。第一层是静态检查，用 `ESLint`、`Prettier`、`Stylelint` 统一代码规范，用 `TypeScript` 做类型安全和编译期兜底。第二层是功能测试，对核心逻辑做单元测试，对组件做组件测试，对关键链路做集成测试。第三层是工程门禁，用 `husky`、`lint-staged`、`commitlint` 在提交前拦住明显问题。第四层是 CI，在合并前自动跑 `lint`、`type-check`、`test`、`build`，保证主分支稳定。第五层是线上监控，用 `window.onerror`、`unhandledrejection` 这类能力补充运行时异常、弱网和兼容性问题。
- 1分钟展开：
  - 先做静态质量兜底：`ESLint` 主要抓代码规范和潜在语法问题，比如未使用变量、危险写法、Hooks 依赖缺失；`Prettier` 统一格式，减少无意义 diff；样式层可以配 `Stylelint`，避免 CSS 命名和书写混乱。
  - 类型安全要尽量前置：如果项目用了 `TypeScript`，我会把它当作代码质量体系的一部分，而不是只当开发体验工具。像 `strict`、`noImplicitAny` 这类配置能把很多参数类型错误、空值问题、返回值不一致的问题提前拦在编译阶段。
  - 功能正确要靠测试验证：静态检查只能发现一部分问题，功能是否真的正确还得靠测试。通常核心工具函数做单元测试，业务组件做组件测试，登录、下单、表单提交流程这类核心链路做集成测试，重点覆盖高风险路径而不是盲目追求百分百覆盖率。

- 提交前就要设门禁：工程里可以用 `husky` 在 `pre-commit` 或 `pre-push` 阶段执行 `lint`、`type-check`、关键测试，用 `lint-staged` 只检查本次改动文件，提高速度；再用 `commitlint` 约束提交信息，方便后续追踪和自动化发布。
- CI 保证主分支质量：本地门禁只能兜住一部分，真正防止问题进主干还得靠 CI。一般会在 Pull Request 或 Merge Request 阶段自动执行 `lint`、类型检查、测试、构建，有条件的话再补关键 E2E 或兼容性验证，确保代码合并前是可运行、可发布的。
- 线上还要补运行时监控：很多问题本地和测试环境不一定能复现，比如弱网、浏览器兼容性、用户真实设备差异、第三方脚本异常。这时候要用 `window.onerror`、`unhandledrejection`、资源加载错误监听、日志上报把异常收集回来，必要时再结合性能指标、接口失败率、浏览器版本分布做持续监测。
- 面试里最稳的表达顺序：可以按“静态检查 -> 类型安全 -> 测试 -> 提交门禁 -> CI -> 线上监控”的顺序回答，这样既能体现你理解工程体系，也能说明你知道代码质量不是单点工具，而是完整流程。
- 高频关键词：代码质量、静态检查、代码规范、潜在 Bug、`ESLint`、`Prettier`、`Stylelint`、`TypeScript`、类型安全、`strict`、单元测试、组件测试、集成测试、工程质量、`husky`、`lint-staged`、`commitlint`、提交门禁、CI、`lint`、`type-check`、`test`、`build`、主分支保护、运行时监控、`window.onerror`、`unhandledrejection`、弱网、浏览器兼容性、日志上报
- 易追问：为什么只配 `ESLint` 还不够？`TypeScript` 在代码质量里具体解决什么问题？单元测试、组件测试、集成测试怎么划分？`husky`、`lint-staged`、`commitlint` 分别负责什么？为什么还需要 CI，不能只靠本地提交检查吗？线上监控和测试的边界是什么？弱网和兼容性问题为什么经常要到线上才暴露？
- 回答雷区：不要把代码质量检测简单等同于“配了 `ESLint` 就够了”，这样答案会非常单薄；不要只讲规范不讲功能正确和工程流程；不要把测试理解成只看覆盖率数字，核心是覆盖关键路径和高风险场景；不要忽略 CI 和主分支保护，否则质量体系是不闭环的；也不要只盯开发阶段，真实用户环境下的弱网、兼容性和运行时异常同样属于代码质量问题。

## 9. 临时 URL 错误配置成 301 后如何补救？

- 一句话结论：临时 URL 如果误配成 `301`，补救的核心不是前端改个地址就完了，而是先让服务端立刻修正错误跳转，再处理浏览器缓存和搜索引擎收录，因为 `301` 会被当成永久迁移长期记住。

- 30秒回答：如果临时地址被错误返回成 **301**，第一步一定是服务端马上改回正确响应，避免新的错误缓存继续扩散。第二步要处理已经被缓存的用户端，因为很多浏览器会记住这个永久跳转，旧 URL 可能短期内根本到不了服务端，这时可以提示用户清缓存或强刷，也可以临时给用户一个带查询参数的新地址绕过已缓存的旧跳转，或者在错误跳转的新地址上再做一次引导，把用户带回正确页面。第三步如果搜索引擎已经收录了错误新地址，还要更新 **sitemap**、修正 **canonical** 和站内链接，并去搜索平台重新提交抓取。
- 1分钟展开：
  - 先止损，别让错误继续放大：**301** 是永久重定向，一旦服务端配错，浏览器、CDN、代理层、搜索引擎都可能记住。第一件事不是让前端兜，而是立刻把服务端改回正确返回，至少别再让新的用户继续拿到错误的永久跳转。
  - 要认识到前端修复有边界：如果浏览器已经缓存了旧 URL 到错误新 URL 的 **301**，用户再次访问旧地址时，请求可能在到达前端之前就已经被浏览器改道了。所以这类问题本质上不能只靠前端页面代码修复，必须服务端和运维一起处理。
  - 对已缓存用户的常见补救方式：最直接的是提示用户强刷或清理浏览器缓存；如果业务很急，可以临时给一个带查询参数的地址，比如在原路径后加 **?v=2** 或其他标识，因为浏览器通常会把它当成一个新 URL，从而绕开旧地址上的 **301** 缓存。
  - 在错误目标页做二次引导：如果已经有大量用户被跳到错误的新 URL，可以先保证这个新页面还能打开，然后在这个页面上增加明显的二次跳转、提示条或按钮，把用户再带回正确地址。这属于兜底方案，虽然不如直接修缓存彻底，但能先保住业务可达性。
  - SEO 修复不能漏：如果搜索引擎已经把错误新 URL 当成正式地址收录，需要尽快更新 **sitemap.xml**、修正 **canonical**、恢复正确内链，并去 Google Search Console、Bing Webmaster Tools 之类的平台重新提交抓取，必要时对错误地址做移除或重新索引申请，尽量把权重和收录引回正确 URL。
  - 后续要补预防措施：这类事故一般说明跳转策略缺少审核。后面最好把 **301/302/307/308** 的使用边界写清楚，在网关、Nginx、CDN 或应用层改跳转规则时加灰度、回滚预案和监控，避免把临时跳转误配成永久跳转。
  - 面试里最稳的回答顺序：可以按“先改服务端止损 -> 再处理已缓存用户 -> 再修 SEO -> 最后补预防机制”的顺序讲，逻辑会比较完整。
- 高频关键词：**301**、永久重定向、临时 URL、错误跳转、服务端修复、浏览器缓存、强刷、清缓存、查询参数绕过、二次引导、错误目标页兜底、SEO、**sitemap.xml**、**canonical**、站内链接、重新抓取、Search Console、Bing Webmaster Tools、收录修复、跳转策略、灰度发布
- 易追问：为什么 **301** 事故不能只靠前端修？**301** 和 **302**、**307**、**308** 的差别是什么？为什么加查询参数有时能绕过问题？如果用户量很大，怎么降低已经缓存的影响？搜索引擎已经收录错误 URL 后会带来什么后果？怎样避免以后再把临时跳转误配成永久跳转？

- 回答雷区：不要把这个问题的说成“改一下前端路由就好了”，很多时候请求在到前端之前就被浏览器按 301 改道了；不要只讲让用户清缓存，这只是补救手段，不是根因修复；不要忽略搜索引擎收录和权重迁移问题，SEO 场景里影响可能比用户侧更久；也不要因为加查询参数理解成恢复旧 URL，本质上它是用一个新 URL 先绕开旧缓存。

## 10. 如何限制 input 框只能输入正整数？

- 一句话结论：限制 `input` 只能输入正整数，不能只依赖 `type="number"`，更稳的做法是 HTML 先做基础约束，再用 JS 在输入阶段过滤非法字符，最后在提交时再做一次校验。
- 30秒回答：最基础的做法是在 HTML 上写 `type="number"`，再配合 `step="1"` 和 `min="0"` 或 `min="1"` 做范围限制，这样浏览器层面会提供一些基础校验。但它不够严，因为很多浏览器里 `number` 仍然可能输入 `e`、负号、小数点，或者通过粘贴塞进非法内容。所以更稳的是再监听 `input` 事件，用正则把非数字内容过滤掉，比如只保留 `0-9`，并按业务决定要不要允许 `0`。最后表单提交前还要再校验一次，避免只靠前端输入框限制。
- 1分钟展开：
  - **HTML 约束先兜底**：可以先写 `type="number"`、`step="1"`，如果不允许负数再配 `min="0"` 或 `min="1"`。这样浏览器会在交互和原生校验层面给一些基础限制，适合做第一层防线。
  - 但 `type="number"` 不是绝对可靠：它的目标更偏“数值输入”，不等于“只能输入正整数”。实际里常见问题是还能输入 `e`、`E`、`-`、`.`，不同浏览器行为也不完全一致，所以面试里一定要强调它只能做基础约束，不能单独当完整方案。
  - **JS 监听 `input` 做实时过滤**：更稳的是监听 `input` 事件，把当前值里的非数字去掉，比如用正则 `value.replace(/\D/g, '')`。如果业务要求必须是正整数且不能为 0，还可以在失焦或提交时继续判断是否大于 0。
  - **粘贴和输入法场景也要考虑**：很多非法值不是键盘直接敲进去的，而是通过粘贴、自动填充或脚本赋值进入，所以只拦键盘事件不够，`input` 事件更适合做统一兜底。
  - **不要只做输入时限制，还要做提交校验**：输入框层面的限制主要是交互优化，真正的数据正确性还是要在表单提交前再校验一遍，必要时后端也要校验。因为用户可以绕过前端约束，或者值可能在别的流程里被改掉。
  - **边界要说清楚**：如果业务允许 `0`，那其实是“非负整数”；如果必须大于 `0`，才是真正的“正整数”。面试里最好把 `min="0"` 和 `min="1"` 的差异说出来，显得你边界意识比较清楚。
  - **回答顺序建议**：可以按“HTML 基础约束 -> JS `input` 实时过滤 -> 提交前再次校验 -> 后端兜底”的顺序讲，会比较完整。

- 高频关键词：正整数、`input`、`type="number"`、`step="1"`、`min="0"`、`min="1"`、`input` 事件、正则表达式、实时过滤、`\D`、`replace()`、粘贴、自动填充、表单校验、提交校验、非负整数、前后端双重校验
- 易追问：为什么 `type="number"` 还不够？为什么很多浏览器里还能输入 `e`、负号和小数点？监听 `keydown` 和监听 `input` 有什么区别？如果不允许输入 `0` 应该怎么处理？粘贴非法内容怎么拦？为什么前端限制了还要后端再校验？
- 回答雷区：不要把 `type="number"` 说成“就能彻底限制正整数”，它只能提供基础能力；不要把 `min="0"` 和“正整数”直接画等号，`0` 严格来说不是正整数；不要只拦键盘输入却忽略粘贴和脚本赋值；也不要只做输入框限制不做提交校验，真实业务里这样不够稳。

## 11. 静态资源预加载的方法有哪些？

- 一句话结论：前端静态资源预加载常见有 `preload`、`prefetch`、`modulepreload` 和 `Service Worker` 预缓存四类，它们的核心区别在于“给当前页面加速”还是“为未来访问提前准备”，以及浏览器调度优先级不同。
- 30秒回答：如果是当前页面很快就要用到的关键资源，一般用 `<link rel="preload" as="script/style/image/font">` 提前加载；如果是用户后续可能访问页面的资源，通常用 `prefetch`，它优先级更低，浏览器会在相对空闲时去拉取。ES Module 场景下可以用 `modulepreload`，它比普通 `preload` 更适合模块依赖预加载。再往上一层，如果项目接入了 `Service Worker`，还可以在 `install` 阶段把静态资源预缓存，后续访问直接从缓存里命中。面试里最好强调：这些方案不是互相替代，而是各自解决不同阶段的加载优化问题。
- 1分钟展开：
  - `preload` 适合当前页面关键资源：比如首屏马上要执行的 JS、关键 CSS、首屏图片、字体文件，可以通过 `<link rel="preload" ...>` 提前告诉浏览器“这个资源很快会用到”。它更偏当前导航的性能优化，优先级通常比 `prefetch` 高。
  - `as` 属性要写对：`preload` 里要写 `as="script"`、`style`、`image`、`font` 这类正确资源类型，浏览器才能做更合理的调度和缓存复用。面试里顺手提一下，标签上是 `rel="preload"`，不是 `ref`。
  - `prefetch` 更偏未来资源预取：它适合“用户下一步大概率会访问”的资源，比如下一个路由页面的 JS chunk、列表页点进详情页会用到的数据或图片。它通常是低优先级请求，浏览器会在网络和主线程相对空闲时去拉，不适合拿来救当前首屏。

- `modulepreload` 是给 ESM 场景准备的：如果项目是 ES Module 体系，`<link rel="modulepreload">` 可以提前加载模块以及相关依赖，比把模块当普通脚本 `preload` 更贴近现代构建产物的加载方式，Vite 这类工具链里会比较常见。
- **Service Worker** 更像“预缓存”：它不是单纯告诉浏览器早点发请求，而是在 Service Worker 的 `install` 阶段把静态资源提前缓存下来。这样后续页面访问、离线访问或弱网场景下都更稳，适合做 PWA、离线兜底和重复访问优化。
- 几个方案的边界要分清：`preload` 用多了会和真正关键资源抢带宽；`prefetch` 可能被浏览器忽略，尤其在弱网、省流模式或资源紧张时；Service Worker 首次访问前本身也需要注册和安装，所以它不一定能直接优化第一次打开页面。
- 回答顺序建议：最稳的是“`preload` 解决当前页面关键资源 -> `prefetch` 解决未来页面资源 -> `modulepreload` 解决模块依赖预加载 -> Service Worker 解决预缓存和离线兜底”，这样层次会很清楚。
- 高频关键词：静态资源预加载、`preload`、`prefetch`、`modulepreload`、Service Worker、预缓存、`install`、PWA、首屏优化、关键资源、未来页面资源、低优先级请求、浏览器空闲、ES Module、依赖图、`as`、路由分包、缓存命中、离线访问、弱网优化
- 易追问：`preload` 和 `prefetch` 的区别是什么？为什么 `prefetch` 不能拿来优化当前首屏？`modulepreload` 和普通 `preload` 的差别是什么？Service Worker 预缓存为什么不一定能优化第一次访问？`preload` 的 `as` 写错会有什么影响？为什么不能把所有资源都做 `preload`？
- 回答雷区：不要把 `preload`、`prefetch`、Service Worker 混成同一种机制，它们的时机和优先级都不一样；不要说 `prefetch` 一定会在 CPU 空闲时执行，更准确地说它通常是在浏览器认为合适、资源相对空闲时低优先级发起；不要把 Service Worker 说成“首次打开就一定生效”，它往往对后续访问更明显；也不要忽略 `rel="preload"` 和 `as` 这两个关键配置，否则答案会显得不够扎实。

## 12. 如何判断用户设备？

- 一句话结论：判断用户设备通常有两类思路，一类是通过 `navigator.userAgent` 这类信息做设备类型识别，另一类是通过视口大小和媒体查询做页面适配；前者更像“猜设备”，后者更像“适配当前展示环境”。
- 30秒回答：前端判断用户设备，最常见的是看 `navigator.userAgent`，它会返回一个字符串，可以通过正则去判断是不是移动端，比如匹配 `Android`、`iPhone` 这类关键字。但 `userAgent` 不是绝对可靠，因为它可能被伪装，也不适合直接等同于页面适配。真正做响应式布局时，更常用的是看视口宽度，比如 `window.innerWidth`、`matchMedia()`，或者直接用 CSS 媒体查询。面试里最好讲成两层：识别设备类型可以看 `userAgent`，做界面适配更推荐视口和媒体查询。

- 1分钟展开:

- `navigator.userAgent` 是最传统的识别方式: 它会返回浏览器和设备相关字符串, 常见写法是用正则判断移动端关键字, 比如 `/Android|iPhone|iPad|Mobile/i.test(navigator.userAgent)`。这种方式实现简单, 很多老项目和统计脚本里都还在用。
- 但 `userAgent` 只能当经验判断: 它本质上是客户端上报的一段字符串, 可能被伪装、兼容模式修改, 也不完全代表真实设备能力。所以它适合做粗粒度设备分类, 不适合当唯一依据。
- 视口大小更适合做布局适配: 如果你的目标是决定走移动布局还是桌面布局, 通常直接看当前视口更合理, 比如 `window.innerWidth`、`document.documentElement.clientWidth`, 或者 JS 里用 `window.matchMedia('(max-width: 768px)')` 判断当前断点。
- 媒体查询是 CSS 层的主力方案: 像 `@media (max-width: 768px)` 这种写法, 本质上不是在识别“用户到底是什么设备”, 而是在根据当前屏幕条件切换样式。对响应式页面来说, 它往往比硬判断“这是手机还是电脑”更稳。
- 要分清“设备识别”和“能力适配”: 有些场景真的是要区分设备类型, 比如埋点统计、下载页跳 App、展示 iOS/Android 不同按钮; 但如果只是做页面展示, 优先应该根据视口、输入方式、触摸能力等条件适配, 而不是死盯设备名称。
- 实际项目里通常是组合使用: 比如业务上先用 `userAgent` 粗分移动端和桌面端, 再用媒体查询做布局细化; 或者根本不判断设备型号, 只做响应式和能力检测, 这种方案维护成本往往更低。
- 回答顺序建议: 最稳的表达是“先讲 `userAgent` 做设备识别, 再讲视口和媒体查询做页面适配, 最后补一句 `userAgent` 不够可靠、适配优先看环境而不是设备名”。这样会更完整。
- 高频关键词: 用户设备判断、`navigator.userAgent`、UA 字符串、正则匹配、移动端识别、`Android`、`iPhone`、`iPad`、`Mobile`、视口、`window.innerWidth`、`clientwidth`、`matchMedia()`、媒体查询、`@media`、响应式布局、设备识别、页面适配、能力检测
- 易追问: 为什么 `userAgent` 不能完全信任? 为什么做页面适配时更推荐媒体查询而不是判断手机型号? `window.innerWidth` 和媒体查询分别适合什么场景? 平板该算移动端还是桌面端? 横屏和缩放会不会影响判断结果? 如果用户用电脑浏览器模拟移动端, `userAgent` 和视口结果会有什么差异?
- 回答雷区: 不要把 `userAgent` 说成设备判断的绝对标准, 它只能算一种经验识别; 不要把“判断设备类型”和“做响应式适配”混成同一件事, 这两者目标不同; 不要只靠视口宽度就断言用户一定是手机, 因为窗口缩放、平板和桌面窄屏都会干扰结果; 也不要只会写一个 `/Mobile/` 正则却说不清为什么真实项目里通常要组合多种条件判断。

## 13. 页面关闭时有哪些执行方法？

- 一句话结论：页面关闭或离开时，传统上可以监听 `beforeunload` 和 `unload`，但现代前端更稳的做法通常是把“离开提醒”和“离开上报”分开处理，前者看 `beforeunload`，后者更推荐 `pagehide`、`visibilitychange` 配合 `navigator.sendBeacon()`。
- 30秒回答：如果想在用户关闭页面前做提醒，可以用 `beforeunload` 事件，比如表单没保存时提示用户确认离开；如果想在页面卸载时做清理，以前会提 `unload`，但它现在并不稳定，而且会影响浏览器优化，所以不适合拿来做关键逻辑。实际项目里，如果只是上报埋点、停留时长、退出日志，更常用的是 `pagehide` 或 `visibilitychange`，再配合 `navigator.sendBeacon()` 做离开时数据发送。面试里最好讲清楚：`beforeunload` 适合提示，`unload` 是老方案，真正的离开上报要考虑更现代的替代方案。
- 1分钟展开：
  - `beforeunload` 主要用来做离开确认：它会在页面即将被刷新、关闭或跳转时触发，典型场景是用户编辑了表单但还没保存，这时可以提示“你有未保存内容，确认离开吗”。它更偏交互提醒，而不是执行复杂业务。
  - `beforeunload` 有使用限制：现代浏览器不会允许你自定义很丰富的弹窗文案，很多情况下只会展示浏览器默认提示；而且通常要有用户交互背景才更容易触发，所以它不能当成完全可靠的流程控制手段。
  - `unload` 是传统卸载事件：它会在页面真正卸载时触发，过去有人会在这里做资源清理、日志发送、状态持久化。但它的可靠性越来越差，而且和浏览器的性能优化、BFCache 机制并不友好，所以现在不建议把关键逻辑压在 `unload` 上。
  - 离开上报更推荐 `pagehide` 和 `visibilitychange`：`pagehide` 更适合感知页面被放入后台、离开当前文档或进入 BFCache；`visibilitychange` 则能在页面变成隐藏状态时触发。很多移动端切后台、切应用的场景里，它们比 `unload` 更实用。
  - 上报数据更推荐 `navigator.sendBeacon()`：如果页面关闭时还要发埋点、停留时长、退出日志，用普通异步请求可能来不及发完；`sendBeacon()` 更适合这种“页面要离开了但还想尽量把数据发出去”的场景。
  - 要分清“提醒用户”和“收尾上报”：提醒离开是 `beforeunload` 的典型职责；而发送日志、保存轻量状态、统计停留时长，更适合放在 `pagehide`、`visibilitychange` 和 `sendBeacon()` 这条链路里，不要混着用。
  - 回答顺序建议：最稳的是“先讲 `beforeunload` 做离开提醒，再讲 `unload` 是老方案，最后补充现代项目更推荐 `pagehide/visibilitychange` + `sendBeacon()`”，这样答案既覆盖基础，也更贴近实际开发。

- 高频关键词：页面关闭、页面离开、`beforeunload`、`unload`、`pagehide`、`visibilitychange`、页面隐藏、BFCache、离开提醒、未保存表单、退出埋点、停留时长、日志上报、`navigator.sendBeacon()`、页面卸载、收尾逻辑、浏览器限制
- 易追问：`beforeunload` 和 `unload` 的区别是什么？为什么 `unload` 现在不推荐做关键逻辑？为什么页面关闭上报更适合 `sendBeacon()`？`pagehide` 和 `visibilitychange` 分别适合什么场景？为什么 `beforeunload` 不能完全依赖？如果用户切到后台但没真正关闭页面，应该监听哪个事件？
- 回答雷区：不要把 `unload` 说成现代项目里最稳的方案，它的可靠性和兼容性边界都比较差；不要把 `beforeunload` 理解成可以随意弹自定义确认框，现代浏览器限制很多；不要在页面离开时还指望普通异步请求一定发得完；也不要把“页面关闭提醒”和“离开日志上报”混成一个方案，它们对应的事件选择并不一样。